

UKACM AUTUMN SCHOOL 2026

Fracture, Differentiability and PhAST

Lecture notes and worked notebooks

Day 3 · 16 September 2026

Allamaprabhu Ani
Sathiskumar A. Ponnusami

CEMS-Lab
City St George's, University of London
Queen Mary University of London

Print edition · 17 September 2026

Contents

Preface	1
Reading code and results	1
Use and distribution	1
Lecture 1: Phase-field fracture and PhAST	2
From a crack to a damage field	2
Energy and stiffness degradation	2
One numerical increment	3
Tensors, matrix-free actions and dynamics	3
Connect the model to the practical	3
NB1 — Intro to PhAST	4
Phase-field fracture - notched square plate	4
A PhAST laboratory for Google Colab	4
Learning objectives	4
Software	5
1. Prepare the Colab environment	5
2. The physics of the model	7
3. A parametric specimen geometry	7
4. Generate the mesh and recover the named regions	10
5. Material and the stable time step	11
6. Loading, boundary conditions and the solver route	13
7. The staggered scheme, one step at a time	18
7b. Run the simulation	20
8. A reusable post-processing toolkit	21
9. Final state	26
10. Animate the four fields	30
11. Energy and crack-tip histories	32
Key takeaways	33
12. Exercises	34
Lecture 2: Differentiability and inverse recovery	40
1. State the experiment	40
2. Build and solve the forward problem with PhAST	40
3. Differentiate a scalar mismatch	40
4. Let the optimiser choose the next modulus	41
5. Interpret and check the result	41
6. Extend the same chain rule to fracture	41
NB2 — Inverse Problem using PhAST	42
Recovering Young's modulus from measured extension	42
A differentiable finite-element exercise with PhAST	42
1. Load PhAST and tools	42
2. Geometry and material setup	43
3. Generate the line mesh	44
4. Boundary Condition and Loads	45
5. Write the forward function	46
6. Record and visualise the forward solution	47
7. Define the observation and inverse question	49
8. Differentiate through the PhAST solve	51
9. Set up a positive modulus and momentum updates	53
10. Run the inverse loop	53

11. Read the recovery history	54
12. Check the recovered solution and save the results	56
13. Recap:	58
Key takeaways	59
Lecture 3: Learned damage updates in PhAST	60
The same physical problem, two damage updates	60
Read one load increment	60
From a mesh to a graph	61
Two comparisons answer different questions	61
Interpret performance without losing accuracy	61
NB3 — Hybrid FEM+DL with PhAST	62
PhAST laboratory: fracture around three holes	62
Replacing the damage subproblem with a trained network	62
Learning objectives	62
2. Load the source and frozen weights	63
3. Define the geometry and generate the mesh	65
4. Specify the material	67
5. Apply boundary conditions and loading	67
6. Choose the solution procedure	69
7. Read the solver loop	70
8. Run conventional FEM	72
9. Examine the FEM evolution	73
10. Load the frozen graph neural network	75
11. Run FEM mechanics with GNN damage prediction	77
12. Examine the learned simulation	78
13. Compare fields at matched loads	79
14. Compare reactions and equation residuals	80
15. Measure cost over the two loading paths	82
16. Benchmark identical damage-subproblem inputs	83
17. Interpret the measured comparison	84
18. Save the calculation	86
19. Record your Colab result	86
20. Download your results	87
Key takeaways	88
Consolidation	88
Crack Representations: Sharp, Cohesive, and Diffuse	89
How Should We Represent a Crack?	89
The Starting Point: A Sharp Crack	90
Cohesive-zone models: separation on an interface	90
XFEM: enrich an approximation space	90
Phase field: replace a surface by a narrow band	91
Comparing Crack Representations	91
Separating Formulation, Discretization, and Solver	92
Exercise: Identifying the Computational Layers	92
Summary of Key Concepts	92
Fracture Energy: AT1, AT2, Degradation, and Bandwidth	93
The Energy Balance: Why Do Materials Break?	93
The Phase-Field Energy Functional	94
The damage convention and degradation derivative	94
Hands-On Lab: Verifying the Degradation Derivative	94
AT2 and AT1 are normalised choices	95
Deriving the strong-form damage residual	95
Enforcing irreversibility	96
Width, mesh, and what must be checked	96
Exercise: one derivative and one scale argument	96

Sources for this chapter	97
The Staggered Solution Algorithm: Alternating Mechanics and Damage	98
Alternating the coupled fields	98
The Mechanical Subproblem	99
Why the damage equation resembles diffusion	99
One load increment, two coupled updates	100
Staggered, monolithic, Newton, and quasi-Newton	100
Static and dynamic fracture	100
Interpreting convergence	101
Applying the equations in PhAST	101
Exercise: write the update before reading the code	101
Sources for this chapter	101
From Weak Forms to Discrete Tensors: The PhAST Pipeline	102
Expressing finite elements with tensors	102
Defining the Boundary-Value Problem	103
From cells to fields	103
Tensor dimensions and indexing	103
Assembled and matrix-free routes	104
Static and dynamic arrays	104
Where automatic differentiation fits	104
Inspecting results: four views, one interpretation	105
Hands-On Lab: Running an End-to-End PhAST Fracture Simulation	105
Exercise: identify a silent shape error	105
Take-away	106
Differentiable Mechanics and Inverse Parameter Discovery	107
Estimating a material property from measurements	107
Formulating the Forward and Inverse Maps	108
Three ways to obtain a sensitivity	108
Follow the gradient one update at a time	109
A bounded parameterisation	113
A minimal inverse experiment	113
Hands-On Lab: Differentiating Mechanics and Parameter Recovery	113
Directional Derivative Verification	114
Why inverse recovery can be difficult	114
Exercise: interpret a gradient result	114
Sources for this chapter	115
Learning field predictions and combining them with a solver	116
Neural predictions within a mechanics calculation	117
Decide What is Being Learned	117
Define the supervised-learning problem	117
Where does the training gradient travel?	118
Train, save, and reload reproducibly	118
Computational lesson: train, save, reload, and compare a toy field model	118
Mapping between representations	119
From a proposal to a checked hybrid calculation	119
Computational lesson: accept a compatible proposal or fall back	119
Dagger-style data aggregation	120
Evaluation: Physical Metrics and Model Validation	120
Exercise: describe the information needed to reuse a model	120
Sources for this chapter	121
Practice, Solutions, Glossary, and References	122
Connecting the four topics	122
Best Practices for Computational Exploration	122
Three classroom notebooks	122

Continue with the PhAST documentation	123
Worked consolidation exercise	123
Glossary	124
Reference trail	124

INTERNAL DOCUMENTATION
NOT TO BE DISTRIBUTED

Preface

UKACM Autumn School 2026 · Day 3 · 16 September 2026

Prepared by [Allamaprabhu Ani](#) and [Sathiskumar A. Ponnusami](#), CEMS-Lab, City St George's, University of London; Queen Mary University of London.

This book supports the Day 3 sessions on fracture, differentiability and PhAST. It brings together three lecture guides, worked practical notebooks and mathematical reference chapters for course participants, including those with limited internet access. Basic mechanics, calculus and introductory Python are the prerequisites. Read each lecture guide before its notebook and consult the reference chapters for the detailed equations.

The practicals progress from dynamic fracture simulation to recovery of a bar's Young's modulus and direct learned replacement of a damage update. Their purpose is to connect physical assumptions, numerical operations and the interpretation of computed fields and gradients. The bar is linear elastic and contains no damage. The hybrid example uses a different specimen and constitutive model from the dynamic plate.

Reading code and results

Code and figures come from the supplied notebooks and retain their saved results. Preparing this edition involved no new numerical runs. Each result should be read with its stated parameters, software revision and physical assumptions. Independent exercises are identified separately from worked results.

Text, equations, code and saved plots can be read offline. Animations appear as selected frames; their links open the complete online notebooks. Internet access is needed to play those animations or use Colab. Long setup logs are abbreviated; result tables, hints and conceptual answers are retained.

The notation uses $d = 0$ for intact material and $d = 1$ for fully damaged material. Numerical time increments describe the model; measured wall-clock times in the hybrid example describe computational cost. They are different quantities.

[Online book and notebook downloads](#)

Use and distribution

Internal documentation - not to be distributed. This copy is for personal study by course participants. Do not forward, redistribute or republish it without the authors' permission. Copyright © 2026 Allamaprabhu Ani and Sathiskumar A. Ponnusami. All rights in the original course material are reserved; third-party material retains its existing rights and acknowledgements.

Lecture 1: Phase-field fracture and PhAST

This lecture connects the physical description of a crack to the numerical operations used to simulate it. Basic calculus and vectors are sufficient preparation; the finite-element ideas are introduced as they arise.

We follow a notched specimen under tensile loading. The main question is how its deformation, damage field and reaction force change as the load increases. Later lectures ask how these observations depend on material parameters and where a learned model could contribute to the calculation.

From a crack to a damage field

A crack creates new surfaces and changes the way a specimen carries load. Phase-field fracture represents this process with a continuous damage field d : zero denotes intact material and one denotes fully damaged material. The length scale ℓ controls the width of the diffuse transition. The [crack-representation chapter](#) places this description alongside sharp-crack, cohesive and enriched finite-element models.

Reading a diffuse crack profile

Compare a notched specimen, its diffuse damage band and a line profile across the band. Increasing ℓ widens an idealised profile. In a computed fracture field, the material model, loading and mesh also affect the observed profile.

Learning question: Which quantity changes when we widen the diffuse band?

Energy and stiffness degradation

The model balances stored elastic energy, the energy required to create a diffuse crack, and external work. The fracture energy G_c sets the cost of creating crack surface. A degradation function reduces the selected elastic energy as d grows; a small residual stiffness is often retained in the numerical model. Damage irreversibility records the material's loading history.

Use the existing [energy-profile figure](#) to compare AT1 and AT2 profiles and the degradation curve. Their functions, normalisation and derivations are in the [phase-field energy chapter](#).

Interpreting degradation and the length scale

Read the degradation curve at intact, partially damaged and nearly broken states. Then compare a change in G_c with a change in ℓ , keeping the other modelling choices fixed. Distinguish the fracture-energy scale, band width and mesh resolution.

Learning question: How do these choices enter the physical model?

One numerical increment

Finite elements interpolate displacement and damage from values stored at mesh nodes. A staggered calculation updates displacement with damage held fixed, evaluates the driving field, and updates damage with the current mechanics held fixed. A convergence check determines whether to repeat these updates or accept the increment. The [staggered-loop figure](#) shows this ordering; the [chapter](#) gives the governing equations.

i Following the staggered calculation

Follow mechanics, the driving field, damage and convergence in order within one fixed load increment. Advance the load after the increment satisfies its convergence criterion. The accepted damage from the preceding increment provides the irreversibility reference.

Learning question: Which loop repeats at a fixed load, and which step changes the loading?

Tensors, matrix-free actions and dynamics

Coordinates describe where nodes lie; connectivity lists the nodes in each element; field arrays contain their displacement and damage. Gathering local values, evaluating element contributions and adding them back to the global field connects the finite-element model to tensor operations. The existing [FEM pipeline](#) and [tensor chapter](#) develop these operations.

A matrix-free calculation evaluates the effect of a matrix on a vector using element calculations, without assembling the full matrix. Its cost depends on the iterative solver and its preconditioner, which helps the iterations converge. A dynamic calculation also updates velocity and acceleration through physical time, using a chosen time-integration method and time step.

i One element and its tensor operations

Follow one triangle from nodal displacement to strain and a residual contribution, then sketch an operator action using the same local data. Add a final state row for displacement, velocity, acceleration and damage to locate the extra quantities in a dynamic calculation.

Learning question: Which arrays represent geometry, fields and physical time evolution?

Connect the model to the practical

In [../classroom/01 simulate fracture](#), inspect the geometry and loading, run the small PhAST example, and interpret the advancing damage field alongside the loading history and elastic, fracture and kinetic energies. The selected configuration uses a 40 mm square glass plate with a 20 mm geometric notch, explicit dynamics and an implicit spectral AT2 damage update. Students generate a graded Gmsh mesh with target sizes of 0.25 mm near the crack path and 2 mm away from it. Actual mesh counts are printed after generation. Equal and opposite vertical displacements act on the top and bottom; the left and right edges are restrained horizontally. The ramp lasts 20 microseconds within a 50-microsecond simulation window.

The notebook uses `phast.load_result` and `phast.compute_field` to inspect displacement, strain, stress and damage, then animate them with fixed colour scales. Run the horizontal-notch reference first; the inclined-notch exercise is an additional simulation. Quantitative path and speed accuracy require mesh refinement.

Finish by choosing one parameter and one observable that could be compared between runs. [Lecture 2](#) follows this relationship through the computational graph.

NB1 — Intro to PhAST

Phase-field fracture - notched square plate

UKACM Autumn School 2026 · 16 September

Developed by Allamaprabhu Ani and Sathiskumar A. Ponnusami, CEMS-Lab, City St George's, University of London Queen Mary University of London

[PhAST repository](#) · [PhAST documentation](#)

A PhAST laboratory for Google Colab

This notebook develops a dynamic fracture simulation with PhAST, from geometry and material data to animated results. Reusable functions support the parameter studies at the end.

The specimen is a single-edge-notched tension (SENT) plate of soda-lime glass, following the configuration used by Borden et al. (2012). The plate is pulled apart on its top and bottom edges. A diffuse crack grows from the notch tip and crosses the remaining ligament.

Learning objectives

On completing this laboratory you should be able to:

1. Describe the AT2 phase-field model and the role of the length scale ℓ_0 .
2. Construct a parametric specimen geometry and generate a graded mesh.
3. Connect named mesh regions to displacement boundary conditions.
4. Explain why an explicit dynamic route imposes a CFL restriction on the time step.
5. Distinguish the mechanics subproblem from the damage subproblem in a staggered scheme.
6. Reload stored fields through the public result interface and animate them.
7. Design and run your own parameter study, starting from an inclined notch.

The calculation performed here is a compact classroom demonstration. The mesh is coarse by research standards, a single simulation is run for each configuration. Results illustrate the method and the software interface. They do not constitute benchmark validation.

Software

All mechanics and all damage physics are executed inside PhAST ([CEMS-Lab/PhAST](#)). Gmsh generates the mesh. NumPy and Matplotlib derive scalar views of stored fields.

Day 3 · PhAST practical. Updated 16 September 2026. Figures and animations are retained from the supplied notebook. Complete the reference before trying the additional parameter studies.

1. Prepare the Colab environment

PhAST requires Python 3.10 or newer.

The solver is installed straight from the public repository.

The installation step also checks that the `phast` it imports is really this solver, by testing for the interfaces the laboratory uses.

Gmsh is distributed as a Python wheel that links against system OpenGL libraries, so one `apt` package is installed first on Colab.

A CPU runtime is sufficient. Execution time depends on the machine, mesh and selected output settings.

Code cell 4

```
import importlib.util
import subprocess
import sys

try:
    IN_COLAB = importlib.util.find_spec("google.colab") is not None
except ModuleNotFoundError:
    IN_COLAB = False

print("Python:", sys.version.split()[0])
print("Running on Colab:", IN_COLAB)

if IN_COLAB:
    # Gmsh's Python wheel needs libGLU at import time.
    subprocess.run(["apt-get", "-qq", "install", "-y", "libglu1-mesa"], check=True)

PHAST_REPOSITORY = "https://github.com/CEMS-Lab/PhAST.git"
# Pinned to the revision this notebook was last executed against end to end.
# Set to "" to follow the default branch, and record whichever value you used
# alongside any result you report.
PHAST_REF = "d94f6b6869ac609f0411f7a93ec210ef4b8c4a7f" # PhAST 0.16.2
REQUIRED = ("FEMMesh", "load_result", "compute_field", "FIELD_REGISTRY")

requirement = f"phast @ git+{PHAST_REPOSITORY}" + (f"@{PHAST_REF}" if PHAST_REF else "")

def course_solver_present():
    """True only when the importable `phast` really is the course solver.

    Testing for the attributes this laboratory uses, rather than for the name
    alone, stops an unrelated package called `phast` on the import path from
    being silently accepted in place of the pinned revision.
    """
    if importlib.util.find_spec("phast") is None:
        return False
    import phast
    return all(hasattr(phast, name) for name in REQUIRED)
```

(continues on next page)

(continued from previous page)

```

if not course_solver_present():
    print("Installing:", requirement)
    subprocess.run(
        [sys.executable, "-m", "pip", "install", "-q", requirement], check=True)
    importlib.invalidate_caches()

import phast

if not all(hasattr(phast, name) for name in REQUIRED):
    raise RuntimeError(
        "A different package named 'phast' is shadowing the course solver:\n"
        f" {phast.__file__}\n"
        f"Restart the kernel after installing:\n {requirement}")

# Importing PhAST selects the non-interactive Agg backend, which several of its
# batch runners require. Restore the inline backend afterwards so that figures
# and animation players appear in the notebook.
%matplotlib inline

import importlib.metadata
import matplotlib

print("PhAST is importable. Matplotlib backend:", matplotlib.get_backend())
try:
    print("PhAST version:", importlib.metadata.version("phast"))
except importlib.metadata.PackageNotFoundError:
    print("PhAST version: source path, no distribution metadata")
print("PhAST module: ", phast.__file__)

```

```

Python: 3.13.15
Running on Colab: True
Installing: phast @ git+https://github.com/CEMS-
Lab/PhAST.git@d94f6b6869ac609f0411f7a93ec210ef4b8c4a7f
PhAST is importable. Matplotlib backend: module://matplotlib_inline.backend_inline
PhAST version: 0.16.2
PhAST module: /usr/local/lib/python3.13/dist-packages/phast/__init__.py

```

The doctor subcommand reports the resolved environment, including the interpreter, the PyTorch build, the accelerator status and the optional sparse backends. Read it before interpreting any timing.

Code cell 6

```
!{sys.executable} -m phast doctor
```

```

[Long console log abbreviated; see the online notebook for the full output.]
  cudss = NVIDIA cuDSS/nvmath CUDA sparse direct solve
  cg     = matrix-free preconditioned CG

Recommended problem-class defaults:
  explicit dynamics: solver_type=explicit, dt_safety=0.8, damage_every=1 for reference
  validation
  explicit throughput: after validation, damage_every=2 or 3 for subcycling sensitivity runs
  quasi-static fracture: solver_type=quasi_static, backend=auto, preconditioner=jacobi
  spectral/Amor QS on CPU/HPC: install PETSc/MUMPS so backend=auto can select mumps
  cohesive contact: sparse quasi-static backend, backend=auto, normal-contact penalty only
  when configured
  J2 plasticity: sparse quasi-static backend, backend=auto, guarded supported material
  combinations
  dataset/deep learning: Zarr trajectory stores; MP4/raster visualisation when animations are
  requested
  learned damage: classical is default; learned_proposal keeps exact correction;

```

(continues on next page)

(continued from previous page)

```
learned_replacement is audited experimental inference
```

Install guidance:

Start with: `pip install -e .`

Add workflow extras as needed: `.[amg], .[viz-fast], .[dataset], .[hpc]`

For HPC MUMPS: `install petsc petsc4py mumps-mpi from conda-forge`

Then rerun: `python -m phast doctor`

2. The physics of the model

The plate occupies a domain Ω . Two fields are solved for: the displacement $u(x, t)$ and the scalar phase field $d(x, t) \in [0, 1]$, where $d = 0$ denotes intact material and $d = 1$ denotes a fully broken state.

Regularised energy. The AT2 model represents a sharp crack by a diffuse band of width controlled by the length scale ℓ_0 :

$$\Psi = \int_{\Omega} [g(d) \psi^+(\varepsilon) + \psi^-(\varepsilon)] d\Omega + \int_{\Omega} \frac{G_c}{2\ell_0} [d^2 + \ell_0^2 |\nabla d|^2] d\Omega,$$

with degradation function $g(d) = (1 - d)^2 + \eta$ and a small residual stiffness η that keeps the stiffness matrix non-singular.

Energy split. The elastic energy density is separated by a spectral decomposition of the strain into a tensile part ψ^+ , which drives damage, and a compressive part ψ^- , which does not. This prevents cracks from forming under pure compression.

Irreversibility. A history variable enforces monotone damage growth:

$$H(x, t) = \max_{s \leq t} \psi^+(\varepsilon(x, s)).$$

Damage equation. Stationarity of Ψ with respect to d gives a linear elliptic problem for the AT2 model:

$$\frac{G_c}{\ell_0} d - G_c \ell_0 \nabla^2 d + 2H d = 2H.$$

Mechanics. Linear momentum balance is integrated in time with the lumped mass matrix:

$$\rho \ddot{u} = \nabla \cdot \sigma(\varepsilon, d).$$

Staggered solution. Within each time step PhAST advances mechanics, updates the history field, and then solves the damage equation. Mechanics is explicit; the damage subproblem is solved implicitly by a matrix-free conjugate gradient method. These two statements are independent of one another.

Mesh requirement. The diffuse band must be resolved by the mesh. A working rule is $h \leq \ell_0/2$ inside the expected crack path, where h is the element size. The configuration below uses $\ell_0 = 0.5$ mm and a refined element size of 0.25 mm.

3. A parametric specimen geometry

The geometry is written as a Gmsh .geo script by the function below. The notch inclination is a parameter, which allows the same function to serve both the reference horizontal notch and the inclined-notch exercise in Section 12.

Symbol	Meaning	Reference value
L	plate edge length	40 mm
a	notch length measured from the left edge	20 mm
angle_deg	notch inclination above the horizontal	0 deg
h_crack	element size in the refined band	0.25 mm
h_coarse	element size in the far field	2.0 mm
band	half-width of the refined band	1.5 mm
eps	half-opening of the notch mouth	0.001 mm

Six named physical curves are created: top, bottom, left, right, notch_upper and notch_lower. Boundary conditions later refer to these names rather than to coordinate tolerances.

Line 9 runs from the notch tip to the right edge. It carries mesh refinement along the anticipated crack path and is treated as an internal interface between the two plate surfaces, so it imposes no mechanical condition.

Code cell 9

```
import math
from pathlib import Path

def write_sent_geo(path, *, L=40.0, a=20.0, angle_deg=0.0,
                  h_coarse=2.0, h_crack=0.25, band=1.5, eps=1.0e-3):
    """Write a Gmsh .geo file for a square plate with a straight inclined notch.

    Parameters
    -----
    path : str or Path
        Destination of the .geo file.
    L : float
        Plate edge length in mm.
    a : float
        Notch length measured from the left edge in mm.
    angle_deg : float
        Notch inclination above the horizontal, in degrees.
    h_coarse, h_crack : float
        Far-field and refined element sizes in mm.
    band : float
        Half-width of the refined band around the notch and expected path.
    eps : float
        Half-opening of the notch mouth on the left edge in mm.

    Returns
    -----
    Path
        The path that was written.
    """
    theta = math.radians(float(angle_deg))
    cy = L / 2.0
    tip_x = a * math.cos(theta)
    tip_y = cy + a * math.sin(theta)

    geo = f"""// PhAST teaching geometry: square plate with a notch at {angle_deg} deg
L = {L}; a = {a}; h_crack = {h_crack}; h_coarse = {h_coarse}; band = {band};

Point(1) = {{0, 0, 0, h_coarse}}; // bottom-left
Point(2) = {{L, 0, 0, h_coarse}}; // bottom-right
Point(3) = {{L, L, 0, h_coarse}}; // top-right
Point(4) = {{0, L, 0, h_coarse}}; // top-left
Point(5) = {{0, {cy + eps:.9g}, 0, h_crack}}; // notch mouth, upper lip
Point(6) = {{0, {cy - eps:.9g}, 0, h_crack}}; // notch mouth, lower lip
Point(7) = {{tip_x:.9g, tip_y:.9g, 0, h_crack}}; // notch tip
Point(8) = {{L:.9g, tip_y:.9g, 0, h_crack}}; // path exit on right edge

Line(1) = {{1, 2}}; // bottom
Line(2) = {{2, 8}}; // right, below the path exit
Line(3) = {{8, 3}}; // right, above the path exit
Line(4) = {{3, 4}}; // top
Line(5) = {{4, 5}}; // left, above the notch mouth
Line(6) = {{5, 7}}; // upper notch face
Line(7) = {{7, 6}}; // lower notch face
Line(8) = {{6, 1}}; // left, below the notch mouth
Line(9) = {{7, 8}}; // anticipated crack path, used for refinement
```

(continues on next page)

(continued from previous page)

```

Curve Loop(1) = {{1, 2, -9, 7, 8}};
Plane Surface(1) = {{1}};
Curve Loop(2) = {{9, 3, 4, 5, 6}};
Plane Surface(2) = {{2}};

Physical Curve("bottom") = {{1}};
Physical Curve("right") = {{2, 3}};
Physical Curve("top") = {{4}};
Physical Curve("left") = {{5, 8}};
Physical Curve("notch_upper") = {{6}};
Physical Curve("notch_lower") = {{7}};
Physical Surface("plate") = {{1, 2}};

// Graded size field: h_crack near the notch faces and the anticipated path.
Field[1] = Distance;
Field[1].CurvesList = {{6, 7, 9}};
Field[1].Sampling = 100;
Field[2] = Threshold;
Field[2].InField = 1;
Field[2].SizeMin = h_crack;
Field[2].SizeMax = h_coarse;
Field[2].DistMin = band;
Field[2].DistMax = 3*band;
Background Field = 2;
Mesh.CharacteristicLengthExtendFromBoundary = 0;
Mesh.CharacteristicLengthFromPoints = 0;
Mesh.CharacteristicLengthFromCurvature = 0;
Mesh.Algorithm = 6; // Frontal-Delaunay
Mesh.ElementOrder = 1;
"""
    path = Path(path)
    path.parent.mkdir(parents=True, exist_ok=True)
    path.write_text(gio, encoding="utf-8")
    return path

WORK = Path("phast_lab")
WORK.mkdir(exist_ok=True)

gio_file = write_sent_gio(WORK / "reference.gio", angle_deg=0.0)
print(gio_file.read_text()[:600])

```

```

// PhAST teaching geometry: square plate with a notch at 0.0 deg
L = 40.0; a = 20.0; h_crack = 0.25; h_coarse = 2.0; band = 1.5;

Point(1) = {0, 0, 0, h_coarse}; // bottom-left
Point(2) = {L, 0, 0, h_coarse}; // bottom-right
Point(3) = {L, L, 0, h_coarse}; // top-right
Point(4) = {0, L, 0, h_coarse}; // top-left
Point(5) = {0, 20.001, 0, h_crack}; // notch mouth, upper lip
Point(6) = {0, 19.999, 0, h_crack}; // notch mouth, lower lip
Point(7) = {20, 20, 0, h_crack}; // notch tip
Point(8) = {40, 20, 0, h_crack}; // path exit on right

```

4. Generate the mesh and recover the named regions

Gmsh is driven through its Python API, which avoids any dependency on a command line binary being present in the runtime. The mesh is written in the legacy msh2 format, which carries the physical group names that PhAST reads.

phast.FEMMesh takes ownership of the mesh from this point: it stores the connectivity, precomputes the element geometry and the lumped mass, and exposes the named node sets.

Code cell 11

```
import gmsh

def build_mesh(geo_path, msh_path, *, verbose=False):
    """Mesh a .geo file with the Gmsh Python API and write msh2 output."""
    gmsh.initialize()
    try:
        gmsh.option.setNumber("General.Terminal", 1 if verbose else 0)
        gmsh.open(str(geo_path))
        gmsh.model.mesh.generate(2)
        gmsh.option.setNumber("Mesh.MshFileVersion", 2.2)
        gmsh.write(str(msh_path))
    finally:
        gmsh.finalize()
    return Path(msh_path)

mesh_file = build_mesh(geo_file, WORK / "reference.msh")

import phast

mesh = phast.FEMMesh(str(mesh_file), device="cpu")
print(mesh.summary())
print("Named node sets:", sorted(mesh.node_sets.keys()))
```

```
[FEMMesh] Loading mesh: phast_lab/reference.msh

[FEMMesh] Mesh file read OK
[FEMMesh] 3590 nodes loaded
[FEMMesh] 6902 T3 elements loaded
[FEMMesh] Node sets: ['bottom', 'right', 'top', 'left', 'notch_upper', 'notch_lower']
[FEMMesh] Precomputing areas, grad_phi, incircle, lumped mass...
[FEMMesh] Precompute done: h_min=9.827042e-02, total_area=1599.980000
FEMMesh: phast_lab/reference.msh
  3590 nodes, 6902 elements
  h_min=9.827042e-02, total_area=1599.980000
  Node set 'bottom': 21 nodes
  Node set 'right': 39 nodes
  Node set 'top': 21 nodes
  Node set 'left': 40 nodes
  Node set 'notch_upper': 81 nodes
  Node set 'notch_lower': 81 nodes
Named node sets: ['bottom', 'left', 'notch_lower', 'notch_upper', 'right', 'top']
```

The plot below shows the mesh and the four loaded or constrained boundaries. The grading is visible as a refined band along the notch and along the anticipated crack path.

Code cell 13

```
import matplotlib.pyplot as plt
import numpy as np
from matplotlib.tri import Triangulation
```

(continues on next page)

(continued from previous page)

```

nodes = np.asarray(mesh.nodes.cpu(), dtype=float)
elements = np.asarray(mesh.elements.cpu(), dtype=int)
triangulation = Triangulation(nodes[:, 0], nodes[:, 1], elements)

fig, ax = plt.subplots(figsize=(6.4, 6.0))
ax.triplot(triangulation, lw=0.25, color="0.75")
styles = {
    "top": ("336b87", "prescribed u_y = +0.002 mm"),
    "bottom": ("336b87", "prescribed u_y = -0.002 mm"),
    "left": ("7a8288", "u_x fixed"),
    "right": ("7a8288", "u_x fixed"),
    "notch_upper": ("a94f42", "traction-free notch face"),
    "notch_lower": ("a94f42", "traction-free notch face"),
}
seen = set()
for name, (colour, label) in styles.items():
    idx = np.asarray(mesh.node_sets[name])
    shown = label if label not in seen else None
    seen.add(label)
    ax.scatter(nodes[idx, 0], nodes[idx, 1], s=8, color=colour, label=shown, zorder=3)
ax.set_aspect("equal")
ax.set_xlabel("x [mm]")
ax.set_ylabel("y [mm]")
ax.set_title(f"Reference mesh: {mesh.n_nodes} nodes, {mesh.n_elems} T3 elements")
ax.legend(loc="lower left", fontsize=8, framealpha=0.9)
plt.show()

```

5. Material and the stable time step

The material is the soda-lime glass of the reference benchmark. Units are millimetres, newtons, megapascals and tonnes per cubic millimetre, which form a consistent set.

Property	Symbol	Value
Young's modulus	E	32 000 MPa
Poisson's ratio	ν	0.20
Density	ρ	2.45×10^{-9} t/mm ³
Critical energy release rate	G_c	3.0×10^{-3} N/mm
Phase-field length scale	ℓ_0	0.5 mm
Residual stiffness	η	10^{-7}

`phast.Material` resolves the constitutive data and reports the dilatational wave speed c_p . The explicit central-difference integrator is stable only when

$$\Delta t \leq \alpha \frac{h_{\min}}{c_p},$$

with a safety factor α of 0.8. PhAST applies this rule internally from the mesh that is actually supplied; the cell below reproduces the arithmetic so that the cost of refining the mesh is visible.

Code cell 15

```

material = phast.Material(
    E=32000.0,          # Young's modulus [MPa]
    nu=0.20,            # Poisson's ratio
    rho=2.45e-9,        # density [tonne/mm^3]
    Gc=3.0e-3,          # critical energy release rate [N/mm]
    l0=0.5,             # phase-field length scale [mm]

```

(continues on next page)

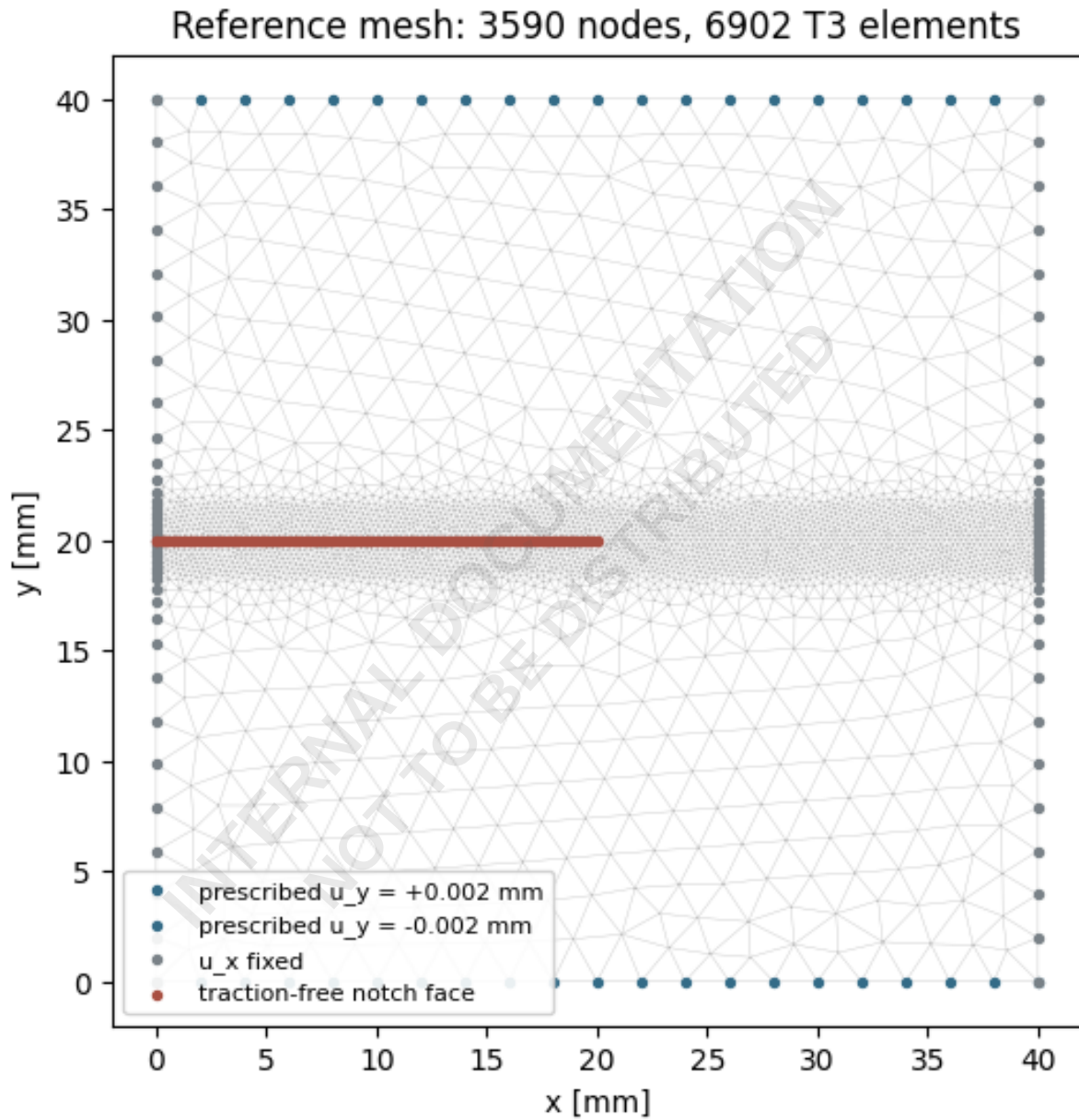


Fig. 1: Saved output from code cell 13.

(continued from previous page)

```

eta_residual=1.0e-7, # residual stiffness
pf_model="AT2",
energy_split="spectral",
plane_stress=False, # plane strain
)

h_min = float(mesh.h_min)
c_p = float(material.c_p)
dt_stable = 0.8 * h_min / c_p

print(f"h_min          = {h_min:.6f} mm")
print(f"c_p            = {c_p:.4e} mm/s")
print(f"stable dt        = {dt_stable:.4e} s")
print(f"l0 / h_min       = {material.l0 / h_min:.2f} (values above 2 resolve the band)")

```

```

h_min          = 0.098270 mm
c_p            = 3.8095e+06 mm/s
stable dt      = 2.0637e-08 s
l0 / h_min     = 5.09 (values above 2 resolve the band)

```

6. Loading, boundary conditions and the solver route

PhAST reads a YAML configuration. The builder below returns that configuration as a Python dictionary so that every exercise can vary one entry and rerun.

Boundary conditions. The top and bottom edges receive equal and opposite prescribed vertical displacements, which produces a symmetric mode-I opening without translating the specimen. The left and right edges are restrained horizontally. The notch faces carry no condition and remain traction-free.

Loading. A smooth-step ramp raises the prescribed displacement to its full amplitude over 20 microseconds and holds it afterwards. A smooth ramp limits the spurious stress waves that a sudden application would introduce.

Route. Two questions are decided separately. Whether inertia is retained is a physical question, answered here by choosing a dynamic analysis. Whether a given subproblem is advanced explicitly or implicitly is a numerical question, answered here by explicit central-difference mechanics combined with an implicit linear solve for the damage field.

Mechanics	Damage	Status in this notebook
Dynamic, explicit central difference	Classical implicit AT2 solve	Used here
Quasi-static equilibrium	Classical implicit damage solve	Supported, separate examples
Dynamic, explicit	Learned proposal or audited replacement	Experimental, see Section 13

Simulated window. The reference run covers 50 microseconds, which is sufficient for the crack to cross the ligament. Set $T_TOTAL = 1.0e-4$ for the longer window used in the published example.

Snapshot cadence. `h5_every` controls how often a full field snapshot is written to the Zarr trajectory store. Every 20th step gives a smooth animation at a modest storage cost.

Code cell 17

```

T_TOTAL = 5.0e-5 # simulated window [s]
SNAPSHOT_EVERY = 20 # store a full field snapshot every N steps

def make_config(mesh_path, *, name="SENT reference", E=32000.0, nu=0.20,

```

(continues on next page)

(continued from previous page)

```

        Gc=3.0e-3, l0=0.5, rho=2.45e-9, eta_residual=1.0e-7,
        u_amp=0.002, t_total=T_TOTAL, t_ramp=2.0e-5,
        dt_safety=0.8, snapshot_every=SNAPSHOT_EVERY):
    """Return a PhAST run configuration for the notched square plate."""
    return {
        "schema_version": 1,
        "problem": {"name": name},
        "geometry": {"units": "mm", "mesh_path": str(mesh_path)},
        "material": {
            "E": E, "nu": nu, "Gc": Gc, "l0": l0, "rho": rho,
            "eta_residual": eta_residual,
            "energy_split": "spectral", "pf_model": "AT2",
        },
        "boundary_conditions": [
            {"nodes": "left", "type": "fix", "component": 0},
            {"nodes": "right", "type": "fix", "component": 0},
            {"nodes": "top", "type": "prescribe", "component": 1, "value": u_amp},
            {"nodes": "bottom", "type": "prescribe", "component": 1, "value": -u_amp},
        ],
        "loading": {
            "protocol": "simple", "ramp_type": "smooth_step",
            "t_ramp": t_ramp, "t_total": t_total,
        },
        "solver": {"solver_type": "explicit", "dt_safety": dt_safety},
        "output": {
            "trajectory": True, "trajectory_format": "zarr",
            "h5_every": snapshot_every, "fast": True, "print_every": 200,
        },
    }

import yaml

config = make_config(mesh_file)
config_file = WORK / "reference.yaml"
config_file.write_text(yaml.safe_dump(config, sort_keys=False), encoding="utf-8")
print(config_file.read_text())

```

```

schema_version: 1
problem:
  name: SENT reference
geometry:
  units: mm
  mesh_path: phast_lab/reference.msh
material:
  E: 32000.0
  nu: 0.2
  Gc: 0.003
  l0: 0.5
  rho: 2.45e-09
  eta_residual: 1.0e-07
  energy_split: spectral
  pf_model: AT2
boundary_conditions:
- nodes: left
  type: fix
  component: 0
- nodes: right
  type: fix
  component: 0
- nodes: top

```

(continues on next page)

(continued from previous page)

```

type: prescribe
component: 1
value: 0.002
- nodes: bottom
  type: prescribe
  component: 1
  value: -0.002
loading:
  protocol: simple
  ramp_type: smooth_step
  t_ramp: 2.0e-05
  t_total: 5.0e-05
solver:
  solver_type: explicit
  dt_safety: 0.8
output:
  trajectory: true
  trajectory_format: zarr
  h5_every: 20
  fast: true
  print_every: 200

```

The precheck subcommand reports wave speeds, the stable increment and mesh quality without advancing the solution. Run it before committing to a long calculation.

Code cell 19

```
!{sys.executable} -m phast precheck --config {config_file}
```

[Long console log abbreviated; see the online notebook for the full output.]

```

GPU (CUDA):    ~0.0 s
(Very rough -- actual time depends on CG iterations, IO, etc.)

```

9. ENERGY SCALES

Fracture parameter scales:

```

Gc * l0 = 0.001500 (stiffness coefficient)
Gc / l0 = 0.0060   (reaction coefficient)
Ratio Gc/l0 : Gc*l0 = 4:1

```

Critical stress for homogeneous nucleation (1D):

```

AT2: sigma_c = sqrt(27*E*Gc/(256*l0)) = 4.50 MPa
AT1: sigma_c = sqrt(3*E*Gc/(8*l0))    = 8.49 MPa

```

```

=====
END OF DIAGNOSTIC REPORT
=====

```

Finding your way around PhAST

The command line interface. `--help` lists the subcommands, and each subcommand takes `--help` of its own. `explain-config` is the most useful of these while you are building a configuration: it resolves and explains a YAML file, reporting the model, the solver route and the loading, *without* generating a mesh or advancing the solution.

Command	What it answers
<code>python -m phast --help</code>	which subcommands exist
<code>python -m phast explain-config <file></code>	what this configuration means
<code>python -m phast precheck --config <file></code>	wave speeds, stable increment, mesh quality
<code>python -m phast doctor</code>	what environment and backends resolved
<code>python -m phast run --help</code>	the options accepted by a run
<code>python -m phast postprocess --help</code>	what can be regenerated from a finished run

Two details are worth noting, because both cost time to rediscover. `precheck` takes its configuration through `--config`, not as a positional argument. And `explain-config` resolves `mesh_path` relative to the directory you run it from, so run it from the notebook's working directory, as below.

The Python objects. Everything imported from `phast` carries a docstring, so `help()` is the reference for the library itself.

You can also check documentation at <https://cems-lab.github.io/PhAST/>

Code cell 21

```
!{sys.executable} -m phast --help
```

Entry point for `python -m phast`.

Subcommands:

```
run          Run a simulation from a YAML config
precheck     Pre-simulation diagnostics (wave speeds, CFL, mesh quality)
explain-config Explain a YAML config without generating meshes or running
schema       Export the JSON Schema for YAML configs
doctor       Report environment and solver backend status
postprocess  Generate plots from a completed run directory
new          Scaffold a starter YAML config for a new benchmark
```

Usage:

```
python -m phast run configs/benchmarks/dynamic/B2_kalthoff_winkler.yaml
python -m phast precheck configs/benchmarks/dynamic/B2_kalthoff_winkler.yaml
python -m phast explain-config configs/benchmarks/dynamic/B3_dynamic_sent.yaml
python -m phast schema --output configs/phast.schema.json
python -m phast doctor
python -m phast postprocess path/to/run_dir --dpi 300
python -m phast new my_benchmark --type quasi_static --material pmma_bleier
```

Available subcommands: run, precheck, explain-config, schema, doctor, postprocess, new

Code cell 22

```
!{sys.executable} -m phast explain-config {config_file}
```

Config: `phast_lab/reference.yaml`

Schema version: 1

Problem: SENT reference

Geometry

source: external mesh (`phast_lab/reference.msh`)

(continues on next page)

(continued from previous page)

```

Material And Fracture Model
  preset: inline properties
  phase-field model: AT2
  energy split: spectral
  E=32000.0, nu=0.2, Gc=0.003, l0=0.5

Solver
  type: explicit
  backend: auto
  preconditioner: default
  damage enabled: yes
  bounds method: post_clamp
  history update: hard_max
  stagger criterion: relative (tol=1e-06, max=500)

Loading
  protocol: simple
  steps: 0
  explicit time: dt=1e-05, t_total=5e-05, ramp=smooth_step, t_ramp=2e-05

Boundary Conditions
  1. fix on left component=0 value=0.0
  2. fix on right component=0 value=0.0
  3. prescribe on top component=1 value=0.002
  4. prescribe on bottom component=1 value=-0.002

Outputs
  output_dir: <auto>
  enabled: zarr trajectory snapshots every 20 steps

Device
  device: auto
  torch.compile: no

Warnings
  none

```

Code cell 23

```
help(phast.Material)
```

```

[Long console log abbreviated; see the online notebook for the full output.]
| rho = 7.8e-09
|
| sigma_ts = 0.0
|
| stress_degradation = 'split'
|
| swift_K = None
|
| swift_eps0 = None
|
| swift_n = None
|
| voce_b = None
|
| voce_q_inf = None
|
| yield_stress = 0.0

```

`help(phast.load_result)` describes the result interface used in Section 8, and `python -m phast run --help` and `python -m phast postprocess --help` list the options for running and for regenerating figures

from a finished run directory. Section 8 defines a `RunViewer` class in this notebook, and `help(RunViewer)` works on it in exactly the same way.

7. The staggered scheme, one step at a time

Before running the full calculation, look at what the solver actually does in each increment. PhAST exposes the staggered scheme as four calls, so the sequence can be driven directly from Python rather than hidden inside a command.

Call	Operation
<code>step_mechanics()</code>	advance the displacement field
<code>step_compute_driving_force()</code>	evaluate the crack-driving energy and update the history H
<code>step_solve_damage()</code>	solve the damage subproblem
<code>advance_step()</code>	commit the increment

Read the order carefully, because it is the whole idea of a staggered scheme. Mechanics is solved with the damage held fixed, then damage is solved with the displacement held fixed. The two subproblems are never solved together, which allows different numerical methods for the two subproblems. Their convergence and stability still require separate checks.

In NB3, the analogous damage-solve operation is replaced by a trained network. Its FEM and GNN routes share the same mechanics, geometry and loading. NB3 itself uses a different specimen, constitutive law and quasi-static solver from this dynamic plate example.

`print_route_report()` states which route each part of the solver resolved to, so you never have to guess what ran.

Code cell 26

```
import numpy as np
import torch

material_model = phast.Material(
    E=32000.0, nu=0.20, Gc=3.0e-3, l0=0.5, rho=2.45e-9,
    eta_residual=1.0e-7, energy_split="spectral", pf_model="AT2")
mesh = phast.FEMMesh(str(mesh_file), device="cpu", dtype=torch.float64)
u_amp, dt_safety = 0.002, 0.8
print(mesh.summary())
print("Named node sets:", sorted(mesh.node_sets.keys()))
```

```
[FEMMesh] Loading mesh: phast_lab/reference.msh

[FEMMesh] Mesh file read OK
[FEMMesh] 3590 nodes loaded
[FEMMesh] 6902 T3 elements loaded
[FEMMesh] Node sets: ['bottom', 'right', 'top', 'left', 'notch_upper', 'notch_lower']
[FEMMesh] Precomputing areas, grad_phi, incircle, lumped mass...
[FEMMesh] Precompute done: h_min=9.827042e-02, total_area=1599.980000
FEMMesh: phast_lab/reference.msh
  3590 nodes, 6902 elements
  h_min=9.827042e-02, total_area=1599.980000
  Node set 'bottom': 21 nodes
  Node set 'right': 39 nodes
  Node set 'top': 21 nodes
  Node set 'left': 40 nodes
  Node set 'notch_upper': 81 nodes
  Node set 'notch_lower': 81 nodes
Named node sets: ['bottom', 'left', 'notch_lower', 'notch_upper', 'right', 'top']
```

Code cell 27

```

solver_settings = phast.SolverConfig(
    solver_type="explicit", dt_safety=dt_safety, num_steps=200)

boundary = phast.BoundaryConditions(mesh.n_nodes, mesh.device, mesh.dtype)
boundary.fix(np.asarray(mesh.node_sets["left"]), 0)
boundary.fix(np.asarray(mesh.node_sets["right"]), 0)
boundary.add(np.asarray(mesh.node_sets["top"]), 1, u_amp)
boundary.add(np.asarray(mesh.node_sets["bottom"]), 1, -u_amp)

demo = phast.StaggeredSolver(mesh, material_model, boundary, solver_settings)
demo.print_route_report()

```

```

[StaggeredSolver] Assembling solver (explicit)...
[FEM0operators] Initializing (spectral split, AT2)...
[FEM0operators] dt_CFL = 2.579599e-08 (c_p=3809523.81)
[PhaseFieldDamageSolver] Initializing CG solver (model=AT2, tol=1.0e-05, max_iter=5000,
bounds=post_clamp)...
[PhaseFieldDamageSolver] CG device: cpu, dtype: float64
[PhaseFieldDamageSolver] Auto-selecting preconditioner for cpu (cpu)
[AMGPreconditioner] GPU-native initialized: 3590 nodes, 6902 elements, device=cpu,
smoother=chebyshev
[PhaseFieldDamageSolver] Ready (preconditioner=amg).
[StaggeredSolver] Dynamic(central_difference): dt=2.063679e-08
[StaggeredSolver] Phase-field subcycling: damage solved every 3 steps
[StaggeredSolver] State allocated: 3590 nodes, 6902 elements, device=cpu, dtype=torch.float64
[PhAST route]
    device=cpu; dtype=torch.float64; elements=T3
    mechanics=ExplicitDynamics; solver_type=explicit; time_integrator=central_difference;
backend_requested=auto; backend_resolved=at first solve
    fracture=AT2; energy_split=spectral; history_update=hard_max; damage_update=classical;
bounds=post_clamp; preconditioner=None
[PhAST route]
    device=cpu; dtype=torch.float64; elements=T3
    mechanics=ExplicitDynamics; solver_type=explicit; time_integrator=central_difference;
backend_requested=auto; backend_resolved=at first solve
    fracture=AT2; energy_split=spectral; history_update=hard_max; damage_update=classical;
bounds=post_clamp; preconditioner=None

```

```

/usr/local/lib/python3.13/dist-packages/phast/solvers/staggered_solver.py:339: RuntimeWarning:
PhaseFieldDamageSolver: requested max_iter=5000 exceeds the inner adjoint/active-set PCG hard
cap of 1000; adjoint solves will be capped at 1000 iterations and may return non-converged.
    self.damage_solver = PhaseFieldDamageSolver(

```

```

{'device': 'cpu',
 'dtype': 'torch.float64',
 'element_type': 'T3',
 'solver_type': 'explicit',
 'mechanics': 'ExplicitDynamics',
 'backend_requested': 'auto',
 'backend_resolved': None,
 'time_integrator': 'central_difference',
 'phase_field_model': 'AT2',
 'energy_split': 'spectral',
 'history_update': 'hard_max',
 'damage': {'model': 'AT2',
            'update': 'classical',
            'bounds': 'post_clamp',
            'preconditioner': None,
            'last_route': 'classical'}}

```

Code cell 28

```

for step in range(1, 6):
    demo.bcs.load_factor = min(1.0, step / 100)
    demo.step_mechanics()           # (a) displacement, damage held fixed
    demo.step_compute_driving_force() # (b) crack-driving energy -> history H
    demo.step_solve_damage()        # (c) damage, displacement held fixed
    demo.advance_step()             # (d) commit
    print(f"step {step}: max damage = {float(demo.d.max()):.3e}"
          f"    max |u| = {float(demo.u.abs().max()):.3e} mm")

```

```

step 1: max damage = 1.750e-03    max |u| = 2.000e-05 mm
step 2: max damage = 6.915e-03    max |u| = 4.000e-05 mm
step 3: max damage = 1.534e-02    max |u| = 6.000e-05 mm
step 4: max damage = 2.675e-02    max |u| = 8.000e-05 mm
step 5: max damage = 4.081e-02    max |u| = 1.000e-04 mm

```

Those five increments are only a demonstration of the sequence. The production run below covers the whole loading window through the same solver, invoked through the command line interface so that its trajectory, energy history and figures are written to disk for the sections that follow.

7b. Run the simulation

Code cell 30

```

import subprocess
import sys
import time

run_dir = WORK / "runs" / "reference"
command = [
    sys.executable, "-m", "phast", "run", str(config_file),
    "--device", "cpu", "--output_dir", str(run_dir),
]
print(" ".join(command))

# Stream the solver's output as it arrives, so the cell reports progress
# while it runs instead of staying silent until the calculation finishes.
start = time.perf_counter()
process = subprocess.Popen(command, stdout=subprocess.PIPE,
                           stderr=subprocess.STDOUT, text=True, bufsize=1)

for line in process.stdout:
    print(line, end="")
process.wait()
elapsed = time.perf_counter() - start
if process.returncode != 0:
    raise RuntimeError("PhAST run failed")
print(f"\nWall-clock time on this runtime: {elapsed:.1f} s")

```

[Long console log abbreviated; see the online notebook for the full output.]
 Step 2400 | t= 49.53us | max(d)=1.00000 | 7.6ms

```

Total: 61.7s (2423 steps, 25.5 ms/step)
Saved: timing_per_step.csv (2423 rows)
Saved: energy.csv (2423 rows)
Saved: solver_telemetry.csv (2423 rows)
Zarr snapshots saved.
Saved: damage_final.png
Saved: run_metadata.json
[PhAST route]
  device=cpu; dtype=torch.float64; elements=T3
  mechanics=ExplicitDynamics; solver_type=explicit; time_integrator=central_difference;

```

(continues on next page)

(continued from previous page)

```
backend_requested=auto; backend_resolved=at first solve
fracture=AT2; energy_split=spectral; history_update=hard_max; damage_update=classical;
bounds=post_clamp; preconditioner=None
```

```
Run: python -m phast postprocess phast_lab/runs/reference
All outputs in: phast_lab/runs/reference/
```

```
Wall-clock time on this runtime: 67.1 s
```

8. A reusable post-processing toolkit

RunViewer is a helper class defined in this notebook, not a public PhAST class. It uses PhAST result and field interfaces and collects the operations that the exercises need: listing the stored snapshots, projecting element fields onto nodes, plotting a single frame, and animating a sequence.

Two storage conventions matter. Displacement, velocity and damage are nodal fields with one value per node. Strain and stress are element fields with one constant value per triangle. `phast.compute_field` performs the projection from elements to nodes and returns a display label and a colour map alongside the values, so derived quantities stay consistent between static plots and animations.

Available field names are damage, displacement_mag, H, and any key of `phast.FIELD_REGISTRY`, which includes stress_xx, stress_yy, stress_xy, von_mises_stress, max_principal_stress, min_principal_stress, hydrostatic_stress, stress_triaxiality, strain_xx, strain_yy, strain_xy, von_mises_strain, max_principal_strain and min_principal_strain.

Choosing a field for fracture post-processing. `von_mises_stress` is a distortional invariant built to predict *yielding* in ductile metals. It is insensitive to the sign of the stress state and carries no information about crack opening, so it is not a fracture criterion. This laboratory reads the crack through three quantities instead:

Field	Sym- bol	Why this is the right view
damage	d	The crack itself in the regularised model
max_principal_stress	σ_1	The tensile stress that opens a Mode-I crack, and the concentration ahead of the tip
H	H	The source term of the damage equation, and the only quantity that drives growth

In a sharp-crack treatment the corresponding measures would be the stress intensity factors K_I and K_{II} , the J -integral, or the crack-tip opening displacement. A phase-field model represents the crack by the field d rather than by a singular tip, so those measures require a separate extraction step and are not computed here.

Code cell 32

```
from matplotlib.animation import FuncAnimation, PillowWriter
import torch
import zarr

class RunViewer:
    """Read-only view of a finished PhAST run, prepared for plotting.

    The whole trajectory is read from the Zarr store once, in bulk, and derived
    fields are cached as arrays of shape (n_snapshots, n_nodes). Animation then
    updates the colour array of a single drawing rather than rebuilding the plot
    on every frame, which keeps a live classroom session responsive.

    Examples
    -----
    >>> view = RunViewer(run_dir)
```

(continues on next page)

(continued from previous page)

```

>>> view.plot_field("damage")
>>> view.animate_field("max_principal_stress", stride=4)
"""

#: fields read in bulk from the trajectory store
_RAW_KEYS = ("step", "time_s", "displacement", "stress", "strain",
             "damage_nodal", "H_nodal", "velocity")

def __init__(self, run_dir, *, nu=0.20):
    self.run_dir = Path(run_dir)
    self.result = phast.load_result(self.run_dir)
    self.nu = float(nu)
    self.mesh = phast.FEMMesh(str(self.run_dir / "mesh.msh"), device="cpu")
    self.nodes = np.asarray(self.mesh.nodes.cpu(), dtype=float)
    self.elements = np.asarray(self.mesh.elements.cpu(), dtype=int)
    self.triangulation = Triangulation(
        self.nodes[:, 0], self.nodes[:, 1], self.elements)
    self._raw = self._load_trajectory()
    self.steps = self._raw["step"]
    self.times = self._raw["time_s"]
    self._series_cache = {}

# -- bulk read -----
def _load_trajectory(self):
    """Read every stored snapshot in one pass over the Zarr store."""
    root = zarr.open(str(self.run_dir / "training_data.zarr"), mode="r")
    trajectory = root["simulation_data"]["trajectory"]
    count = int(trajectory.attrs.get("count", len(trajectory["step"])))
    raw = {key: np.asarray(trajectory[key][:count])
           for key in self._RAW_KEYS if key in trajectory}
    raw["step"] = raw["step"].astype(int)
    raw["time_s"] = raw["time_s"].astype(float)
    return raw

def index_of(self, step):
    """Position of a stored step number inside the snapshot arrays."""
    return int(np.clip(np.searchsorted(self.steps, int(step)),
                      0, len(self.steps) - 1))

def time_of(self, step):
    """Physical time in seconds for a stored step number."""
    return float(self.times[self.index_of(step)])

# -- derived nodal fields -----
def series(self, field_name):
    """Return (values, label, colour map) with values of shape (frames, nodes).

    Accepts ``damage``, ``displacement_mag``, ``H`` and any key of
    ``phast.FIELD_REGISTRY``. Element fields are projected onto the nodes by
    ``phast.compute_field``. Results are cached, so a second call is free.
    """
    if field_name in self._series_cache:
        return self._series_cache[field_name]

    if field_name == "displacement_mag":
        values = np.linalg.norm(self._raw["displacement"], axis=2)
        label, cmap = "Displacement magnitude [mm]", "viridis"
    else:
        frames = len(self.steps)
        values = np.empty((frames, self.nodes.shape[0]), dtype=float)
        label = cmap = None

```

(continues on next page)

(continued from previous page)

```

    for i in range(frames):
        stress = self._raw["stress"][i].astype(np.float64)
        strain = self._raw["strain"][i].astype(np.float64)
        frame, label, cmap = phast.compute_field(
            field_name,
            tuple(torch.as_tensor(stress[:, k]) for k in range(3)),
            tuple(torch.as_tensor(strain[:, k]) for k in range(3)),
            mesh=self.mesh,
            d=torch.as_tensor(self._raw["damage_nodal"][i].astype(np.float64)),
            H=torch.as_tensor(self._raw["H_nodal"][i].astype(np.float64)),
            nu=self.nu)
        values[i] = np.asarray(torch.as_tensor(frame).cpu(), dtype=float)

    self._series_cache[field_name] = (values, label, cmap)
    return self._series_cache[field_name]

def limits(self, field_name, values=None):
    """Colour limits held fixed across every frame."""
    if field_name == "damage":
        return 0.0, 1.0
    if values is None:
        values, _, _ = self.series(field_name)
    low, high = float(np.nanmin(values)), float(np.nanmax(values))
    if low < 0.0 < high:
        # A signed field is drawn on a diverging colour map. Centring the
        # scale on zero makes the neutral colour mean "no stress" instead
        # of the midpoint of the range.
        bound = max(abs(low), high)
        return -bound, bound
    return low, max(high, low + 1.0e-12)

def frame(self, field_name, step=-1):
    """Nodal values of one field at one stored step."""
    values, _, _ = self.series(field_name)
    index = len(self.steps) - 1 if step == -1 else self.index_of(step)
    return values[index]

def deformed(self, step, scale):
    """Triangulation on the displaced configuration, magnified by ``scale``."""
    if not scale:
        return self.triangulation
    index = len(self.steps) - 1 if step == -1 else self.index_of(step)
    moved = self.nodes + scale * self._raw["displacement"][index]
    return Triangulation(moved[:, 0], moved[:, 1], self.elements)

# -- single frame -----
def plot_field(self, field_name, step=-1, *, ax=None, vmin=None, vmax=None,
               deform_scale=0.0, colorbar=True, title=None, shading="gouraud"):
    """Draw one field at one stored step."""
    values, label, cmap = self.series(field_name)
    index = len(self.steps) - 1 if step == -1 else self.index_of(step)
    low, high = self.limits(field_name, values[index][None, :])
    vmin = low if vmin is None else vmin
    vmax = high if vmax is None else vmax
    if ax is None:
        _, ax = plt.subplots(figsize=(5.4, 4.8))
    art = ax.tripcolor(self.deformed(self.steps[index], deform_scale),
                       values[index], shading=shading, cmap=cmap,
                       vmin=vmin, vmax=vmax)
    ax.set_aspect("equal")
    ax.set_xlabel("x [mm]")

```

(continues on next page)

(continued from previous page)

```

ax.set_ylabel("y [mm]")
ax.set_title(title or f"{label}\\nstep {self.steps[index]}, "
               f"t = {self.times[index] * 1e6:.2f} us")

if colorbar:
    ax.figure.colorbar(art, ax=ax, shrink=0.85)
return ax

# -- animations -----
def animate_field(self, field_name, *, stride=4, fps=12, gif_path=None,
                  figsize=(5.2, 4.7)):
    """Animate one field. Only the colour array changes between frames."""
    values, label, cmap = self.series(field_name)
    vmin, vmax = self.limits(field_name, values)
    values, times = values[::stride], self.times[::stride]

    fig, ax = plt.subplots(figsize=figsize)
    art = ax.tripcolor(self.triangulation, values[0], shading="gouraud",
                      cmap=cmap, vmin=vmin, vmax=vmax)
    fig.colorbar(art, ax=ax, shrink=0.85)
    ax.set_aspect("equal")
    ax.set_xlabel("x [mm]")
    ax.set_ylabel("y [mm]")
    title = ax.set_title("")

    def draw(i):
        art.set_array(values[i])
        title.set_text(f"{label}\\nt = {times[i] * 1e6:.2f} us")
        return art, title

    animation = FuncAnimation(fig, draw, frames=len(times), interval=1000 / fps)
    if gif_path is not None:
        animation.save(str(gif_path), writer=PillowWriter(fps=fps))
    plt.close(fig)
    return animation

def animate_panel(self, field_names, *, stride=4, fps=12, gif_path=None,
                  ncols=2, figsize=(9.0, 7.6)):
    """Animate several fields side by side on a shared time axis."""
    panels = []
    for name in field_names:
        values, label, cmap = self.series(name)
        vmin, vmax = self.limits(name, values)
        panels.append((values[::stride], label, cmap, vmin, vmax))
    times = self.times[::stride]

    nrows = int(np.ceil(len(field_names) / ncols))
    fig, axes = plt.subplots(nrows, ncols, figsize=figsize)
    axes = np.atleast_1d(axes).ravel()
    artists = []
    for (values, label, cmap, vmin, vmax), ax in zip(panels, axes):
        art = ax.tripcolor(self.triangulation, values[0], shading="gouraud",
                          cmap=cmap, vmin=vmin, vmax=vmax)
        fig.colorbar(art, ax=ax, shrink=0.82)
        ax.set_aspect("equal")
        ax.set_xticks([])
        ax.set_yticks([])
        ax.set_title(label, fontsize=10)
        artists.append(art)
    for ax in axes[len(field_names):]:
        ax.axis("off")
    heading = fig.suptitle("")

```

(continues on next page)

(continued from previous page)

```

def draw(i):
    for art, panel in zip(artists, panels):
        art.set_array(panel[0][i])
        heading.set_text(f"t = {times[i] * 1e6:.2f} microseconds")
    return artists

animation = FuncAnimation(fig, draw, frames=len(times), interval=1000 / fps)
if gif_path is not None:
    animation.save(str(gif_path), writer=PillowWriter(fps=fps))
plt.close(fig)
return animation

def crack_tip_history(view, *, threshold=0.9):
    """Return (time [us], furthest broken x [mm]) over the stored snapshots.

    A node counts as broken when its damage exceeds ``threshold``. The furthest
    such node in x is reported as the crack tip position.
    """
    damage, _, _ = view.series("damage")
    tips = np.full(len(view.steps), np.nan)
    for i in range(len(view.steps)):
        broken = damage[i] > threshold
        if broken.any():
            tips[i] = view.nodes[broken, 0].max()
    return view.times * 1e6, tips

view = RunViewer(run_dir)
print(f"{len(view.steps)} stored snapshots, "
      f"steps {view.steps[0]} to {view.steps[-1]}, "
      f"t up to {view.times[-1] * 1e6:.1f} us")
print("Stored fields:", view.result.field_names())

```

```

[FEMMesh] Loading mesh: phast_lab/runs/reference/mesh.msh

[FEMMesh] Mesh file read OK
[FEMMesh] 3590 nodes loaded
[FEMMesh] 6902 T3 elements loaded
[FEMMesh] Node sets: ['bottom', 'right', 'top', 'left', 'notch_upper', 'notch_lower']
[FEMMesh] Precomputing areas, grad_phi, incircle, lumped mass...
[FEMMesh] Precompute done: h_min=9.827042e-02, total_area=1599.980000
122 stored snapshots, steps 0 to 2420, t up to 49.9 us
Stored fields: ['acceleration', 'damage', 'displacement', 'history_field',
'history_field_nodal', 'psi_plus', 'strain', 'stress', 'velocity']

```

Code cell 33

```

# The same built-in help works on the objects defined in this notebook.
help(RunViewer.animate_field)

```

Help on function animate_field in module __main__:

```

animate_field(
    self,
    field_name,
    *,
    stride=4,
    fps=12,
    gif_path=None,

```

(continues on next page)

(continued from previous page)

```
figsize=(5.2, 4.7)
)
Animate one field. Only the colour array changes between frames.
```

9. Final state

The four panels below show the state at the last stored snapshot. Read them together. The displacement field shows the imposed opening and the discontinuity across the crack. The vertical strain concentrates in the process zone. The maximum principal stress σ_1 falls close to zero inside the broken band, which is the mechanical signature of the degradation function $g(d)$, and it peaks just ahead of the crack tip, where intact material still carries the tensile load that drives further growth. The phase field shows the diffuse crack itself, with a band width set by ℓ_0 .

Code cell 35

```
fields = ("displacement_mag", "strain_yy", "max_principal_stress", "damage")

fig, axes = plt.subplots(2, 2, figsize=(11.0, 9.2))
for name, ax in zip(fields, axes.ravel()):
    view.plot_field(name, step=-1, ax=ax)
fig.suptitle(f"Final state, t = {view.times[-1] * 1e6:.1f} microseconds", fontsize=13)
fig.tight_layout()
plt.show()
```

The same fields may be drawn on the deformed configuration. The displacements are of the order of a few micrometres, so a large magnification factor is needed for the opening to be visible.

Code cell 37

```
fig, ax = plt.subplots(figsize=(6.0, 5.4))
view.plot_field("damage", step=-1, ax=ax, deform_scale=500.0,
               title="Damage on the deformed mesh, magnification 500")
plt.show()
```

The quantity that drives damage

Section 2 defined the history variable $H(x, t) = \max_{s \leq t} \psi^+(\epsilon(x, s))$, and the AT2 damage equation

$$\frac{G_c}{\ell_0} d - G_c \ell_0 \nabla^2 d + 2H d = 2H$$

carries H as its only source term. Damage responds to H , not to any stress invariant, so plotting the two side by side shows the driver and its consequence together.

Read the two quantities differently, because they answer different questions.

σ_1 shows where the material is being driven now. Its peak sits a short distance *ahead* of the leading edge of the damage band, in intact material that is still carrying tensile load, and that offset is what advances the crack.

H is a running maximum, so it is a record rather than a snapshot. Its largest value stays near the original notch tip, where the tensile energy density first peaked, while its leading edge tracks how far the crack has travelled. It never decreases anywhere, which is precisely the irreversibility condition that stops a crack from healing when the local strain relaxes.

In the reference run the mid-propagation snapshot puts the damage band between about $x = 19.5$ and $x = 30.3$ mm, the peak of σ_1 near $x = 31.7$ mm, and the peak of H near $x = 20.5$ mm. Those three numbers together describe the mechanism: driving ahead, damage behind, history retained.

The comparison is drawn at a mid-propagation snapshot as well as at the end. While the crack is still advancing, the tensile concentration sits ahead of the damage band and the offset between them is easy to see.

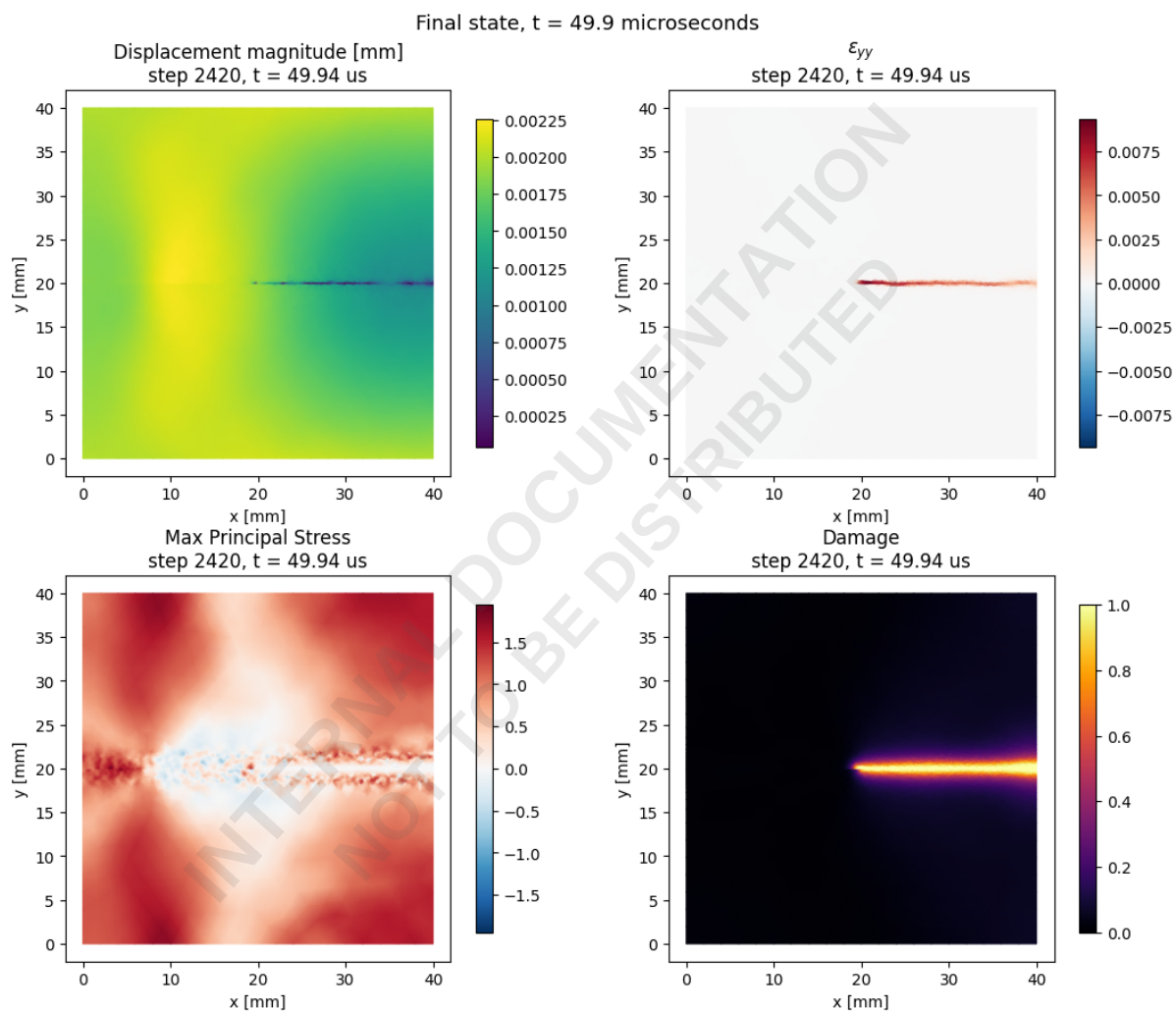


Fig. 2: Saved output from code cell 35.

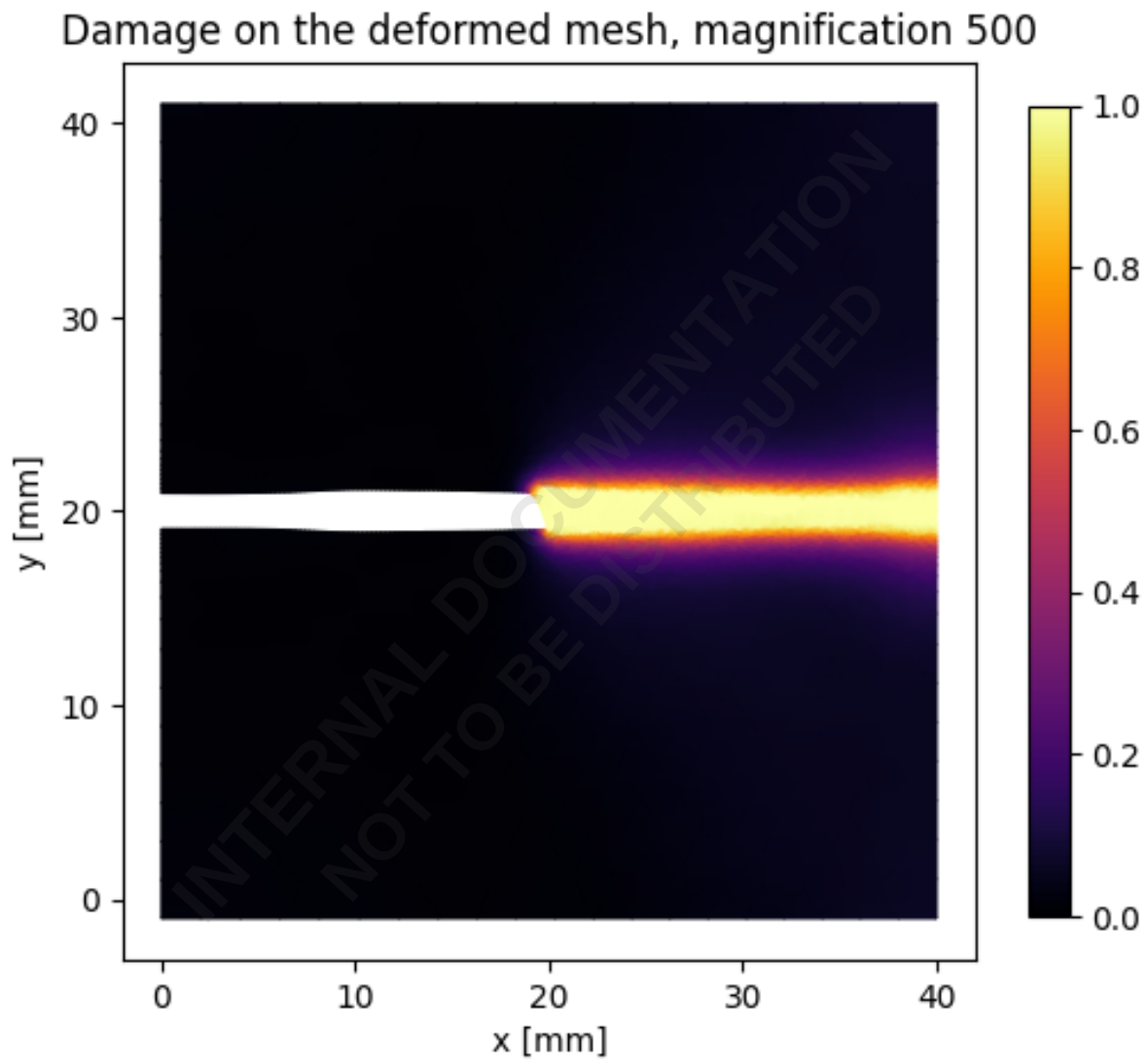


Fig. 3: Saved output from code cell 37.

Once the crack has crossed the ligament the specimen unloads, the peak σ_1 in this run falls by roughly a factor of three from its maximum, and the offset is no longer informative. Exercise 1 uses the mid-propagation snapshot for that reason.

Code cell 39

```
mid = len(view.steps) // 2          # crack still advancing
last = len(view.steps) - 1          # ligament crossed, specimen unloading

fig, axes = plt.subplots(2, 2, figsize=(11.0, 9.0))
for row, index in enumerate((mid, last)):
    step = view.steps[index]
    view.plot_field("H", step=step, ax=axes[row, 0])
    view.plot_field("max_principal_stress", step=step, ax=axes[row, 1])
    axes[row, 0].set_title(f"History $H$, t = {view.times[index]*1e6:.1f} us")
    axes[row, 1].set_title(f"$\\sigma_1$, t = {view.times[index]*1e6:.1f} us")
fig.tight_layout()
plt.show()
```

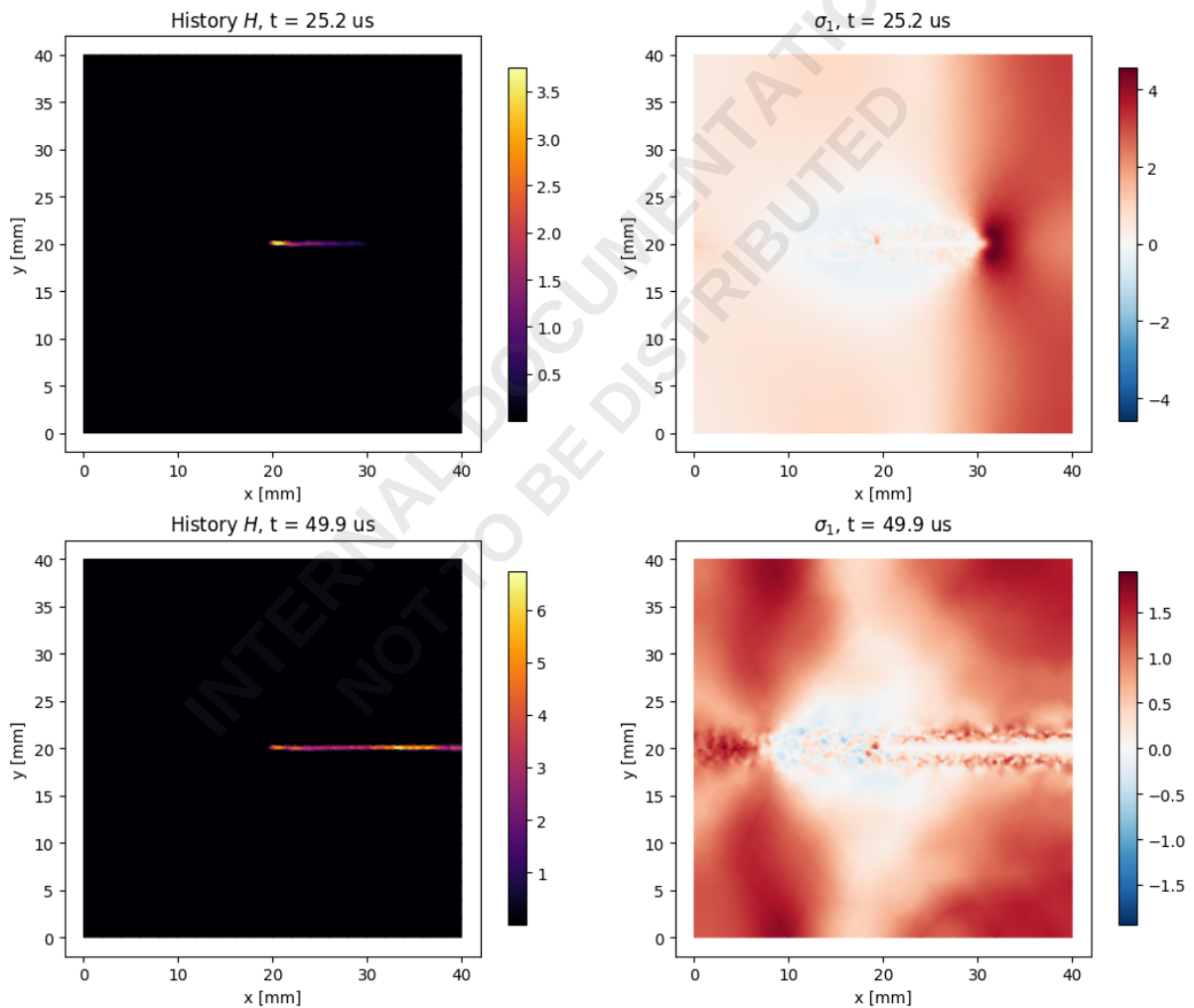


Fig. 4: Saved output from code cell 39.

10. Animate the four fields

The derived series were built once in Section 8 and cached, so the only work left is rendering. Each frame updates the colour array of an existing drawing, which is considerably cheaper than redrawing the contours. `stride` selects every `n`th stored snapshot. A stride of 4 reduces the number of frames while retaining the main stages of crack propagation. Set `SAVE_GIFS = True` to save GIF files for a report or offline viewing.

The animations are displayed inline with `to_jshtml`, which embeds a player with frame controls.

Code cell 41

```
from IPython.display import HTML, display

STRIDE = 4          # render every 4th stored snapshot
SAVE_GIFS = False   # also write .gif files into the working directory

plt.rcParams["animation.embed_limit"] = 100 # MB of embedded animation per notebook

animations = {}
for name in fields:
    gif = WORK / f"reference_{name}.gif" if SAVE_GIFS else None
    animations[name] = view.animate_field(name, stride=STRIDE, fps=12, gif_path=gif)
print("Rendered:", list(animations))
```

Rendered: ['displacement_mag', 'strain_yy', 'max_principal_stress', 'damage']

Code cell 42

```
display(HTML(animations["displacement_mag"].to_jshtml()))
```

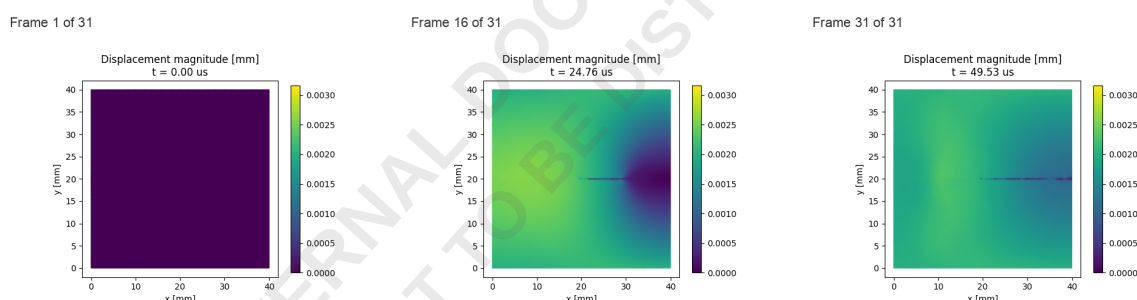


Fig. 5: Three stages from the saved animation (31 frames). [Play the animation online.](#)

Code cell 43

```
display(HTML(animations["strain_yy"].to_jshtml()))
```

Code cell 44

```
display(HTML(animations["max_principal_stress"].to_jshtml()))
```

Code cell 45

```
display(HTML(animations["damage"].to_jshtml()))
```

A combined panel places the four fields on one time axis, which makes the sequence of events easier to follow: the stress wave arrives first, the process zone forms at the notch tip, and the damage band then advances while the stress behind it collapses.

Code cell 47

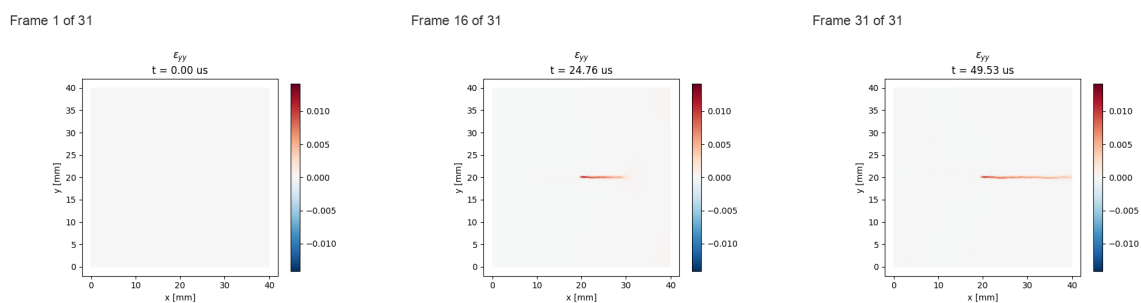


Fig. 6: Three stages from the saved animation (31 frames). [Play the animation online.](#)

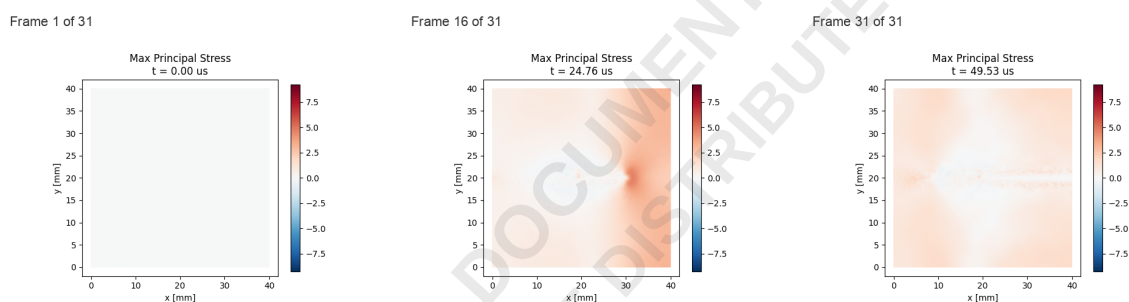


Fig. 7: Three stages from the saved animation (31 frames). [Play the animation online.](#)

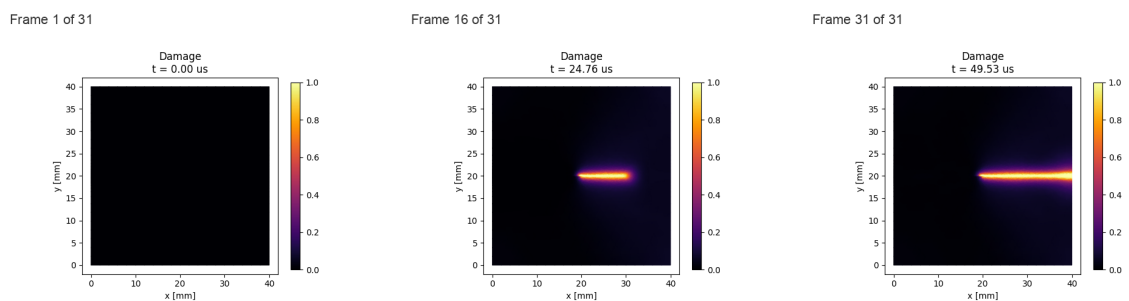


Fig. 8: Three stages from the saved animation (31 frames). [Play the animation online.](#)

```

panel = view.animate_panel(
    fields, stride=STRIDE, fps=12,
    gif_path=WORK / "reference_panel.gif" if SAVE_GIFS else None)
display(HTML(panel.to_jshtml()))

```

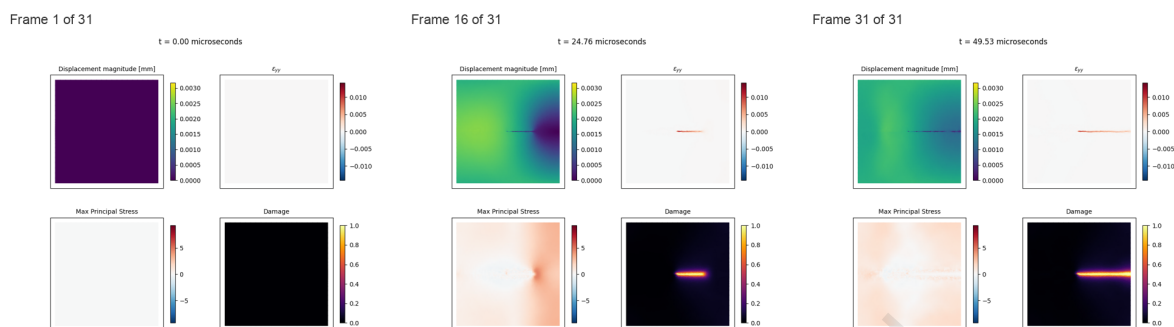


Fig. 9: Three stages from the saved animation (31 frames). [Play the animation online.](#)

11. Energy and crack-tip histories

A dynamic result should be judged from histories as well as from final images. PhAST writes `energy.csv` and related tables into the run directory, and they are reachable through the public result interface.

The energy plot separates the stored elastic energy, the kinetic energy and the dissipated fracture energy. The fracture energy rises as the crack advances. The crack-tip trace is obtained from the toolkit function `crack_tip_history` and gives an estimate of the propagation speed, which should be compared with the Rayleigh wave speed of the material.

Code cell 49

```

print("Available histories:", view.result.history_names())

energy = view.result.history("energy")
columns = {key: np.array([row[key] for row in energy], dtype=float)
            for key in energy[0]}
print("Energy columns:", list(columns))

time_key = next(key for key in ("time", "t_s", "t") if key in columns)
time_us = columns[time_key] * 1e6

crack_time, crack_x = crack_tip_history(view)

fig, axes = plt.subplots(1, 2, figsize=(12.0, 4.4))
for name in ("elastic", "kinetic", "fracture", "external", "total"):
    if name in columns:
        axes[0].plot(time_us, columns[name], label=name)
axes[0].set_xlabel("Time [microseconds]")
axes[0].set_ylabel("Energy [N mm]")
axes[0].set_title("Energy history")
axes[0].grid(alpha=0.25)
axes[0].legend()

axes[1].plot(crack_time, crack_x, color="#a94f42")
axes[1].set_xlabel("Time [microseconds]")
axes[1].set_ylabel("Furthest broken node, x [mm]")
axes[1].set_title("Crack-tip position, damage threshold 0.9")
axes[1].grid(alpha=0.25)

```

(continues on next page)

(continued from previous page)

```

fig.tight_layout()
plt.show()

valid = np.isfinite(crack_x)
if valid.sum() > 2:
    speed = np.gradient(crack_x[valid], crack_time[valid] * 1e-6) # mm/s
    print(f"Mean propagation speed over the traced window: {np.nanmean(speed):.3e} mm/s")
    print(f"Dilatational wave speed c_p: {c_p:.3e} mm/s")

```

Available histories: ['energy', 'max_damage', 'solver_telemetry', 'timing_per_step']
 Energy columns: ['step', 'time', 'elastic', 'fracture', 'kinetic', 'external', 'total']

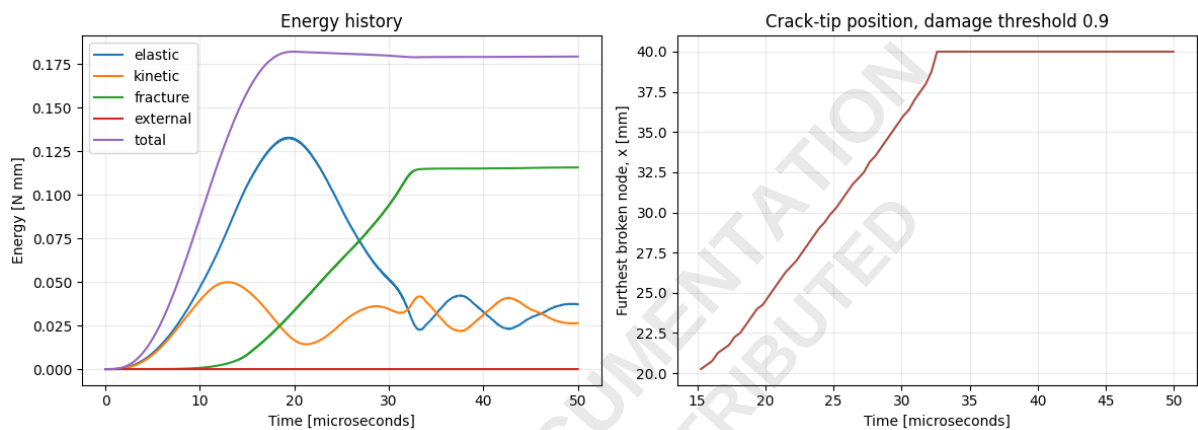


Fig. 10: Saved output from code cell 49.

Mean propagation speed over the traced window: 5.665e+05 mm/s
 Dilatational wave speed c_p: 3.810e+06 mm/s

Key takeaways

- Named mesh regions connect geometry to boundary conditions.
- Dynamic mechanics retains inertia and requires a stable time step.
- Fixed colour scales make evolving fields comparable.

Consolidation

Why does refining the mesh increase dynamic computation time?

Hint

Consider the smallest element and the wave speed.

Conceptual answer

The explicit stability limit scales with the smallest element dimension divided by wave speed. A smaller element reduces the stable time step and increases the number of updates over the same physical duration.

12. Exercises

The exercises reuse `write_sent_geo`, `build_mesh`, `make_config` and `RunViewer`. Each one asks for a short written answer as well as a calculation. Record the wall-clock time of every run you report.

Exercise 1 — Read the reference result

No new calculation is required.

1. State the element count, $h_{\min}$ and the stable increment of the reference mesh.
2. Measure the width of the damage band across the crack at $x = 30$ mm, taking the band edges where $d = 0.5$, and compare it with ℓ_0 .
3. Explain why the maximum principal stress σ_1 falls to near zero behind the crack tip while the displacement field remains finite there, and name the factor in the constitutive law that is responsible.
4. Using the mid-propagation panels of Section 9, locate the peak of σ_1 relative to the leading edge of the damage band, and account for the offset using the history variable H . Then state why the same reading taken at the final snapshot would be misleading.

Exercise for independent work.

Code cell 53

```
# Exercise 1, part 2: a starting point.
damage = view.frame("damage", step=-1)
column = np.abs(view.nodes[:, 0] - 30.0) < 0.3
order = np.argsort(view.nodes[column, 1])
y = view.nodes[column, 1][order]
d = damage[column][order]

fig, ax = plt.subplots(figsize=(5.6, 4.0))
ax.plot(y, d, marker="o", ms=3)
ax.axhline(0.5, color="0.6", ls="--")
ax.set_xlabel("y [mm]")
ax.set_ylabel("Damage")
ax.set_title("Damage profile across the crack at x = 30 mm")
ax.grid(alpha=0.25)
plt.show()
```

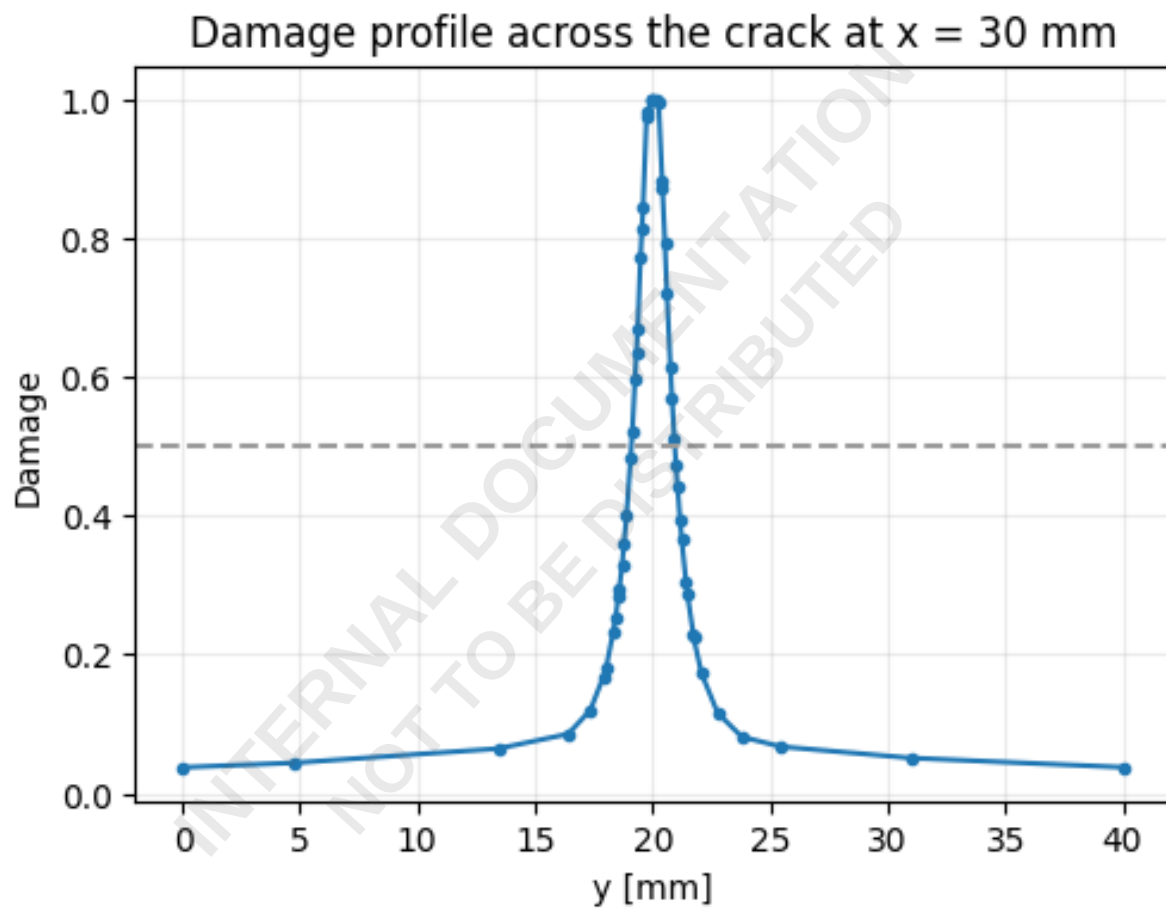


Fig. 11: Saved output from code cell 53.

Exercise 2 — An inclined notch

Repeat the calculation with the notch inclined at 20 degrees above the horizontal. The specimen is still pulled vertically, so the notch is no longer aligned with the direction that a mode-I crack prefers.

1. Run the case with the cell below.
2. Compare the crack path with the reference result. Measure the angle at which the crack leaves the notch tip, and the angle it settles into further on.
3. Explain the kink using the tensile part of the strain energy: the crack turns towards the plane on which the maximum principal stress acts normally.
4. Repeat for 30 and 45 degrees. Report how the initiation time and the crack path change, and state at which inclination the refinement band supplied by `write_sent_geo` stops covering the actual path.

Exercise for independent work.

Code cell 55

```
def run_case(tag, *, angle_deg=0.0, geo_kwargs=None, config_kwargs=None):
    """Mesh, configure and run one variant. Returns a RunViewer for the result."""
    geo_kwargs = dict(geo_kwargs or {})
    config_kwargs = dict(config_kwargs or {})
    case_geo = write_sent_geo(WORK / f"{tag}.geo", angle_deg=angle_deg, **geo_kwargs)
    case_mesh = build_mesh(case_geo, WORK / f"{tag}.msh")
    case_config = make_config(case_mesh, name=tag, **config_kwargs)
    case_config_file = WORK / f"{tag}.yaml"
    case_config_file.write_text(yaml.safe_dump(case_config, sort_keys=False),
                                encoding="utf-8")
    case_dir = WORK / "runs" / tag

    start = time.perf_counter()
    outcome = subprocess.run(
        [sys.executable, "-m", "phast", "run", str(case_config_file),
         "--device", "cpu", "--output_dir", str(case_dir)],
        capture_output=True, text=True)
    elapsed = time.perf_counter() - start
    if outcome.returncode != 0:
        print(outcome.stdout[-2000:])
        print(outcome.stderr[-2000:])
        raise RuntimeError(f"Run {tag} failed")
    print(f"{tag}: finished in {elapsed:.1f} s")
    return RunViewer(case_dir)

inclined = run_case("notch_20deg", angle_deg=20.0)

fig, axes = plt.subplots(1, 2, figsize=(11.0, 4.6))
view.plot_field("damage", step=-1, ax=axes[0], title="Horizontal notch, final damage")
inclined.plot_field("damage", step=-1, ax=axes[1], title="20 degree notch, final damage")
fig.tight_layout()
plt.show()
```

```
notch_20deg: finished in 67.2 s
[FEMMesh] Loading mesh: phast_lab/runs/notch_20deg/mesh.msh

[FEMMesh] Mesh file read OK
[FEMMesh] 3658 nodes loaded
[FEMMesh] 7036 T3 elements loaded
[FEMMesh] Node sets: ['bottom', 'right', 'top', 'left', 'notch_upper', 'notch_lower']
[FEMMesh] Precomputing areas, grad_phi, incircle, lumped mass...
[FEMMesh] Precompute done: h_min=1.002033e-01, total_area=1599.981206
```

Code cell 56

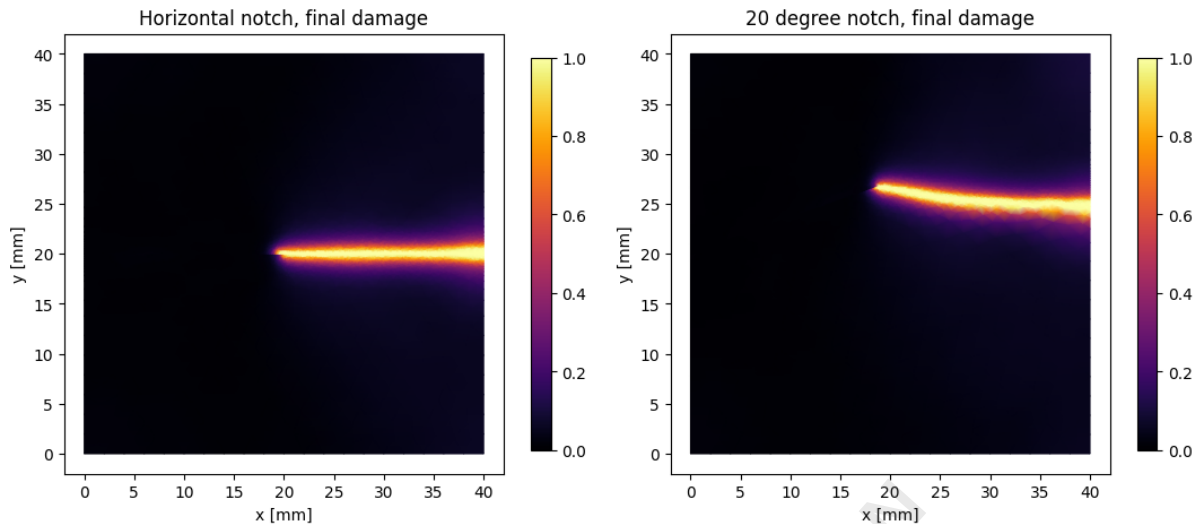
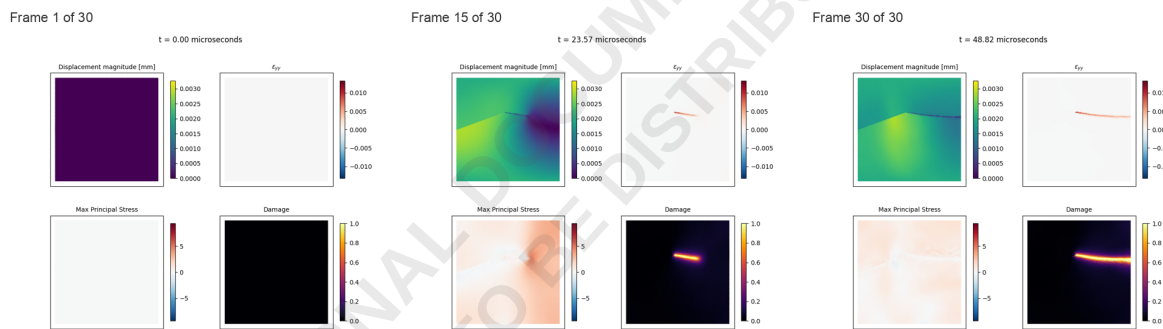


Fig. 12: Saved output from code cell 55.

```
# Animate the inclined case. All four fields are available here as well.
inclined_panel = inclined.animate_panel(fields, stride=STRIDE, fps=12)
display(HTML(inclined_panel.to_jshtml()))
```

Fig. 13: Three stages from the saved animation (30 frames). [Play the animation online.](#)

Exercise 3 — The length scale and the mesh

The length scale ℓ_0 sets the width of the diffuse band and, through the regularised model, the effective strength of the material. The mesh must resolve that width.

1. Rerun the reference case with $\ell_0 = 0.25$ mm and with $\ell_0 = 1.0$ mm, keeping the mesh fixed. Report the band width and the initiation time in each case.
2. State which of the three runs violates $h \leq \ell_0/2$ inside the crack path, and describe what the damage field looks like when the requirement fails.
3. Rerun the $\ell_0 = 0.25$ mm case with $h_{\text{crack}}=0.1$. Report the change in element count, in the stable increment and in the wall-clock time, and explain why refining the mesh raises the cost more than proportionally.

Exercise for independent work.

Code cell 58

```
# Exercise 3: one variant is set up for you. Set RUN_THIS to True to execute it,
# then copy the cell and change the parameters for the remaining cases.
```

(continues on next page)

(continued from previous page)

```

RUN_THIS = False

if RUN_THIS:
    coarse_l0 = run_case("l0_1p0", angle_deg=0.0, config_kwargs={"l0": 1.0})

    fig, axes = plt.subplots(1, 2, figsize=(11.0, 4.6))
    view.plot_field("damage", step=-1, ax=axes[0], title="l0 = 0.5 mm")
    coarse_l0.plot_field("damage", step=-1, ax=axes[1], title="l0 = 1.0 mm")
    fig.tight_layout()
    plt.show()
else:
    print("Set RUN_THIS = True to run the l0 = 1.0 mm variant "
          "(about one solve, comparable to Section 7).")

```

Set RUN_THIS = True to run the l0 = 1.0 mm variant (about one solve, comparable to Section 7).

Exercise 4 — Loading amplitude and rate

1. Rerun the reference case with u_{amp} reduced to 0.0015 mm and raised to 0.003 mm. Report the initiation time and the mean propagation speed in each case.
2. Keep u_{amp} at 0.002 mm and shorten the ramp to $t_{\text{ramp}} = 5 \cdot 10^{-6}$. Describe what changes in the early part of the maximum principal stress animation and explain it in terms of the stress wave that the ramp launches.
3. At the highest amplitude, inspect whether the crack remains a single band. Comment on what a branching pattern would require of the mesh.

Exercise for independent work.

Exercise 5 — Add a field of your own

`RunViewer.series` and `RunViewer.frame` accepts any key of `phast.FIELD_REGISTRY`. Choose one that the notebook has not used, for example `stress_triaxiality` or `max_principal_strain`.

1. Plot it at three times: before initiation, during propagation and at the end.
2. Animate it and state what it reveals that the four fields in Section 10 do not.
3. Write a short function `band_width(view, x, step)` that returns the damage band width at a given station, following the profile in Exercise 1, and use it to plot band width against x for the reference run.

Exercise for independent work.

Code cell 61

```

print("Fields available through phast.compute_field:")
print(sorted(phast.FIELD_REGISTRY.keys()))

```

```

Fields available through phast.compute_field:
['hydrostatic_stress', 'max_principal_strain', 'max_principal_stress', 'min_principal_strain',
 'min_principal_stress', 'strain_xx', 'strain_xy', 'strain_yy', 'stress_triaxiality',
 'stress_xx', 'stress_xy', 'stress_yy', 'von_mises_strain', 'von_mises_stress']

```

Further reading

- Borden, Verhoosel, Scott, Hughes and Landis (2012), *A phase-field description of dynamic brittle fracture*, CMAME 217–220, 77–95.
- Miehe, Welschinger and Hofacker (2010), *Thermodynamically consistent phase-field models of fracture*, IJNME 83, 1273–1311.
- PhAST documentation: <https://cems-lab.github.io/PhAST/>

INTERNAL DOCUMENTATION
NOT TO BE DISTRIBUTED

Lecture 2: Differentiability and inverse recovery

The forward problem predicts a mechanical response from known inputs. The inverse problem estimates an unknown input from an observed response. Begin with the small elastic bar, then connect the same operations to the particle-position exhibits.

1. State the experiment

A uniform bar has length $L = 100$ mm and area $A = 10$ mm². Its left end is fixed and a tensile force $F = 4000$ N acts at the right. The unknown is the uniform Young's modulus E . The observation is the tip displacement y_{obs} . Geometry, area and force are known independently. This example uses linear elasticity; damage evolution belongs to the later fracture applications.

2. Build and solve the forward problem with PhAST

The continuum equation and its finite-element form are

$$\begin{aligned} \frac{d}{dx} \left(EA \frac{du}{dx} \right) &= 0, \\ u(0) &= 0, \quad EAu'(L) = F, \\ K_{ff}(E)u_f &= f_f. \end{aligned}$$

The subscript f denotes unconstrained degrees of freedom. Ten line elements give eleven nodes. Reuse this forward function throughout recovery:

```
mesh = phast.line_mesh(length=100.0, n_elements=10)

def forward(E):
    return phast.solve_bar(mesh, young_modulus=E,
                           area=10.0, end_force=4000.0)
```

For this uniform bar, $u(x) = Fx/(EA)$ supplies an analytical comparison. The notebook creates a synthetic observation with $E_{\text{ref}} = 210$ GPa, giving $y_{\text{obs}} \approx 0.190476$ mm. Inspect nodal displacement, reaction and the free-degree residual before beginning recovery.

3. Differentiate a scalar mismatch

At a trial modulus, predict $y(E) = u(L; E)$ and define

$$\begin{aligned} \mathcal{L}(E) &= \left(\frac{y(E) - y_{\text{obs}}}{y_{\text{obs}}} \right)^2, \\ \frac{d\mathcal{L}}{dE} &= \frac{2(y - y_{\text{obs}})}{y_{\text{obs}}^2} \frac{dy}{dE}, \\ \frac{dy}{dE} &= -\frac{FL}{AE^2}. \end{aligned}$$

The sequence is $E \rightarrow u \rightarrow y \rightarrow \mathcal{L}$. `loss.backward()` differentiates it, including the PhAST solve. At the initial $E = 100$ GPa the bar extends too much, so increasing E locally reduces the loss. This gives a gradient at one state, not a sweep over moduli.

4. Let the optimiser choose the next modulus

Use $q = \log(E/E_{\text{scale}})$ with $E_{\text{scale}} = 1$ MPa, so $E = E_{\text{scale}} \exp(q)$ remains positive. The code stores modulus values in MPa. The Day 3 notebook uses SGD with momentum:

```
log_E = torch.nn.Parameter(torch.log(torch.tensor(100000.)))
optimiser = torch.optim.SGD([log_E], lr=0.1, momentum=0.5)
optimiser.zero_grad()
y = forward(log_E.exp()).displacement[-1]
loss = ((y - observed) / observed)**2
loss.backward()
optimiser.step()
```

Repeat prediction, loss, backward and update until the relative tip mismatch is below 10^{-4} for five consecutive evaluations, with at most 80 updates. The force stays fixed. Momentum can produce overshoot and oscillation.

5. Interpret and check the result

In [NB2](#), compare the recovered field with the observed tip and analytical extension. The synthetic, noise-free data test this implementation; experimental identification adds measurement and model uncertainty.

Finite differences and Taylor remainders are optional derivative checks:

$$\mathcal{L}'(E) \approx \frac{\mathcal{L}(E+h) - \mathcal{L}(E-h)}{2h},$$

$$R(h) = |\mathcal{L}(E+h) - \mathcal{L}(E) - h\mathcal{L}'(E)|.$$

The first compares the gradient with nearby solves. For a smooth function and a correct derivative, the second decreases as h^2 until numerical error dominates. These checks assess a local derivative separately from the inverse loop. The [reference derivation](#) gives details.

6. Extend the same chain rule to fracture

For observations $y(\theta)$ and a symmetric positive-semidefinite weighting matrix W ,

$$\mathcal{L} = \frac{1}{2}(y - y_{\text{obs}})^T W (y - y_{\text{obs}}),$$

$$\nabla_{\theta} \mathcal{L} = J^T W (y - y_{\text{obs}}),$$

$$J = \frac{\partial y}{\partial \theta}.$$

This vector–Jacobian product combines the sensitivity of each observation into a gradient of the scalar loss. For an implicitly solved equilibrium equation, write the force residual as $R(u, \theta) = 0$. If the residual is differentiable and its displacement Jacobian is invertible, differentiating this equation locally gives

$$R_u \frac{\partial u}{\partial \theta} = -R_{\theta}.$$

In fracture, gradients also pass through earlier states, history and damage. Particle-position recovery uses inclusion coordinates as unknowns and field observations as data. Sensitivity, active constraints and non-uniqueness require assessment; see the optional [inverse visual laboratory](#).

NB2 — Inverse Problem using PhAST

Recovering Young's modulus from measured extension

A differentiable finite-element exercise with PhAST

UKACM Autumn School 2026 · 16 September

Developed by Allamaprabhu Ani and Sathiskumar A. Ponnusami, CEMS-Lab, City St George's, University of London Queen Mary University of London

[PhAST repository](#) · [PhAST documentation](#)

We will build a 1D model, calculate its extension, and recover Young's modulus by differentiating through the PhAST forward solve.

The notebook saves its figures, animation and numerical tables in `bar_results/` in the working directory.

Day 3 · PhAST practical. Updated 16 September 2026. Figures and animation are retained from the supplied notebook. This book update preserves those outputs. The calculation uses SGD with momentum 0.5; the lecture deck also illustrates plain SGD, which has a different optimisation history.

1. Load PhAST and tools

Code cell 4

```
import importlib.util
import subprocess
import sys

PHAST_SPEC = "phast @ git+https://github.com/CEMS-Lab/PhAST.
->git@157412953099bfcace1668bb35cb911826a4e95e"
if sys.version_info[:2] < (3, 10):
    raise RuntimeError("PhAST requires Python 3.10 or newer.")
REQUIRED = ("line_mesh", "solve_bar")

def course_solver_present():
    """True only when the importable `phast` really is the course solver."""
    if importlib.util.find_spec("phast") is None:
        return False
    import phast
    return all(hasattr(phast, name) for name in REQUIRED)

if not course_solver_present():
    subprocess.check_call([sys.executable, "-m", "pip", "install", "-q", PHAST_SPEC])
    importlib.invalidate_caches()

import phast

if not all(hasattr(phast, name) for name in REQUIRED):
    raise RuntimeError(
        "A different package named 'phast' is shadowing the course solver:\n"
        f" {phast.__file__}\n"
```

(continues on next page)

(continued from previous page)

```
f"Install with %pip install '{PHAST_SPEC}', then restart the kernel.")

print("PhAST module: ", phast.__file__)
print("Forward solver:", phast.solve_bar.__module__)
```

```
PhAST module: /usr/local/lib/python3.13/dist-packages/phast/__init__.py
Forward solver: phast.bar
```

Code cell 5

```
from pathlib import Path
from time import perf_counter
import json
import csv
import numpy as np
import torch
import matplotlib.pyplot as plt
from IPython.display import display, HTML, Image

torch.set_default_dtype(torch.float64)
torch.set_num_threads(1)
lesson_start = perf_counter()
output = Path("bar_results")
output.mkdir(exist_ok=True)
plt.rcParams.update({
    "font.family": "STIXGeneral", "mathtext.fontset": "stix",
    "font.size": 13, "axes.titlesize": 16, "axes.labelsize": 14,
    "figure.facecolor": "white", "axes.facecolor": "white",
    "axes.spines.top": False, "axes.spines.right": False,
    "svg.fonttype": "path",
})

def show_figure(fig, name):
    fig.savefig(output / f"{name}.png", dpi=180, bbox_inches="tight")
    fig.savefig(output / f"{name}.svg", bbox_inches="tight")
    display(Image(filename=str(output / f"{name}.png"), width=950))
    plt.close(fig)
```

2. Geometry and material setup

The bar has length $L = 100$ mm and area $A = 10$ mm².

For the reference forward calculation we choose $E_{\text{ref}} = 210000$ MPa = 210 GPa.

We will use its tip displacement as a synthetic measurement.

The inverse loop receives that measurement and a starting guess; it does not use the reference modulus to choose updates.

$$\varepsilon = \frac{du}{dx}, \quad \sigma = E\varepsilon, \quad N = A\sigma.$$

Here u is displacement, ε strain, σ stress and N internal axial force. With mm–N units, 1 MPa = 1 N/mm².

Code cell 7

```
length = 100.0 # mm
area = 10.0 # mm^2
E_reference = torch.tensor(210000.0) # MPa; generates the synthetic observation
print(f"Length: {length:g} mm; area: {area:g} mm^2")
print(f"Reference modulus: {E_reference.item()/1000:g} GPa")
```

Length: 100 mm; area: 10 mm²
Reference modulus: 210 GPa

Code cell 8

```
fig, ax = plt.subplots(figsize=(8, 2.3), layout="constrained")
ax.plot([0, length], [0, 0], lw=12, color="#2980B9", solid_capstyle="butt")
ax.annotate("", (length, -0.32), (0, -0.32), arrowprops={"arrowstyle": "<->", "color": "gray"}
↪)
ax.text(length/2, -0.4, r"$L=100\,\mathrm{mm}$", ha="center", va="top")
ax.text(length/2, 0.2, r"$A=10\,\mathrm{mm}^2$", ha="center")
ax.set(xlim=(-8, 120), ylim=(-.65, .55), title="Original, undeformed bar")
ax.axis("off")
show_figure(fig, "01_geometry")
```

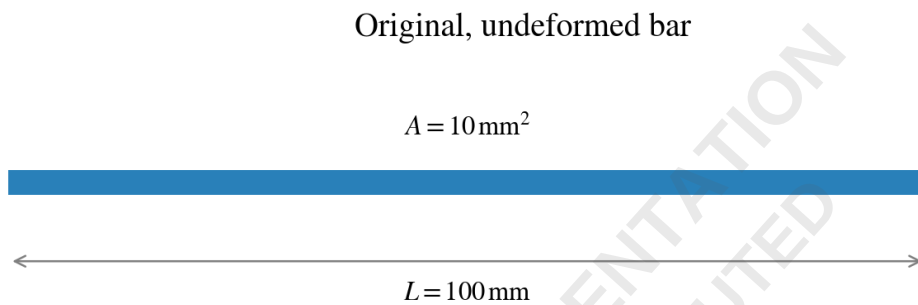


Fig. 14: Saved output from code cell 8.

3. Generate the line mesh

PhAST creates 10 two-node axial elements and 11 nodes. Each node has one axial displacement. Within an element of length h , displacement varies linearly. PhAST assembles the element stiffness

$$K_e = \frac{EA}{h} \begin{bmatrix} 1 & -1 \\ -1 & 1 \end{bmatrix}.$$

Code cell 10

```
mesh = phast.line_mesh(length=length, n_elements=10)
x = mesh.nodes.detach().numpy()
print("Coordinates [mm]:", mesh.nodes.tolist())
print("Element connectivity:")
print(mesh.elements)
```

```
Coordinates [mm]: [0.0, 10.0, 20.0, 30.0, 40.0, 50.0, 60.0, 70.0, 80.0, 90.0, 100.0]
Element connectivity:
tensor([[ 0,  1],
        [ 1,  2],
        [ 2,  3],
        [ 3,  4],
        [ 4,  5],
        [ 5,  6],
        [ 6,  7],
        [ 7,  8],
        [ 8,  9],
        [ 9, 10]])
```

Code cell 11

3. Generate the line mesh

```
fig, ax = plt.subplots(figsize=(8, 2.2), layout="constrained")
ax.plot(x, np.zeros_like(x), "o-", color="#2980B9", lw=2)
for node, position in enumerate(x):
    ax.text(position, .09, str(node), ha="center")
ax.set(xlim=(-5, 105), ylim=(-.12, .28), xlabel="Position x [mm]",
       title="Ten elements; eleven nodes", yticks=[])
show_figure(fig, "02_mesh")
```

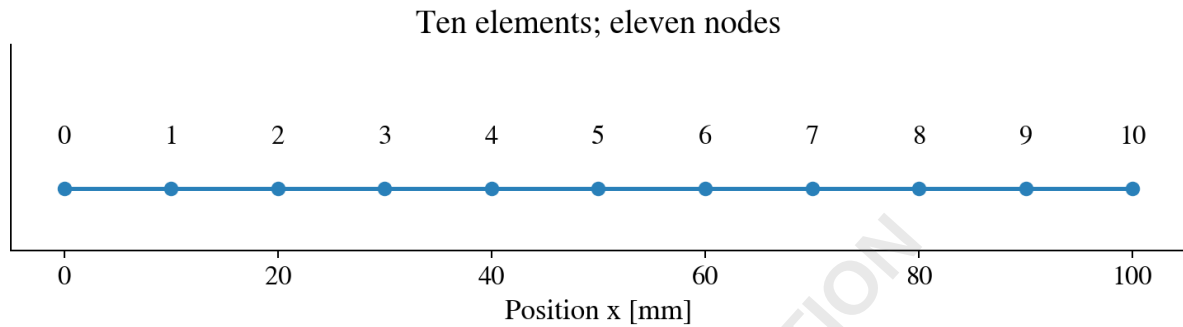


Fig. 15: Saved output from code cell 11.

4. Boundary Condition and Loads

Fix the left end: $u(0) = 0$.

Apply force $F = 4000$ N to the right end, directed along positive x .

$$u(0) = 0$$

$$EA \frac{du}{dx} \Big|_{x=L} = F.$$

Code cell 13

```
left_displacement = 0.0 # mm
force = 4000.0 # N; held fixed in every forward evaluation
```

Code cell 14

```
fig, ax = plt.subplots(figsize=(8, 2.4), layout="constrained")
ax.plot(x, np.zeros_like(x), "o-", color="#2980B9", lw=3)
ax.plot([0, 0], [-.18, .18], color="#222222", lw=2)
for y in np.linspace(-.16, .16, 5):
    ax.plot([-3, 0], [y-.05, y], color="gray", lw=.8)
ax.annotate("", (118, 0), (100, 0), arrowprops={"arrowstyle": "->", "color": "#C0392B", "lw": 2})
ax.text(111, .17, r"$F=4000\,\mathrm{N}$", ha="center", color="#C0392B")
ax.text(0, .24, r"$u(0)=0$", ha="center")
ax.set(xlim=(-8, 135), ylim=(-.32, .48), title="Fixed left end; tensile force at the right end")
ax.axis("off")
show_figure(fig, "03_boundary_force")
```

Fixed left end; tensile force at the right end



Fig. 16: Saved output from code cell 14.

5. Write the forward function

PhAST assembles the axial stiffness, and solves static equilibrium:

$$\mathbf{K} * (\mathbf{E})\mathbf{u} = \mathbf{f}$$

The subscript f denotes free displacement degrees of freedom. This 1D problem uses PhAST's differentiable sparse solve with a SciPy SuperLU CPU backend.

One call solves the linear system directly.

Returning the result as tensors preserves the connection between E and the displacements for autograd.

Code cell 16

```
def forward(E):
    return phast.solve_bar(
        mesh,
        young_modulus=E,
        area=area,
        end_force=force,
        left_displacement=left_displacement,
    )

reference = forward(E_reference)
u_reference = reference.displacement.detach()
print(f"Tip displacement: {u_reference[-1].item():.6f} mm")
print(f"Support reaction: {reference.reaction.item():.1f} N")
print(f"Maximum free-node residual: {reference.free_residual.abs().max().item():.3e} N")
```

```
Tip displacement: 0.190476 mm
Support reaction: -4000.0 N
Maximum free-node residual: 1.091e-11 N
```

```
/usr/local/lib/python3.13/dist-packages/phast/bar.py:88: UserWarning: Sparse invariant checks
are implicitly disabled. Memory errors (e.g. SEGFAULT) will occur when operating on a sparse
tensor which violates the invariants, but checks incur performance overhead. To silence this
warning, explicitly opt in or out. See `torch.sparse.check_sparse_tensor_invariants.__doc__`
for guidance. (Triggered internally at /pytorch/aten/src/ATen/Context.cpp:760.)
K = torch.sparse_coo_tensor(indices, values, (n, n)).coalesce()
```

6. Record and visualise the forward solution

The displacement increases linearly from the fixed end to the loaded end. Strain and stress are uniform. For this bar, the independent analytical solution is

$$u(x) = \frac{Fx}{EA}, \quad u(L) = 0.190476 \text{ mm.}$$

We can use this analytical formula to check PhAST's output.

Code cell 18

```
print("Node    x [mm]    u [mm]")
for node, (position, displacement) in enumerate(zip(x, u_reference)):
    print(f"{node:4d}    {position:6.1f}    {displacement.item():.8f}")

with (output / "forward_nodes.csv").open("w", newline="") as stream:
    writer = csv.writer(stream)
    writer.writerow(["node", "x_mm", "u_mm"])
    writer.writerows((i, float(xi), float(ui)) for i, (xi, ui) in enumerate(zip(x, u_
reference)))
```

Node	x [mm]	u [mm]
0	0.0	0.00000000
1	10.0	0.01904762
2	20.0	0.03809524
3	30.0	0.05714286
4	40.0	0.07619048
5	50.0	0.09523810
6	60.0	0.11428571
7	70.0	0.13333333
8	80.0	0.15238095
9	90.0	0.17142857
10	100.0	0.19047619

Code cell 19

```
stress = reference.stress.detach().numpy()

fig, axes = plt.subplots(1, 2, figsize=(10, 3.4), layout="constrained")

axes[0].plot(x, u_reference.numpy(), "o-", color="#2980B9")
axes[0].set(xlabel="Position $x$ [mm]", ylabel="Displacement $u$ [mm]",
            title="Nodal displacements", xlim=(-5, 105))

axes[1].stairs(stress, x, baseline=0, fill=True, fc="#C0392B", alpha=0.30, ec="#C0392B", lw=1.6)
axes[1].ticklabel_format(axis="y", style="plain", useOffset=False)
axes[1].set(xlabel="Position $x$ [mm]", ylabel=r"Stress $\sigma$ [MPa]",
            title="Axial stress (constant within each element)",
            xlim=(-5, 105), ylim=(0, 1.25 * stress.max()))
axes[1].annotate(rf"$\sigma = F/A = \{stress[0]:.0f\}$ MPa",
                xy=(50, stress[0]), xytext=(0, 12), textcoords="offset points",
                ha="center", color="#C0392B", fontsize=11)

for ax in axes:
    ax.grid(axis="y", color="#E8E8E8", lw=.6)
    ax.set_axisbelow(True)

show_figure(fig, "04_forward_fields")
```

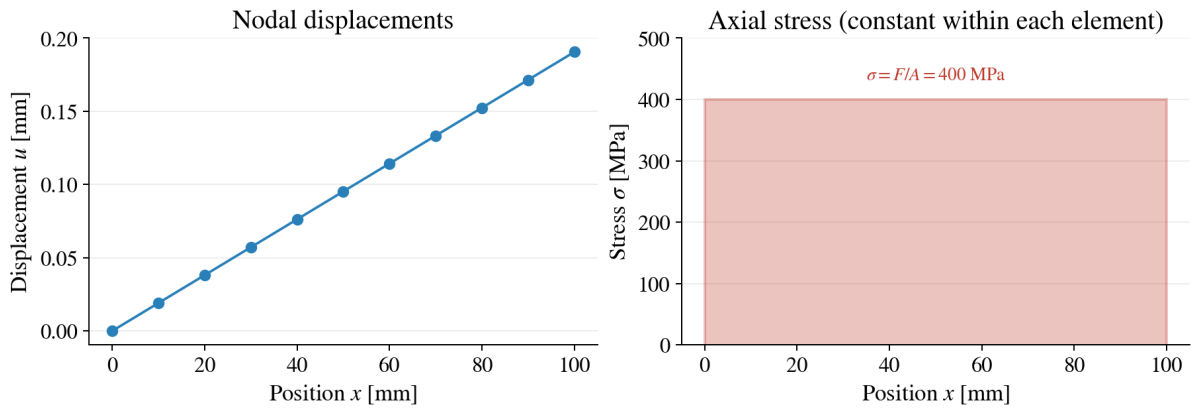


Fig. 17: Saved output from code cell 19.

See the actual extension

The plotted displacement is magnified 40 times so the extension is visible.

Code cell 21

```
magnification = 40
deformed_x = x + magnification * u_reference.numpy()
fig, ax = plt.subplots(figsize=(8, 3), layout="constrained")
ax.plot(x, np.ones_like(x), "o--", color="#888888", lw=2)
ax.plot(deformed_x, np.zeros_like(x), "o-", color="#2980B9", lw=3)
ax.plot([0, 0], [-.22, 1.22], color="#222222", lw=2)
ax.annotate("", (deformed_x[-1]+12, 0), (deformed_x[-1]+1, 0),
            arrowprops={"arrowstyle": ">", "color": "#C0392B", "lw": 2})
ax.text(50, 1.25, "Undeformed", ha="center", color="#777777")
ax.text(50, -.3, "Deformed: displacement magnified 40x", ha="center", color="#2980B9")
ax.text(deformed_x[-1]+6, .2, "4000 N", ha="center", color="#C0392B")
ax.set(xlim=(-4, 128), ylim=(-.6, 1.6), title="A tensile force extends the bar")
ax.axis("off")
show_figure(fig, "09_deformed_bar")
```

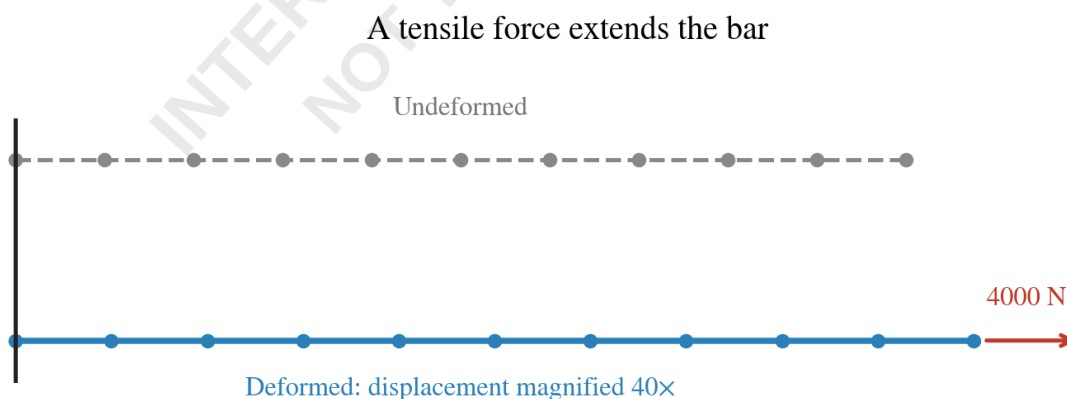


Fig. 18: Saved output from code cell 21.

Strain and stored elastic energy

The axial strain is uniform because this uniform bar carries the same axial force everywhere. Each equal-length element stores the same elastic energy:

$$\varepsilon_e = \frac{u_{e+1} - u_e}{h_e}, \quad U_e = \frac{1}{2} E \varepsilon_e^2 A h_e.$$

The plots below are computed from the same forward solve, with force fixed at 4000 N.

Code cell 23

```
strain = reference.strain.detach().numpy()
stress = reference.stress.detach().numpy()
lengths = mesh.element_lengths.numpy()

# Strain energy per element: U_e = 1/2 * sigma * eps * A * h
element_energy = 0.5 * stress * strain * area * lengths
total_energy = element_energy.sum()
external_work = 0.5 * force * reference.displacement[-1].item()

fig, axes = plt.subplots(1, 2, figsize=(10, 3.5), layout="constrained")

axes[0].stairs(strain * 100, x, baseline=0, fill=True, fc="#2980B9", alpha=0.30, ec="#2980B9",
               lw=1.6)
axes[0].ticklabel_format(axis="y", style="plain", useOffset=False)
axes[0].set(xlabel="Position $x$ [mm]", ylabel="Axial strain [%]",
            title="Uniform extension per unit length",
            xlim=(-5, 105), ylim=(0, 1.25 * (strain * 100).max()))
axes[0].annotate(rf"$\varepsilon = \{strain[0]*100:.3f\} \%", xy=(50, strain[0] * 100), xytext=(0, 12), textcoords="offset points",
                ha="center", color="#2980B9", fontsize=11)

axes[1].bar(np.arange(1, len(element_energy) + 1), element_energy, width=.55, color="#27AE60")
axes[1].set(xlabel="Element number", ylabel="Elastic energy [N mm]",
            title="Energy stored in each element",
            xticks=np.arange(1, len(element_energy) + 1),
            ylim=(0, 1.3 * element_energy.max()))
axes[1].annotate(f"total $U = \{total\_energy:.2f\}$ N mm",
                rf"external work $\frac{1}{2} F u(L) = \{external\_work:.2f\}$ N mm",
                xy=(0.5, 0.92), xycoords="axes fraction", ha="center", va="top", fontsize=10)

for ax in axes:
    ax.grid(axis="y", color="#E8E8E8", lw=.6)
    ax.set_axisbelow(True)

show_figure(fig, "10_strain_energy")
```

7. Define the observation and inverse question

We retain one scalar observation, the tip displacement from the reference solve.

Detaching it treats it as a fixed measurement during differentiation.

##Inverse question: Based on the given extension and load, estimate Young's Modulus (E)

Start from $E_0 = 100$ GPa.

This softer trial bar extends too much.

$$y(E) = u(L; E)$$

$$r(E) = y(E) - y^{\text{obs}},$$

$$L(E) = \left(\frac{r(E)}{y^{\text{obs}}} \right)^2.$$

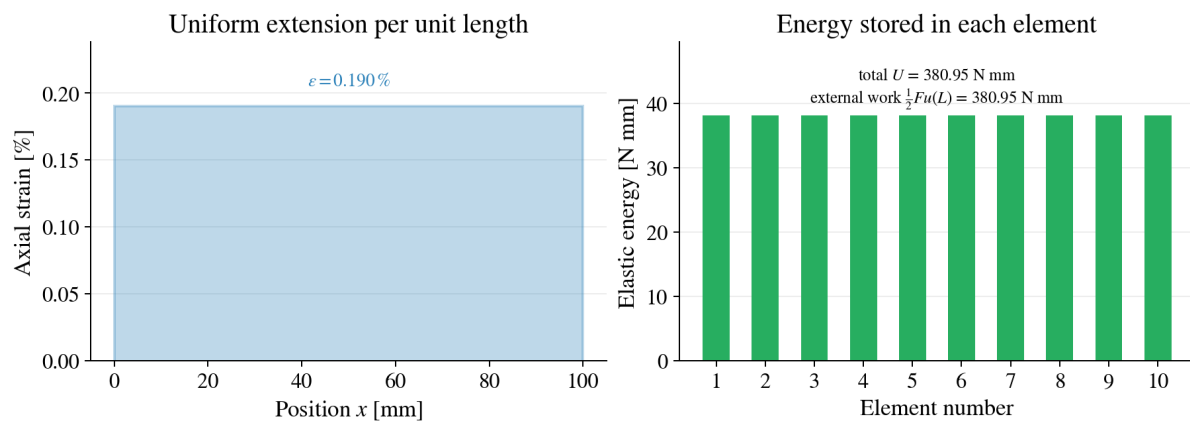


Fig. 19: Saved output from code cell 23.

The dimensionless loss is the squared relative displacement mismatch.
Zero loss means the predicted and observed tip displacements coincide.

Code cell 25

```
observed = u_reference[-1].clone()
E_initial = torch.tensor(100000.0)

def loss_from_tip(tip):
    return ((tip - observed) / observed).square()

initial = forward(E_initial)
print(f"Observed tip: {observed.item():.6f} mm")
print(f"Initial tip: {initial.displacement[-1].item():.6f} mm")
print(f"Initial loss: {loss_from_tip(initial.displacement[-1]).item():.6f}")
```

```
Observed tip: 0.190476 mm
Initial tip: 0.400000 mm
Initial loss: 1.210000
```

Code cell 26

```
fig, ax = plt.subplots(figsize=(7, 3.4), layout="constrained")
ax.plot(x, initial.displacement.detach().numpy(), "o-", color="#2980B9", label="Initial: 100_
    GPa")
ax.plot(x, u_reference.numpy(), "--", color="#777777", label="Reference field")
ax.plot([length], [observed.item()], "o", color="#C0392B", ms=8, label="Observation used in_
    loss")
ax.set(xlabel="Position x [mm]", ylabel="Displacement u [mm]", title="The softer trial bar_
    extends too much")
ax.legend(frameon=False)
show_figure(fig, "05_initial_prediction")
```

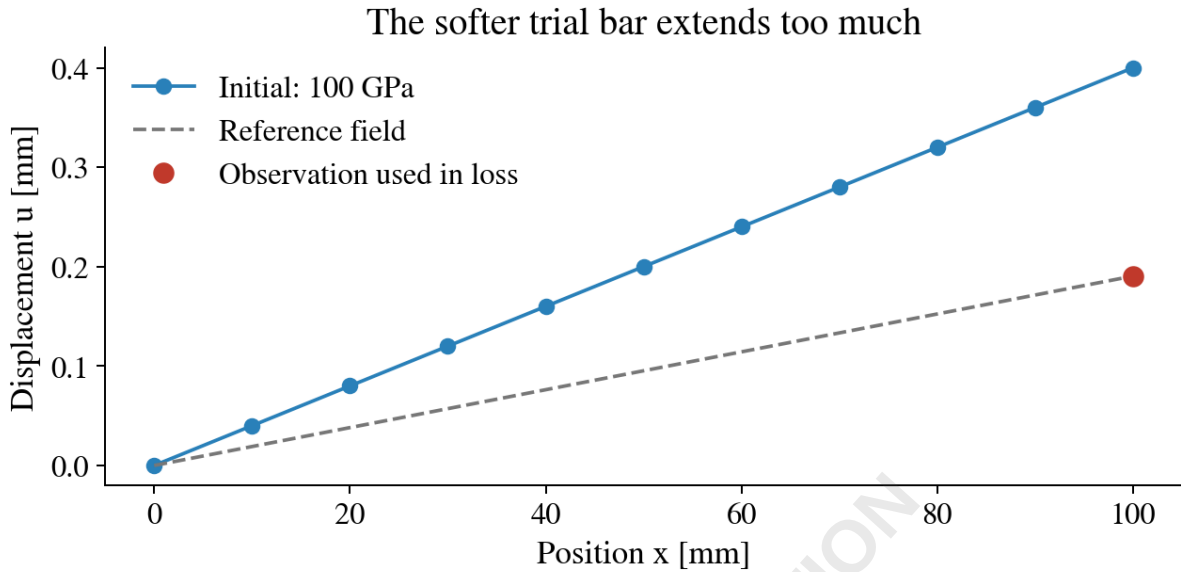


Fig. 20: Saved output from code cell 26.

8. Differentiate through the PhAST solve

Mark E with `requires_grad=True` and `autograd` records every tensor operation that consumes it. The chain it walks backwards is

$$E \longrightarrow K(E) \longrightarrow u(E) \longrightarrow y(E) \longrightarrow L(E)$$

with K the assembled stiffness, u the displacement field, $y = u(L)$ the tip extension we observe, and L the scalar mismatch.

The awkward link is the solve. Differentiating $Ku = f$ at fixed force,

$$K \frac{\partial u}{\partial E} = -\frac{\partial K}{\partial E} u$$

Same matrix, new right-hand side — the sensitivity reuses the forward factorisation and costs one back-substitution. This is what PhAST's `SparseSolveAutograd` implements.

The same derivative by hand

L depends on E only through y , so the chain rule splits it in two:

$$\frac{dL}{dE} = \frac{dL}{dy} \cdot \frac{dy}{dE} = \frac{2(y - y^{\text{obs}})}{(y^{\text{obs}})^2} \cdot \left(-\frac{y}{E}\right)$$

The first factor is the loss we chose, differentiated with y^{obs} held fixed. The second is the mechanics:

$$y = FL/EA = (FL/A)E^{-1}$$

$$dy/dE = -y/E$$

Both derivatives are negative at the initial guess, and both signs are physically obvious once stated: a stiffer bar stretches less, and the trial bar currently stretches too much, so increasing E reduces the mismatch.

Code cell 28

```

fig, ax = plt.subplots(figsize=(10, 3.2), layout="constrained")
locations = [0, 2, 4, 6, 8]
symbols = [r"$E$", r"${\mathbf{K}}(E)$", r"${\mathbf{u}}(E)$", r"$y(E)$", r"$L$"]
labels = ["Modulus", "Stiffness", "Displacements", "Tip prediction", "Loss"]
for pos, symbol, label in zip(locations, symbols, labels):
    ax.text(pos, .75, symbol, ha="center", fontsize=24, color="#2980B9")
    ax.text(pos, .38, label, ha="center", fontsize=13)
for a, b in zip(locations[:-1], locations[1:]):
    ax.annotate("", (b-.55, .8), (a+.55, .8),
                arrowprops={"arrowstyle": "->", "lw": 1.4, "color": "#222222"})
ax.annotate("", (0, -.05), (8, -.05),
            arrowprops={"arrowstyle": "->", "lw": 1.7, "color": "#C0392B"})
ax.text(4, -.35, "Backward: chain rule propagates loss sensitivity to E",
        ha="center", color="#C0392B")
ax.set(xlim=(-.9, 8.9), ylim=(-.65, 1.65),
        title="Forward: predict the extension and evaluate the mismatch")
ax.axis("off")
show_figure(fig, "11_autograd_graph")

```

Forward: predict the extension and evaluate the mismatch

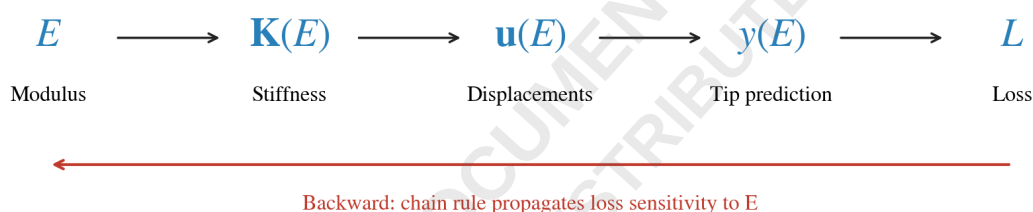


Fig. 21: Saved output from code cell 28.

Code cell 29

```

# requires_grad_(True) tells PyTorch to record every operation that consumes E,
# building the graph. Nothing is differentiated until we ask for a gradient.
E_probe = E_initial.detach().clone().requires_grad_(True)

tip_probe = forward(E_probe).displacement[-1]
loss_probe = loss_from_tip(tip_probe)

# loss_probe is downstream of tip_probe: one shared graph. Retain it on the first
# call, or the second raises "backward through the graph a second time".
dy_dE = torch.autograd.grad(tip_probe, E_probe, retain_graph=True)[0]
dL_dE = torch.autograd.grad(loss_probe, E_probe)[0]

# --- independent hand derivative, valid because this bar has y = FL/EA ---
assert tip_probe.shape == observed.shape == torch.Size([1]), \
    "this check assumes a single load case"
expected_dy = -tip_probe.detach() / E_probe.detach() # dy/dE = -y/E
expected_dL = 2 * (tip_probe.detach() - observed) / observed.square() * expected_dy

print(f"{'quantity':<10}{ 'autograd':>17}{ 'analytical':>17}{ 'rel. error':>13}  units")
for name, ad, an, unit in (("dy/dE", dy_dE, expected_dy, "mm/MPa"),
                           ("dL/dE", dL_dE, expected_dL, "1/MPa")):
    rel = abs((ad - an) / an).item()
    print(f"{'name':<10}{ad.item():>17.8e}{an.item():>17.8e}{rel:>13.1e}  {unit}")

```

(continues on next page)

(continued from previous page)

```

assert torch.allclose(dy_dE, expected_dy, rtol=1e-12, atol=0.0), (
    f"du/dE mismatch: autograd {dy_dE.item():.6e} vs analytical {expected_dy.item():.6e}. "
    "Did the mesh, area or force change since `observed` was recorded?"
)
assert torch.allclose(dL_dE, expected_dL, rtol=1e-12, atol=0.0), (
    f"dL/dE mismatch: autograd {dL_dE.item():.6e} vs analytical {expected_dL.item():.6e}. "
    "Check that loss_from_tip still matches the expression derived above."
)
print("\nPhAST autograd agrees with the analytical derivative to machine precision.")
print("Both are negative: a stiffer bar stretches less, so increasing E reduces the mismatch.
↵")

```

quantity	autograd	analytical	rel. error	units
dy/dE	-4.00000000e-06	-4.00000000e-06	2.1e-16	mm/MPa
dL/dE	-4.62000000e-05	-4.62000000e-05	1.6e-15	1/MPa

PhAST autograd agrees with the analytical derivative to machine precision.
Both are negative: a stiffer bar stretches less, so increasing E reduces the mismatch.

9. Set up a positive modulus and momentum updates

We optimise a dimensionless logarithmic parameter z :

$$E = (1 \text{ MPa})e^z, \quad \frac{dL}{dz} = E \frac{dL}{dE}.$$

This parameterisation keeps E positive. We use gradient descent with learning rate $\eta = 0.1$ and momentum $\beta = 0.5$:

$$m_k = \beta m_{k-1} + \frac{dL}{dz}(z_k), \quad z_{k+1} = z_k - \eta m_k, \quad m_{-1} = 0.$$

Momentum carries information from previous gradients. It can produce an overshoot followed by diminishing corrections. The plotted oscillations will come from actual solver evaluations.

Stop after the relative tip mismatch is below 10^{-4} (0.01%) for five consecutive evaluations, with a maximum of 80 updates. The reference modulus is used only for later assessment.

Code cell 31

```

log_E = torch.nn.Parameter(torch.log(E_initial.clone()))
optimizer = torch.optim.SGD([log_E], lr=0.1, momentum=0.5)
relative_tolerance = 1e-4
required_consecutive = 5
max_updates = 80
history, inverse_fields = [], []
consecutive = 0

```

10. Run the inverse loop

Each evaluation uses the same mesh, support, force and observation.

1. Solve the bar at the current modulus.
2. Record its displacement and loss.
3. Differentiate the loss with `backward()`.
4. Let the optimiser update the log modulus.

`zero_grad()` clears the previous gradient. `detach()` is used for storing results; the loss used by `backward()` stays connected to the solve.

Code cell 33

```

inverse_start = perf_counter()
for update in range(max_updates + 1):
    optimizer.zero_grad(set_to_none=True)
    E = log_E.exp()
    result = forward(E)
    loss = loss_from_tip(result.displacement[-1])
    history.append([update, E.item(), result.displacement[-1].item(), loss.item()])
    inverse_fields.append(result.displacement.detach().clone())
    consecutive = consecutive + 1 if loss.item() < relative_tolerance**2 else 0
    if consecutive >= required_consecutive or update == max_updates:
        break
    loss.backward()
    optimizer.step()

inverse_seconds = perf_counter() - inverse_start
E_recovered = torch.tensor(history[-1][1])
print(f"Recovered E: {E_recovered.item()/1000:.6f} GPa")
print(f"Updates: {history[-1][0]}; forward evaluations in recovery: {len(history)}")
print(f"Recovery-loop time on this machine: {inverse_seconds:.4f} s")

```

```

Recovered E: 209.990865 GPa
Updates: 30; forward evaluations in recovery: 31
Recovery-loop time on this machine: 0.0591 s

```

11. Read the recovery history

The blue curve shows the modulus after each inverse update. The dashed line is the known reference used to generate the synthetic observation.

The red curve shows the same run's displacement loss on a logarithmic vertical axis. Equal vertical distances represent equal factors of reduction. Dips occur when the trial modulus passes close to the correct value.

Overshoot describes the optimiser's path; it is not physical vibration of the bar. Iteration count depends on parameterisation, step size, momentum and the stopping rule.

Code cell 35

```

records = np.asarray(history)
iterations, moduli, tips, losses = records.T
fig, axes = plt.subplots(1, 2, figsize=(10, 3.8), layout="constrained")
axes[0].plot(iterations, moduli/1000, color="#2980B9", lw=2, label="Trial modulus")
axes[0].axhline(E_reference.item()/1000, color="#888888", ls="--", label="Reference")
axes[0].set(xlabel="Inverse update", ylabel="E [GPa]", title="Recover the elastic modulus")
axes[0].legend(frameon=False)
axes[1].semilogy(iterations, losses, color="#C0392B", lw=2)
axes[1].set(xlabel="Inverse update", ylabel=r"Normalised loss $L$",
            title="Match the observed extension")
for ax in axes:
    ax.grid(axis="y", color="#E8E8E8", lw=.6)
show_figure(fig, "06_recovery_history")

```

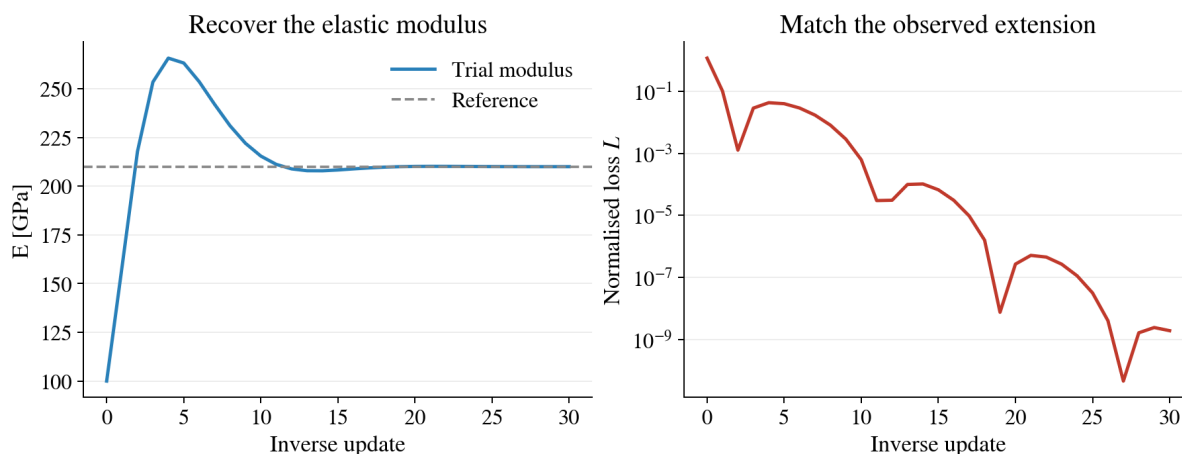


Fig. 22: Saved output from code cell 35.

Animation showing the inverse prediction updates

Each frame displays a recorded displacement field from an inverse evaluation. The applied force is constant. This animation shows how the predicted extension changes as the modulus estimate changes.

The red dot is the single observation used in the loss. The grey reference field is shown for interpretation.

Code cell 37

```
from matplotlib.animation import FuncAnimation, PillowWriter

fields = torch.stack(inverse_fields).numpy()
fig, ax = plt.subplots(figsize=(7, 3.5), layout="constrained")
ax.plot(x, u_reference.numpy(), "--", color="#888888", label="Reference field")
ax.plot([length], [observed.item()], "o", color="#C0392B", label="Observed tip")
line, = ax.plot([], [], "o-", color="#2980B9", label="Current prediction")
ax.set(xlim=(0, length), ylim=(0, fields.max()*1.08),
       xlabel="Position x [mm]", ylabel="Displacement u [mm]")
ax.legend(frameon=False, loc="upper left")
title = ax.set_title("")

def animate(frame):
    line.set_data(x, fields[frame])
    title.set_text(f"Inverse update {int(iterations[frame])}: E = {moduli[frame]/1000:.2f} GPa")
    return line, title

animation = FuncAnimation(fig, animate, frames=len(history), interval=160, blit=False)
animation.save(output / "07_inverse_prediction.gif", writer=PillowWriter(fps=6), dpi=100)
plt.close(fig)
import base64
encoded_gif = base64.b64encode((output / "07_inverse_prediction.gif").read_bytes()).decode("ascii")
display(HTML(f''))
```

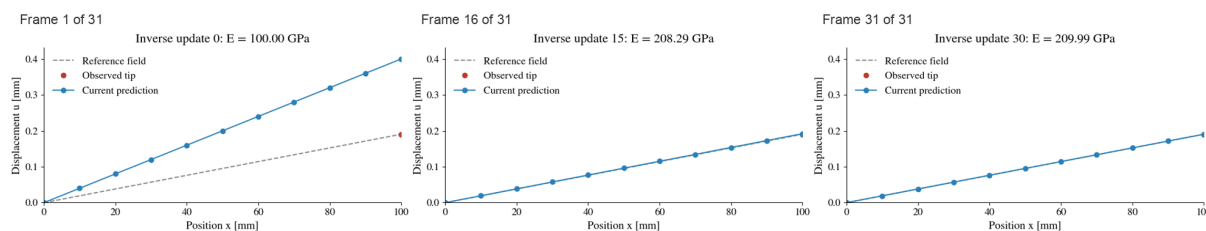


Fig. 23: Three stages from the saved animation (31 frames). [Play the animation online.](#)

12. Check the recovered solution and save the results

The final check uses the same 4000 N force. The forward reference, initial trial and recovered trial can be compared directly.

For this uniform bar, $E = FL/(Ay^{\text{obs}})$ also gives an analytical inverse solution. The purpose of the iterative example is to learn the differentiable-solver workflow, which extends to problems without that closed-form inverse.

The observation is synthetic and noise-free, generated with the same model. With known L and A , one nonzero force suffices for one unknown uniform modulus. Autograd supplies gradients; these results do not establish a speed advantage over other recovery methods.

Code cell 39

```
recovered = forward(E_recovered)
u_recovered = recovered.displacement.detach()
tip_error = abs(u_recovered[-1].item()/observed.item() - 1)
E_error = abs(E_recovered.item()/E_reference.item() - 1)
assert torch.allclose(u_reference, force*mesh.nodes/(E_reference*area), rtol=1e-10, atol=1e-12)
assert abs(reference.reaction.item() + force) < 1e-6
assert reference.free_residual.abs().max().item() < 1e-6
assert consecutive >= required_consecutive, "Increase the update budget or review optimiser settings."
assert tip_error < relative_tolerance and E_error < 2*relative_tolerance
print(f"Observed / recovered tip: {observed.item():.8f} / {u_recovered[-1].item():.8f} mm")
print(f"Relative modulus error: {100*E_error:.6f}%")
print("All forward, derivative and recovery checks passed.")
```

```
Observed / recovered tip: 0.19047619 / 0.19048448 mm
Relative modulus error: 0.004350%
All forward, derivative and recovery checks passed.
```

Code cell 40

```
fig, ax = plt.subplots(figsize=(7, 3.4), layout="constrained")
ax.plot(x, u_reference.numpy(), "--", color="#888888", lw=2, label="Reference field")
ax.plot(x, u_recovered.numpy(), "o", color="#2980B9", label="Recovered PhAST field")
ax.plot([length], [observed.item()], "o", color="#C0392B", ms=8, label="Observed tip")
ax.set(xlabel="Position x [mm]", ylabel="Displacement u [mm]", title="Recovered extension at the same force")
ax.legend(frameon=False)
show_figure(fig, "08_recovered_field")

print("Node    x [mm]    Reference u    Recovered u [mm]")
for i, (xi, ur, uf) in enumerate(zip(x, u_reference, u_recovered)):
    print(f"{i:4d}    {xi:6.1f}    {ur.item():.8f}    {uf.item():.8f}")
```

Node	x [mm]	Reference u	Recovered u [mm]
0	0.0	0.00000000	0.00000000

(continues on next page)

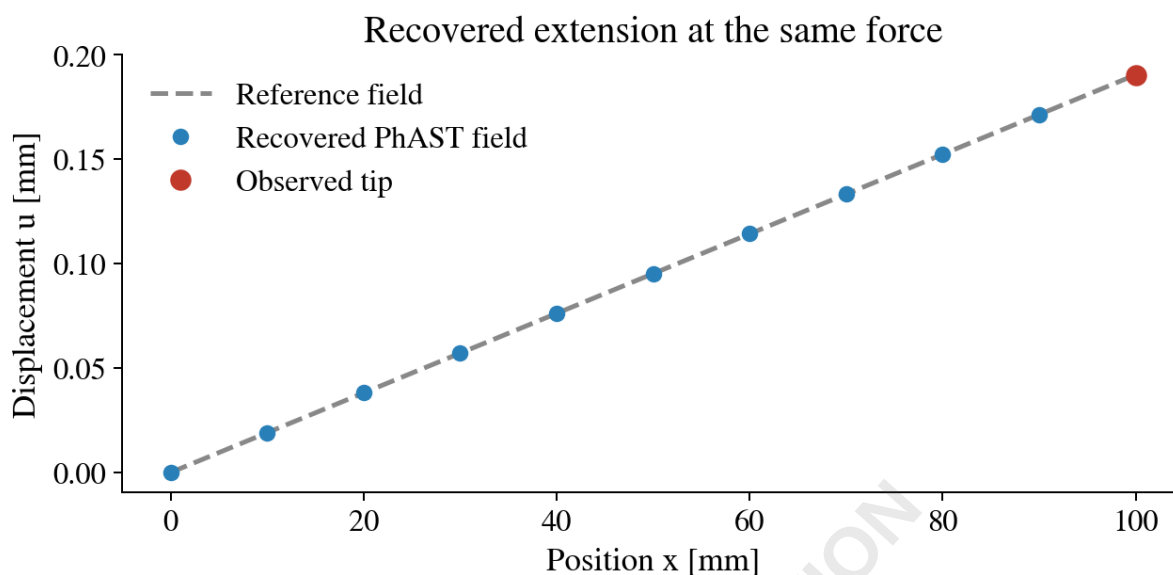


Fig. 24: Saved output from code cell 40.

(continued from previous page)

1	10.0	0.01904762	0.01904845
2	20.0	0.03809524	0.03809690
3	30.0	0.05714286	0.05714534
4	40.0	0.07619048	0.07619379
5	50.0	0.09523810	0.09524224
6	60.0	0.11428571	0.11429069
7	70.0	0.13333333	0.13333913
8	80.0	0.15238095	0.15238758
9	90.0	0.17142857	0.17143603
10	100.0	0.19047619	0.19048448

Code cell 41

```

with (output / "inverse_history.csv").open("w", newline="") as stream:
    writer = csv.writer(stream)
    writer.writerow(["inverse_update", "E_MPa", "tip_mm", "normalised_loss"])
    writer.writerows(history)
with (output / "recovered_nodes.csv").open("w", newline="") as stream:
    writer = csv.writer(stream)
    writer.writerow(["node", "x_mm", "reference_u_mm", "recovered_u_mm"])
    writer.writerows((i, float(xi), float(ur), float(uf))
                      for i, (xi, ur, uf) in enumerate(zip(x, u_reference, u_recovered)))
summary = {
    "force_N": force, "observation_mm": observed.item(),
    "optimizer": "SGD with momentum", "learning_rate": 0.1, "momentum": 0.5,
    "updates": int(iterations[-1]), "recovery_forward_evaluations": len(history),
    "recovered_E_GPa": E_recovered.item()/1000,
    "relative_E_error": E_error, "relative_tip_error": tip_error,
    "inverse_seconds": inverse_seconds,
    "lesson_seconds_excluding_installation": perf_counter() - lesson_start,
    "phast_revision": "157412953099bfcace1668bb35cb911826a4e95e",
    "checks_passed": True,
}
(output / "results.json").write_text(json.dumps(summary, indent=2))
print(json.dumps(summary, indent=2))
print("Figures, animation and numerical tables:", output.resolve())

```

```
{
  "force_N": 4000.0,
  "observation_mm": 0.19047619047618994,
  "optimizer": "SGD with momentum",
  "learning_rate": 0.1,
  "momentum": 0.5,
  "updates": 30,
  "recovery_forward_evaluations": 31,
  "recovered_E_GPa": 209.99086454579844,
  "relative_E_error": 4.350216286452824e-05,
  "relative_tip_error": 4.350405538677826e-05,
  "inverse_seconds": 0.05914312299989888,
  "lesson_seconds_excluding_installation": 28.26430550500004,
  "phast_revision": "157412953099bfcace1668bb35cb911826a4e95e",
  "checks_passed": true
}
Figures, animation and numerical tables: /content/bar_results
```

13. Recap:

The sequence was:

geometry → mesh → boundary condition and load → forward solve → record observations → reuse *the same* forward function inside a loss → autograd → update E

The forward solver was never modified. The inverse problem was built entirely *around* it, and the only thing that made that possible is that PhAST's sparse solve carries a backward rule.

Please note, for this exercise:

- The data were generated by the same model we then fitted, and contain no noise. This is verification, not validation against a real experiment.
- We demonstrated a single scalar parameter with one perfectly identifiable quantity. Real calibration problems are ill-posed, and that is where regularisation and uncertainty quantification enter.

Fun exercises:

1. Change the starting guess to 500 GPa.
2. Use a different optimiser like Adam or LBFGS; Try different optimisation step sizes $\text{lr}=0.3$ and $\text{lr}=0.005$. How many iterations do they take?
3. Add noise. `observed` is a single scalar, so it has no spread of its own to scale by. Apply a fixed relative perturbation instead, keeping the result zero-dimensional so the rest of the notebook still works: `noisy = observed * (1 + 0.002 * torch.randn(()))`. Rerun the recovery several times and report the spread of the recovered E . With one noisy observation and one unknown, can that spread be reduced by tightening the stopping rule alone?
4. Two unknowns. Make both E and A trainable. Watch the optimiser find *some* pair with the right product EA , not the right pair.

Key takeaways

- The force remains 4000 N throughout the inverse loop.
- Autograd differentiates the current forward solve and scalar loss.
- The optimiser selects the next modulus; each update requires another forward evaluation.

Consolidation

What happens to the tip displacement when E doubles at the same force?

Hint

Use $u(L) = F L / (E A)$.

Conceptual answer

For this uniform linear-elastic bar, the displacement halves. The analytical relation also provides an independent check of the PhAST result.

INTERNAL DOCUMENTATION
NOT TO BE DISTRIBUTED

Lecture 3: Learned damage updates in PhAST

A trained network can replace one operation inside a finite-element calculation. The central questions are what it receives, what it predicts, and whether the resulting fields remain accurate. NB3 compares conventional FEM with a frozen graph neural network that directly predicts damage. Mechanics remains in PhAST.

The same physical problem, two damage updates

The specimen is a 10 mm square plate with a horizontal edge slot and three holes. The bottom boundary is fixed. The top boundary has zero horizontal displacement and an increasing vertical displacement, ending at 0.04 mm. The slot and hole surfaces are traction-free geometric boundaries, not regions of prescribed damage.

The material has $E = 210000$ MPa, $\nu = 0.3$, $G_c = 2.7$ N/mm and $\ell_0 = 0.4$ mm. Both routes retain the checkpoint-specific rational degradation law

$$g(d) = \frac{(1-d)^2}{d^2 - d + 1}, \quad 0 \leq d \leq 1.$$

The retained model degrades the full elastic stress while using tensile energy to drive the history. This combination is a non-variational constitutive choice: it should not be presented as the variation of one common split-energy functional. NB1 uses a different constitutive model for its dynamic glass plate.

Read one load increment

The calculation is quasi-static: inertia is neglected. At increment n ,

$$d_{n-1} \longrightarrow u_n \longrightarrow H_n \longrightarrow d_n.$$

1. Solve mechanical equilibrium with the previous damage fixed.
2. Update the maximum-history driving field.
3. Solve the FEM damage equation, or predict damage with the GNN.
4. Project onto the damage bounds, measure the residual and advance the load.

Each increment uses one mechanics–damage pass. It is not an outer staggered iteration continued to coupled convergence. QUICK_TEST=True selects 60 increments; setting it to False selects 500 unless PHAST_DEMO_STEPS overrides the count. Changing that count changes the load discretisation, so compare both routes with the same setting and assess increment sensitivity.

Route	Damage operation	What follows
Conventional FEM	FEM damage solve	Projection and residual evaluation
Direct learned replacement	GNN damage prediction	Projection and residual evaluation; no FEM damage correction

Both routes enforce $d_n \geq d_{n-1}$ and $d_n \leq 1$. Projection enforces these bounds; it does not solve the damage equation. The learned residual is measured, not used to trigger an FEM fallback.

From a mesh to a graph

Every finite-element vertex is a graph node. Triangle edges define connections in both directions. The network receives an assembled history feature and a topological boundary indicator at each node, together with edge lengths scaled by the phase-field length. For the homogeneous linear triangles used here,

$$q_i = \sum_{e \ni i} \frac{A_e H_e}{3G_c \ell_0}.$$

The network transforms this feature as $\log_{10}(1 + q_i)$, applies an encoder, ten message-passing blocks and a decoder, and returns one damage value per node. The FEM field is reconstructed using the same nodal ordering:

$$d_h(x) = \sum_i N_i(x) d_i.$$

The notebook downloads `phast_gnn_assets.zip`, which contains the compatible PhAST source and `moon_pi_dd.pt`. It verifies the supplied hashes before loading. These weights were trained on an inclined-slot specimen and remain fixed here: the exercise tests transfer to a different geometry, not training.

Two comparisons answer different questions

Independent loading paths. The FEM and hybrid calculations start from independent initial states. Each GNN prediction influences its own next mechanics solve. Compare their damage fields at the same applied displacement, with common colour scales, and inspect reaction forces and residual histories.

Identical damage inputs. Replay stored FEM history and previous-damage fields through both damage updates. This isolates the replacement operation from differences that accumulate along the two loading paths.

The relative nodal field error is

$$e_d = \frac{\|d_{\text{GNN}} - d_{\text{FEM}}\|_2}{\|d_{\text{FEM}}\|_2}.$$

This unweighted nodal norm is not a mesh-independent integral error. A small field error and a small constrained residual are different checks. Reactions are evaluated after damage is updated without re-equilibrating displacement within that increment; interpret them in the context of the one-pass scheme.

Interpret performance without losing accuracy

The saved notebook includes fields, animations and measured comparisons. Damage-update timing includes inference or FEM solution, transfers, projection and residual evaluation. Complete-loop timing also includes mechanics, history, reactions and storage, but excludes installation, model loading and figures. Report the device and measurement scope with any ratio. Faster damage updates do not by themselves establish a faster or equally accurate complete simulation.

The optional [Radius-GNO case study](#) uses a different model and experiment. Its measurements are not results for this Moon-style graph-network exercise. The optional [Helmholtz tutorial](#) explains training and reloading a model; it is separate from this fracture application.

NB3 — Hybrid FEM+DL with PhAST

PhAST laboratory: fracture around three holes

Replacing the damage subproblem with a trained network

UKACM Autumn School 2026 · 16 September

Developed by Allamaprabhu Ani and Sathiskumar A. Ponnusami,

CEMS-Lab,

City St George's, University of London

Queen Mary University of London

[PhAST repository](#) · [PhAST documentation](#)

In this laboratory, we solve one fracture problem twice. First, finite elements calculate both displacement and damage. Second, finite elements still calculate displacement, but a frozen graph neural network predicts each damage update. We compare the evolving fields, reactions, residuals and computational cost.

Learning objectives

By the end of the laboratory, you should be able to:

1. Construct a geometry, generate a mesh and prescribe material and loading data.
2. Explain the mechanics–history–damage sequence in a staggered calculation.
3. Identify the graph inputs and the nodal damage prediction.
4. Distinguish field accuracy, equation residual and computational cost.
5. Measure damage-update cost without confusing it with complete simulation time.

[Download PhAST GNN assets ZIP](#)

Day 3 · PhAST practical. Updated 16 September 2026. Select a GPU runtime in Colab. The setup downloads `phast_gnn_assets.zip` automatically and verifies its source and checkpoint hashes. Figures and animations are retained from the supplied notebook; this publication does not represent a new numerical execution.

Code cell 3

```
# 1. Colab: select a GPU before execution
# Choose **Runtime → Change runtime type → GPU**, then connect.
# Python cannot allocate a GPU by installing PyTorch or setting a variable.
# Colab normally includes PyTorch; we preserve that CUDA installation.
# Official guidance: https://docs.pytorch.org/tutorials/beginner/colab
#
# Mechanics and the rational damage solver's sparse Newton factorization use
# CPU SciPy in this teaching implementation. The GNN can use the GPU. Moving
# a tensor to CUDA does not turn that sparse backend into a GPU solver.

import os
import sys
import subprocess
import importlib.util
from pathlib import Path

try:
    IN_COLAB = importlib.util.find_spec("google.colab") is not None
```

(continues on next page)

(continued from previous page)

```

except ModuleNotFoundError:
    IN_COLAB = False
if IN_COLAB:
    # Gmsh's Linux wheel needs the OpenGL utility library even without a GUI.
    subprocess.run(["apt-get", "-qq", "update"], check=True)
    subprocess.run(["apt-get", "-qq", "install", "-y", "libglu1-mesa"], check=True)
    subprocess.run([sys.executable, "-m", "pip", "-q", "install",
                    "gmsh==4.13.1", "meshio==5.3.5", "scipy", "matplotlib",
                    "numpy", "networkx", "tqdm", "pyyaml", "h5py", "pillow"], check=True)
if importlib.util.find_spec("torch") is None:
    subprocess.run([sys.executable, "-m", "pip", "install", "torch"], check=True)

import torch
DEVICE = "cuda" if torch.cuda.is_available() else "cpu"
print("PyTorch:", torch.__version__, "| GNN device:", DEVICE)
if DEVICE == "cuda":
    name = torch.cuda.get_device_name(0)
    capability = torch.cuda.get_device_capability(0)
    memory = torch.cuda.get_device_properties(0).total_memory / 1024**3
    print(f"GPU: {name} | compute capability {capability[0]} | {capability[1]} "
          f"| {memory:.1f} GiB")
    # The network runs in float32, so this GPU's float64 throughput does not
    # apply. That matters on a T4, whose float64 rate is 1/32 of its float32.
    print("Network precision: float32 (the FEM interface stays float64)")
else:
    print("CPU-only execution is supported; GPU speedup is not implied.")
# Keep CPU thread allocation identical for both simulations.
THREADS = min(2, os.cpu_count() or 1)
torch.set_num_threads(THREADS)

REQUIRE_GPU = True
if REQUIRE_GPU and not torch.cuda.is_available():
    raise RuntimeError("Select a GPU in Colab runtime settings, reconnect, and restart this
notebook. Set REQUIRE_GPU=False only for an intentional CPU exercise.")

```

```

PyTorch: 2.11.0+cu128 | GNN device: cuda
GPU: Tesla T4 | compute capability 7.5 | 14.6 GiB
Network precision: float32 (the FEM interface stays float64)

```

2. Load the source and frozen weights

The setup downloads `phast_gnn_assets.zip` automatically. You can also [download the archive](#) and place it beside the notebook.

Code cell 5

```

# 2. Make the original PhAST source available
# Upload `phast_gnn_assets.zip` when prompted. No HPC account is required.
# The archive includes the original Python source and the frozen checkpoint,
# not a newly trained model. Source hashes are checked before imports; the
# checkpoint hash is checked before the original loader deserializes it.
# Use the supplied source/checkpoint archive.

import zipfile
import hashlib
import json

# Set ASSETS_URL to a direct download link for phast_gnn_assets.zip and the

```

(continues on next page)

(continued from previous page)

```

# archive is fetched automatically, which avoids every student uploading the
# same 3 MB file by hand on every fresh runtime. Leave it empty to be prompted.
# Whatever arrives is still verified against the manifest hashes below, so a
# download is no less checked than an upload.
ASSETS_URL = "https://cems-lab.github.io/autumn-school/datasets/phast/phast_gnn_assets.zip"

HERE = Path(__file__).resolve().parent if "__file__" in globals() else Path.cwd()
ARCHIVE = HERE / "phast_gnn_assets.zip"
WORK = HERE / "phast_demo_work"

if not ARCHIVE.exists() and ASSETS_URL:
    import urllib.request
    print("Downloading", ASSETS_URL)
    urllib.request.urlretrieve(ASSETS_URL, ARCHIVE)
    print(f"Retrieved {ARCHIVE.stat().st_size / 1024:.0f} kB")

if not ARCHIVE.exists():
    for candidate in (Path("phast_gnn_assets.zip"),
                      Path("/content/drive/MyDrive/phast_gnn_assets.zip")):
        if candidate.exists():
            ARCHIVE = candidate
            print("Using the archive found at", candidate)
            break

if not ARCHIVE.exists() and IN_COLAB:
    from google.colab import files
    print("Select phast_gnn_assets.zip")
    files.upload()
    ARCHIVE = Path.cwd() / "phast_gnn_assets.zip"

if not ARCHIVE.exists():
    raise FileNotFoundError(
        "phast_gnn_assets.zip was not found. Set ASSETS_URL to a download link, "
        "place the archive beside this notebook, or upload it when prompted.")
WORK.mkdir(exist_ok=True)
with zipfile.ZipFile(ARCHIVE) as archive:
    # Reject paths outside the extraction folder.
    for item in archive.infolist():
        if not (WORK / item.filename).resolve().is_relative_to(WORK.resolve()):
            raise ValueError("Unsafe archive member")
    archive.extractall(WORK)
ASSETS = WORK / "assets"
manifest = json.loads((ASSETS / "manifest.json").read_text())
def sha256(path):
    return hashlib.sha256(Path(path).read_bytes()).hexdigest()
for name, expected in manifest["sha256"].items():
    assert sha256(ASSETS / name) == expected, f"File hash mismatch: {name}"
CHECKPOINT = ASSETS / "moon_pi_dd.pt"
CHECKPOINT_SHA = "953b29a05cc622b36e6174c713612b994c412263d84c0010505300cceb953cc1"
assert sha256(CHECKPOINT) == CHECKPOINT_SHA
# Install this exact PhAST snapshot into the interpreter's import search path.
# We do not substitute a similarly named package from PyPI.
sys.path[:0] = [str(ASSETS / "source" / "src"), str(ASSETS / "source"), str(ASSETS)]
print("Verified source files:", len(manifest["sha256"]))

# Geometry transfer with a frozen model
# The plate is 10 x 10 mm, with a horizontal edge slot of length 2 mm at y=5.
# Three holes have radius 0.55 mm and centres (4,4.4), (6,5.8), (8,4.6) mm.
# These are geometric voids with traction-free boundaries, not imposed damage.
# The checkpoint was trained on the inclined-slot specimen. Here its
# weights remain frozen: this is a geometry-transfer experiment. The original

```

(continues on next page)

(continued from previous page)

```

# rational material law is retained; this is not the separate Radius-GNO study.
# The mesh size remains 0.2 mm and the regularisation length remains 0.4 mm.
# PHAST_DEMO_STEPS may be set for a short installation test only.

import time
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.tri as mtri
from scripts.paper4.run_moon_full_fem_baseline import (
    BaselineConfig, FEMMesh, Material,
    BoundaryConditions, SolverConfig, StaggeredSolver, _node_sets,
)
from scripts.paper4.live_quasistatic_replacement import (
    build_live_adapter, moon_boundary_context,
)
from phast.solvers.damage_solver import damage_kkt_metrics
from arithmetic import count_call
from tqdm.auto import tqdm
from matplotlib.animation import FuncAnimation, PillowWriter
%matplotlib inline

# QUICK_TEST shortens the run for a first hardware check on a new GPU.
# Read the DAMAGE-UPDATE TIME RATIO in Section 16 for the hardware question:
# it replays identical states with a warm-up and CUDA synchronisation, so it is
# meaningful at any run length. The cumulative saving in Section 15 scales with
# the number of increments and is not comparable across different N_STEPS.
QUICK_TEST = True
N_STEPS = int(os.environ.get("PHAST_DEMO_STEPS", "60" if QUICK_TEST else "500"))
FINAL_DISPLACEMENT = 0.04 # mm; each route uses the same loading schedule
OUTPUT = WORK / "results_three_hole_classroom"
OUTPUT.mkdir(exist_ok=True)

```

Downloading https://cems-lab.github.io/autumn-school/datasets/phast/phast_gnn_assets.zip
 Retrieved 3105 kB
 Verified source files: 262

3. Define the geometry and generate the mesh

Quantity	Value
Plate	10 × 10 mm
Horizontal edge slot	Length 2 mm, centred at $y = 5$ mm; width 0.02 mm
Hole centres [mm]	(4, 4.4), (6, 5.8), (8, 4.6)
Hole radius	0.55 mm
Characteristic mesh size	0.2 mm
Elements	Three-node triangles

Gmsh subtracts the slot and holes from the plate before meshing.

These voids are part of the geometry: their interiors contain no material or finite elements. Generate the mesh once so both simulations use identical nodes and elements. The mesh and boundary-condition plot appears in Section 5.

Code cell 7

```

# 3. Define or load the geometry, then generate the finite-element mesh.
# Construct the established three-hole steering geometry directly with Gmsh.
# The mesh is generated once and shared by the independent FEM and GNN runs.

```

(continues on next page)

(continued from previous page)

```

import gmsh
HOLE_CENTRES = ((4.0, 4.4), (6.0, 5.8), (8.0, 4.6))
HOLE_RADIUS = 0.55
SLOT_WIDTH = 0.02
config = BaselineConfig(out_dir=OUTPUT, case="three_hole", num_steps=N_STEPS,
                        mesh_size=0.2, length_scale=0.4,
                        final_displacement=FINAL_DISPLACEMENT, history_order="fresh")
torch.manual_seed(config.seed)
np.random.seed(config.seed % (2**32-1))
mesh_path = OUTPUT / "specimen.msh"

def generate_three_hole_mesh(path: Path) -> None:
    """Mesh a square minus the horizontal edge slot and three circular voids."""
    if gmsh.isInitialized():
        gmsh.finalize()
    gmsh.initialize()
    try:
        gmsh.option.setNumber("General.Verbosity", 2)
        gmsh.model.add("three_hole_steering")
        occ = gmsh.model.occ
        plate = occ.addRectangle(0., 0., 0., 10., 10.)
        slot = occ.addRectangle(-SLOT_WIDTH, 5.-SLOT_WIDTH/2, 0.,
                               2.+SLOT_WIDTH, SLOT_WIDTH)
        holes = [(2, occ.addDisk(x, y, 0., HOLE_RADIUS, HOLE_RADIUS))
                  for x, y in HOLE_CENTRES]
        cut, _ = occ.cut([(2, plate)], [(2, slot)] + holes,
                        removeObject=True, removeTool=True)
        occ.synchronize()
        surfaces = [tag for dim, tag in cut if dim == 2]
        if len(surfaces) != 1:
            raise RuntimeError("Expected one connected plate surface")
        gmsh.model.addPhysicalGroup(2, surfaces, name="plate")
        gmsh.model.mesh.setSize(gmsh.model.getEntities(0), config.mesh_size)
        gmsh.option.setNumber("Mesh.MeshSizeFromCurvature", 0)
        gmsh.option.setNumber("Mesh.MeshSizeExtendFromBoundary", 0)
        gmsh.option.setNumber("Mesh.MeshSizeMin", SLOT_WIDTH)
        gmsh.option.setNumber("Mesh.MeshSizeMax", config.mesh_size)
        gmsh.option.setNumber("Mesh.Algorithm", 6)
        gmsh.model.mesh.generate(2)
        gmsh.option.setNumber("Mesh.MshFileVersion", 2.2)
        gmsh.write(str(path))
    finally:
        gmsh.finalize()

generate_three_hole_mesh(mesh_path)
mesh = FEMMesh(str(mesh_path), device="cpu", dtype=torch.float64)
bottom, top = _node_sets(mesh)
nodes = mesh.nodes.numpy()
elements = mesh.elements.numpy()
tri = mtri.Triangulation(nodes[:, 0], nodes[:, 1], elements)

```

```
[FEMMesh] Loading mesh: /content/phast_demo_work/results_three_hole_classroom/specimen.msh
```

```
[FEMMesh] Mesh file read OK
```

```
[FEMMesh] 3009 nodes loaded
```

```
[FEMMesh] 5747 T3 elements loaded
```

```
[FEMMesh] Node sets: []
```

```
[FEMMesh] Precomputing areas, grad_phi, incircle, lumped mass...
```

```
[FEMMesh] Precompute done: h_min=1.782349e-02, total_area=97.166550
```

4. Specify the material

We use millimetres, newtons and MPa. The elastic modulus is 210,000 MPa, Poisson's ratio is 0.3, fracture energy is 2.7 N/mm and the phase-field length is 0.4 mm. The retained implementation uses plane-stress spectral elasticity and the checkpoint's rational degradation law:

$$g(d) = \frac{(1-d)^2}{d^2-d+1}, \quad 0 \leq d \leq 1.$$

Here $d = 0$ denotes intact material and $d = 1$ denotes fully damaged material. The same material formulation is used in both routes. Changing the law would change the problem seen by the frozen model.

Code cell 9

```
# 4. Apply material properties.
# The original rational AT2 model is retained, including its reduced-2D
# plane-stress spectral approximation. Units are mm, N and MPa.
material = Material(
    E=210000., nu=0.3, Gc=2.7, l0=0.4, rho=7.8e-9,
    eta_residual=1e-7, energy_split="spectral", stress_degradation="full",
    pf_model="AT2", degradation_type="moon_rational", plane_stress=True,
)
```

5. Apply boundary conditions and loading

Fix both displacement components on the bottom boundary. On the top boundary, fix horizontal displacement and increase vertical displacement to 0.04 mm over N_STEPS increments. The slot and hole boundaries are traction-free. No damage Dirichlet value is prescribed on their surfaces.

Code cell 11

```
# 5. Apply boundary conditions and the prescribed load.
# The geometric slot is traction-free. No damage value is imposed on it.
# The bottom is fixed; the top cannot move horizontally and is displaced
# vertically to FINAL_DISPLACEMENT through N_STEPS equal load increments.
def apply_boundary_conditions():
    bcs = BoundaryConditions(mesh.n_nodes, mesh.device, mesh.dtype)
    bcs.fix(bottom, 0).fix(bottom, 1)
    bcs.fix(top, 0).add(top, 1, FINAL_DISPLACEMENT)
    return bcs

boundary_conditions = apply_boundary_conditions()

fig, ax = plt.subplots(figsize=(6, 6))
ax.triplot(tri, lw=0.25, color="0.7")
ax.scatter(nodes[bottom, 0], nodes[bottom, 1], s=7, color="#2878A8", label="Fixed bottom")
ax.scatter(nodes[top, 0], nodes[top, 1], s=7, color="#29936B", label="Prescribed top")
ax.set(xlabel="x [mm]", ylabel="y [mm]", title="Finite-element mesh and boundary conditions",
       aspect="equal")
ax.legend(loc="lower right")
fig.savefig(OUTPUT / "mesh_and_boundary_conditions.png", dpi=180)
plt.show()
```

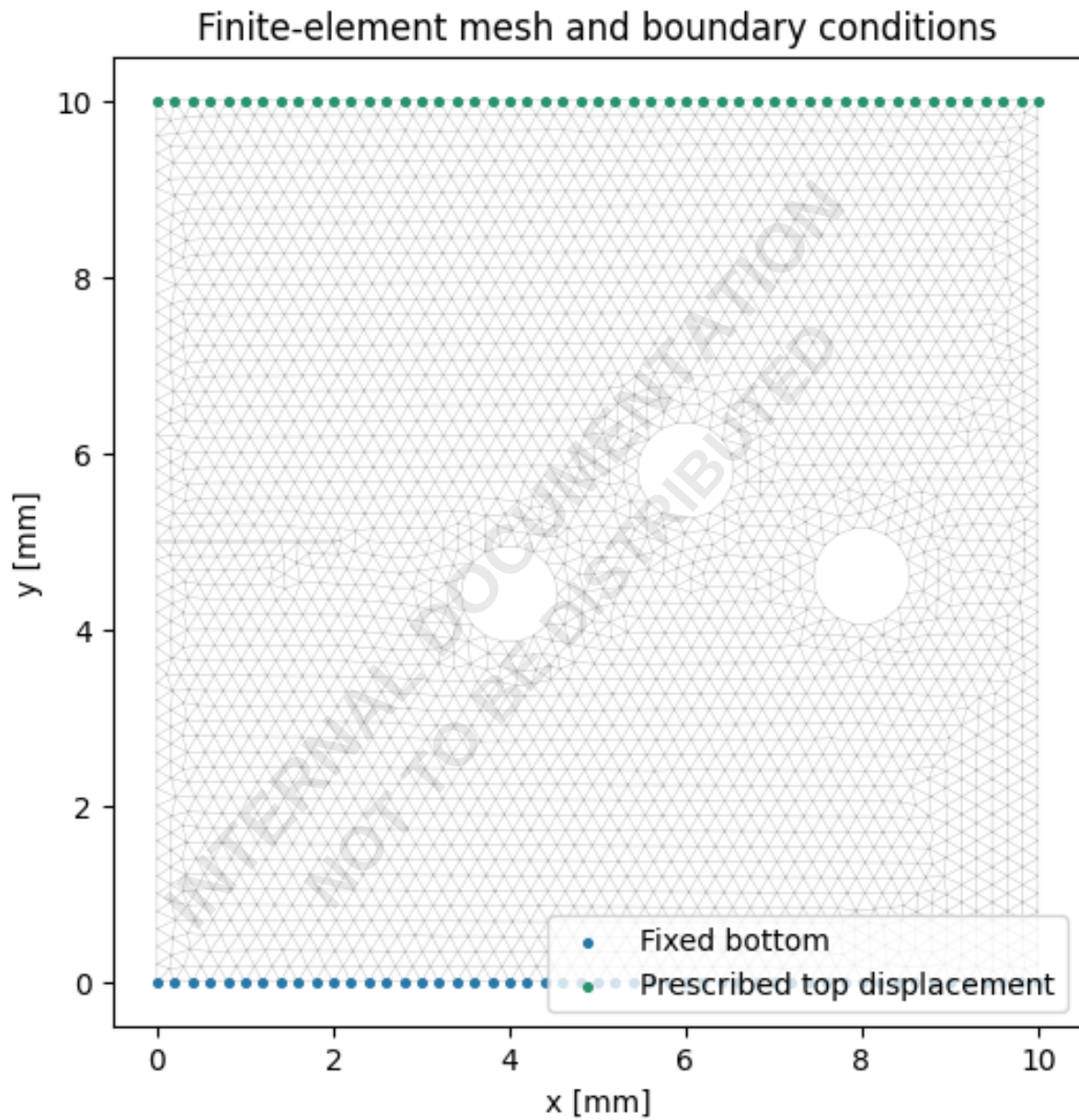


Fig. 25: Saved output from code cell 11.

Exercise 1 — Inspect the model

Locate the horizontal slot and all three holes. Which boundaries carry prescribed displacement, and which are traction-free? Explain why prescribing damage on a hole is a different model.

Exercise for independent work.

Hint

The hole is absent material; its boundary is not a set of nodes that must have damage equal to one.

6. Choose the solution procedure

This is an implicit, quasi-static calculation. Inertia is neglected. The load increments describe the loading schedule; they are not dynamic time steps. The supplied code selects 60 increments with `QUICK_TEST=True` and 500 with `QUICK_TEST=False`, unless `PHAST_DEMO_STEPS` overrides the count. Use the same count for both routes. A coarser schedule is not evidence of load-increment convergence. Each increment contains one mechanics solve followed by one damage update. Each conventional subsolver has its own internal iterations.

We retain this one-pass staggered procedure to match the tested teaching example. It is not an outer staggered iteration continued until coupled mechanics and damage convergence at every increment.

Code cell 14

```
# 6. Select the analysis and solver settings.
# This example is quasi-static: inertia is neglected and mechanical equilibrium
# is solved implicitly at each load increment. N_STEPS counts load increments,
# not explicit dynamic time steps.
# Dynamic/explicit analyses need mass, physical time-step and stability choices;
# the frozen model's present demonstration uses the quasi-static setting.
ANALYSIS = "quasi_static"
MECHANICS_UPDATE = "implicit"
COUPLING = "one_pass_staggered"
MECHANICS_BACKEND = "scipy"

if (ANALYSIS, MECHANICS_UPDATE, COUPLING) != (
    "quasi_static", "implicit", "one_pass_staggered"):
    raise ValueError("This frozen-model example is validated for implicit quasi-static, one-
pass staggered coupling.")

settings = SolverConfig(
    solver_type=ANALYSIS, num_steps=N_STEPS,
    static_tol=1e-8, static_max_iter=60,
    damage_tol=1e-8, damage_max_iter=1000,
    bounds_method="projected_cg", use_multigrid=False,
    preconditioner="jacobi", backend=MECHANICS_BACKEND,
    newton_line_search=True, fail_on_mechanics_nonconvergence=True,
)

def new_solver():
    # Each simulation receives fresh displacement, history, damage and BC state.
    return StaggeredSolver(mesh, material, apply_boundary_conditions(), settings)
```

7. Read the solver loop

At increment n , both routes follow the same sequence:

$$d_{n-1} \longrightarrow u_n \longrightarrow H_n \longrightarrow d_n.$$

1. Solve mechanical equilibrium using the previous damage field.
2. Update the maximum-history driving quantity.
3. Calculate damage with FEM, or predict it with the frozen GNN.
4. Enforce damage bounds, evaluate the assembled residual and advance.

Irreversibility requires $d_n \geq d_{n-1}$. We impose this with projection onto $[d_{n-1}, 1]$. The GNN residual is measured, not followed by an FEM damage correction. The two routes begin from independent initial states; the learned route never receives the reference FEM damage trajectory.

Where the damage subproblem is replaced

The whole exercise turns on a single branch in the function below, so it is worth reading before the code. `damage_update()` performs step (c) of the loop. Its `adapter` argument selects who answers:

adapter	What computes the damage field
None	<code>solver.step_solve_damage(...)</code> — finite elements assemble and solve the damage equation
the predictor	<code>adapter.propose(history, previous)</code> — the trained network returns the field directly

On the learned route the network's output is the damage field for that increment, and it feeds directly into the next mechanics solve.

Everything else in the function is shared by both routes: the projection onto $[d_{n-1}, 1]$ that enforces irreversibility, the phase-field Dirichlet conditions, the finiteness check, and the residual measurement. Mechanics, geometry, mesh, material and loading are untouched.

And the choice is made by one argument at the call site:

```
fem_result = simulate()           # Section 8 - conventional
gnn_result = simulate(adapter)    # Section 11 - learned damage
```

The assembled residual is measured on both routes and reported alongside the fields, so you can read how well each damage field satisfies the damage equation. The field you see is the one that route produced.

What `adapter.propose()` receives and returns

```
damage, diagnostics = adapter.propose(history_elem, damage_previous)
```

It is handed exactly the two quantities the finite-element damage solve uses: the element history field and the previously accepted nodal damage. It returns one damage value per node, on the same node indices, plus a diagnostics dictionary carrying the inference time.

Code cell 17

```
# 7. Define the staggered operations.
# For each load increment:
#   (a) solve solid-mechanics equilibrium using the previous damage;
#   (b) update the maximum-history driving quantity;
#   (c) solve damage, or predict damage with the frozen network;
#   (d) apply bounds, evaluate the residual and advance the load state.
# One mechanics update followed by one damage update is the retained coupling.
```

(continues on next page)

(continued from previous page)

```

# The mechanics and damage solvers have their own internal iterations.

def synchronize():
    if DEVICE == "cuda":
        torch.cuda.synchronize()

def damage_update(solver, history, previous, adapter=None):
    """One update including projection and residual evaluation; no mechanics."""
    solver.H_elem = history
    solver.d = previous.clone()

    # =====
    # THE DAMAGE SUBPROBLEM.
    # This branch is the only place where the two routes differ at all.
    # =====
    if adapter is None:
        # ROUTE A, conventional: finite elements solve the damage equation.
        solver.step_solve_damage(d_prev_step=previous)
        if not solver.damage_solver.last_converged:
            raise RuntimeError("FEM damage solver did not converge")
    else:
        # ROUTE B, learned: the trained network returns the damage field, and
        # its output becomes the damage for this increment.
        # Inference reads the current hybrid history of this run.
        candidate, _ = adapter.propose(history, previous)
        solver.d = candidate.to(device=mesh.device, dtype=mesh.dtype)
    # ===== everything below here is shared by both =====

    solver.d = torch.maximum(solver.d, previous).clamp_max(1.)
    solver._apply_pf_dirichlet()
    if not bool(torch.isfinite(solver.d).all()):
        raise RuntimeError("Non-finite damage")
    residual = solver.damage_solver.compute_residual(history, solver.d)
    measures = damage_kkt_metrics(residual, solver.d, previous)
    # We measure residuals but do not replace a rejected GNN result with FEM.
    # Bounds alone do not establish satisfaction of the damage equation.
    return solver.d, measures

def simulate(adapter=None):
    solver = new_solver() # independent state for each route
    history_rows, fields, replay_states = [], [], []
    selected = set(np.linspace(1, N_STEPS, min(4, N_STEPS), dtype=int))
    start_total = time.perf_counter()
    progress = tqdm(range(1, N_STEPS + 1),
                    desc="FEM solve" if adapter is None else "FEM + GNN solve",
                    unit="increment")
    for step in progress:
        solver.bcs.load_factor = step / N_STEPS
        previous = solver.d.clone()
        old_history = solver.H_elem.clone()
        synchronize(); start = time.perf_counter()
        solver.step_mechanics()
        synchronize(); mechanics_time = time.perf_counter() - start
        strain = solver.fem.compute_strain(solver.u)
        drive = solver.fem.compute_driving_force(solver.u, strain=strain, d=previous)
        history = torch.maximum(old_history, drive)
        if step in selected and adapter is None:
            replay_states.append((step, history.clone(), previous.clone()))
        synchronize(); start = time.perf_counter()
        damage, measures = damage_update(solver, history, previous, adapter)
        synchronize(); damage_time = time.perf_counter() - start

```

(continues on next page)

(continued from previous page)

```

solver.H_elem = history
solver.H_nodal = mesh.elem_to_node(history)
solver.advance_step()
reaction = solver.fem.compute_reaction_force(solver.u, solver.d, top, component=1)
history_rows.append(dict(step=step, displacement=FINAL_DISPLACEMENT*step/N_STEPS,
                        reaction=float(reaction), mechanics_s=mechanics_time,
                        damage_s=damage_time, residual_l2=float(measures["kkt_
->projected_l2"])))
fields.append(damage.detach().cpu().numpy().copy())
if step == 1 or step % max(1, N_STEPS//10) == 0:
    progress.set_postfix(d_max=f"{float(damage.max()):.3f}",
                        residual=f"{float(measures['kkt_projected_l2']):.2e}")
synchronize()
return dict(rows=history_rows, fields=np.array(fields), replay=replay_states,
            total_s=time.perf_counter()-start_total)

```

Exercise 2 — Follow the state

In the code, identify the previous damage, current history and candidate damage. Which value ensures irreversibility? Where would an outer staggered convergence loop have to be added?

Exercise for independent work.

Hint

Trace previous, history, damage_update and torch.maximum. The present increment loop advances immediately after one pair of subsolves.

8. Run conventional FEM

Execute the reference calculation first. The progress bar shows the current increment, maximum damage and projected residual. A conventional damage solve that fails to converge stops the calculation rather than producing a successful benchmark label. Record N_STEPS with the results so comparisons use the same loading schedule.

Code cell 20

```

# 8. Solve the specimen using conventional FEM
# This is the numerical reference. No learned model is involved.
# No adapter is passed, so damage_update() takes ROUTE A and the damage
# equation is solved by finite elements at every increment.

fem_result = simulate()

```

```

[StaggeredSolver] Assembling solver (quasi_static)...
[FEM0operators] Initializing (spectral split, AT2)...
[FEM0operators] dt_CFL = 3.276809e-09 (c_p=5439282.93)
[PhaseFieldDamageSolver] Initializing CG solver (model=AT2, tol=1.0e-08, max_iter=1000,
bounds=projected_cg)...
[PhaseFieldDamageSolver] CG device: cpu, dtype: float64
[PhaseFieldDamageSolver] Ready (preconditioner=jacobi).
[StaggeredSolver] QuasiStaticSolver ready
[StaggeredSolver] State allocated: 3009 nodes, 5747 elements, device=cpu, dtype=torch.float64

```

```

[QuasiStaticSolver] mechanics backend: scipy (SciPy SuperLU sparse-direct LU); tangent=frozen-
state secant sparse tangent; free_dofs=5814

```

```

/content/phast_demo_work/assets/source/src/phast/core/fem_operators.py:974: RuntimeWarning:
FEM0operators._psi_plus_spectral: spectral energy split is approximate under plane stress: this

```

(continues on next page)

(continued from previous page)

path is a reduced 2D in-plane strain-spectral projection, not a fully condensed 3D plane-stress spectral decomposition with damage-dependent out-of-plane strain. For validated production paths prefer plane-strain spectral or plane-stress Amor. Audit T1.4 (W4).

```
return self._psi_plus_spectral(strain)
```

9. Examine the FEM evolution

The plot and GIF are generated from your calculation. Observe crack initiation, interaction with the first hole and propagation through the remaining ligament. The damage colour scale remains fixed from zero to one.

Code cell 22

```
# 9. Visualise the FEM solution and save its evolution as a GIF.
# Damage is interpolated within each triangle from the FEM nodal values.
# Identical frame indices and damage colour limits are used for both routes.
GIF_FRAMES = 60
GIF_FPS = 8

def display_gif(path):
    if IN_COLAB:
        from IPython.display import display, Image
        display(Image(filename=str(path)))
    else:
        print("Animation:", path)

def visualise_and_animate(result, label, filename):
    fields = result["fields"]
    fig, ax = plt.subplots(figsize=(5.8, 5))
    artist = ax.tripcolor(tri, fields[-1], shading="gouraud",
                        cmap="viridis", vmin=0, vmax=1)
    fig.colorbar(artist, ax=ax, label="Damage d")
    ax.set(xlabel="x [mm]", ylabel="y [mm]", aspect="equal", title=label)
    fig.tight_layout()
    fig.savefig(OUTPUT / (filename + "_final.png"), dpi=180)
    plt.show()
    plt.close(fig)

    indices = np.unique(np.linspace(0, len(fields)-1, min(GIF_FRAMES, len(fields)),
dtype=int))
    fig, ax = plt.subplots(figsize=(5.8, 5))
    artist = ax.tripcolor(tri, fields[0], shading="gouraud",
                        cmap="viridis", vmin=0, vmax=1)
    fig.colorbar(artist, ax=ax, label="Damage d")
    ax.set(xlabel="x [mm]", ylabel="y [mm]", aspect="equal")
    title = ax.set_title(label)
    fig.tight_layout()
    def update(index):
        artist.set_array(fields[index])
        displacement = result["rows"][index]["displacement"]
        title.set_text(f"{label} | increment {index+1} | u = {displacement:.4f} mm")
        return artist, title
    animation = FuncAnimation(fig, update, frames=indices, blit=False)
    path = OUTPUT / (filename + ".gif")
    animation.save(path, writer=PillowWriter(fps=GIF_FPS), dpi=100)
    plt.close(fig)
    display_gif(path)

visualise_and_animate(fem_result, "FEM damage", "fem_damage")
```

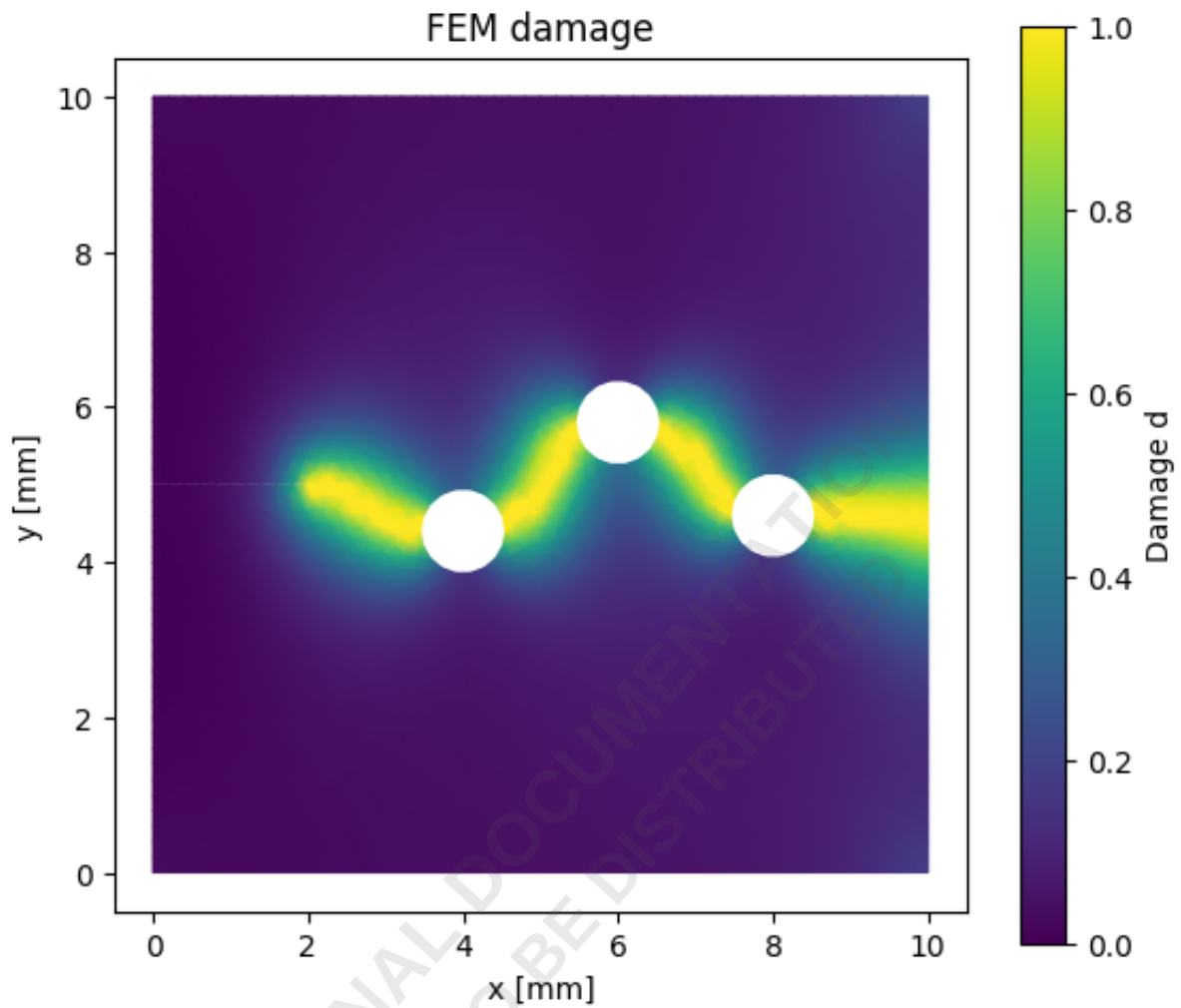
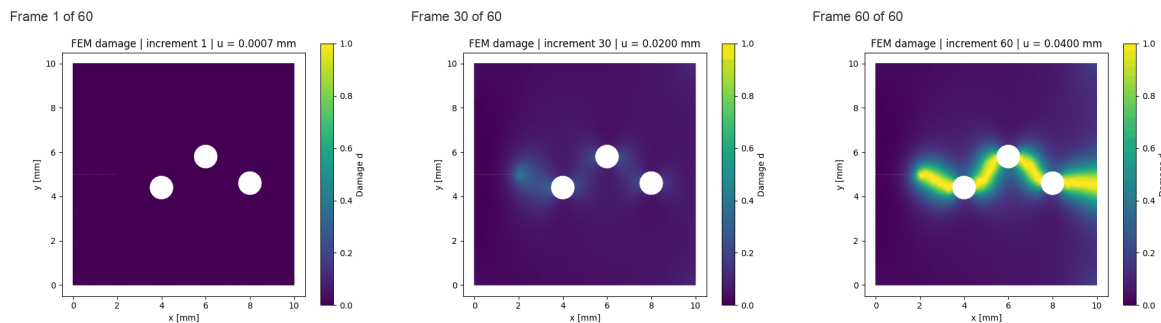


Fig. 26: Saved output from code cell 22.

Fig. 27: Three stages from the saved animation (60 frames). [Play the animation online.](#)

Exercise 3 — Read the reference solution

At what approximate prescribed displacement does a connected damaged region first reach the first hole? Record your observation and the frame used.

Exercise for independent work.

Hint

Use the displacement shown above the animation. State the damage threshold you use to identify a damaged region.

10. Load the frozen graph neural network

Each finite-element vertex becomes one graph node. Triangle edges define graph connections in both directions; no image rasterisation is used.

Model component	Information
Node inputs	Assembled history quantity and topological boundary indicator
History transform	$\log_{10}(1 + q_i)$, applied inside the network
Edge input	Edge length divided by phase-field length
Encoder	Maps inputs to 32-component latent vectors
Processor	Ten neighbour message-passing blocks
Decoder	One predicted damage value per node

For homogeneous linear triangles, the history feature is assembled as $q_i = \sum_{e \ni i} A_e H_e / (3G_c \ell)$. The boundary indicator includes the outer boundary, slot and hole boundaries. Predictions map back to the same FEM node indices, with $d_h(x) = \sum_i N_i(x) d_i$ inside elements.

These weights were trained on an inclined-slot specimen. Applying them here is a frozen-weight geometry-transfer experiment. No optimiser is constructed.

Code cell 25

```
# 10. Load the trained graph model
# Each FEM vertex becomes a graph node with the same node index. Triangle
# connectivity defines graph edges, represented in both directions.
#
# Node inputs: log10(1 + assembled history/(Gc ell)) and a topology-boundary
# indicator. The assembly includes element area and shape-function weights;
# it is not simply the history value at a point. The edge input is length/ell.
# The encoders produce 32-component latent vectors. Ten message-passing blocks
# update the graph, using degree-normalised mean aggregation. The decoder
# returns one damage scalar per node. There is no rasterisation or remeshing.
#
# No optimiser or training dataset is used here. Weights remain fixed.

adapter = build_live_adapter(
    "literal_moon", checkpoint_path=CHECKPOINT,
    checkpoint_sha256=CHECKPOINT_SHA, device=DEVICE,
    nodes=nodes, elements=elements,
    boundary_context=moon_boundary_context(nodes, elements),
    Gc_elem=2.7, length_scale=0.4,
)
adapter.model.eval()
for parameter in adapter.model.parameters():
    parameter.requires_grad_(False)
print(adapter.model)
print("Trainable parameters:", sum(p.numel() for p in adapter.model.parameters() if p
    .requires_grad))
print("Graph nodes:", len(nodes), "| directed edges:", adapter.edges.shape[1])
```

```
[Long console log abbreviated; see the online notebook for the full output.]
(4): Linear(in_features=32, out_features=32, bias=True)
(5): ReLU()
(6): Linear(in_features=32, out_features=32, bias=True)
)
)
)
(decoder): Sequential(
  (0): Linear(in_features=32, out_features=32, bias=True)
  (1): ReLU()
  (2): Linear(in_features=32, out_features=32, bias=True)
  (3): ReLU()
  (4): Linear(in_features=32, out_features=32, bias=True)
  (5): ReLU()
  (6): Linear(in_features=32, out_features=1, bias=True)
)
)
Trainable parameters: 0
Graph nodes: 3009 | directed edges: 17516
```

The two routes on identical inputs

The comparison below is the most direct one available: `damage_update()` is called twice with the same history and the same previous damage, taken from a stored increment of the finite-element run. The only difference between the two calls is the `adapter` argument.

Each call gets its own fresh solver, so each result stands on its own. Any difference that appears is produced by the damage subproblem alone.

Code cell 27

```
# Both routes, identical inputs, one increment.
# The only difference between these two calls is the `adapter` argument.
step, history, previous = fem_result["replay"][-1]

classical_damage, classical_measures = damage_update(
    new_solver(), history, previous, adapter=None)          # ROUTE A: finite elements
classical_damage = classical_damage.clone()

learned_damage, learned_measures = damage_update(
    new_solver(), history, previous, adapter=adapter)       # ROUTE B: the network
learned_damage = learned_damage.clone()

print(f"increment {step}: the same history and previous damage given to both routes\n")
print(f"{'route':<20}{ 'max d':>11}{ 'mean d':>11}{ 'projected KKT L2':>20}")
for name, field, measures in (("A  finite elements", classical_damage, classical_measures),
                              ("B  the network", learned_damage, learned_measures)):
    print(f"{name:<20}{float(field.max()):>11.6f}{float(field.mean()):>11.6f}"
          f"{float(measures['kkt_projected_l2']):>20.3e}")

relative = 100.0 * float((learned_damage - classical_damage).norm() /
                          classical_damage.norm())
print(f"\nrelative L2 difference between the two damage fields: {relative:.2f} %")
print("The projected KKT norm reports how well each field satisfies the")
print("damage equation, measured the same way for both routes.")
```

```
[StaggeredSolver] Assembling solver (quasi_static)...
[FEMOperators] Initializing (spectral split, AT2)...
[FEMOperators] dt_CFL = 3.276809e-09 (c_p=5439282.93)
[PhaseFieldDamageSolver] Initializing CG solver (model=AT2, tol=1.0e-08, max_iter=1000,
bounds=projected_cg)...
```

(continues on next page)

(continued from previous page)

```
[PhaseFieldDamageSolver] CG device: cpu, dtype: float64
[PhaseFieldDamageSolver] Ready (preconditioner=jacobi).
[StaggeredSolver] QuasiStaticSolver ready
[StaggeredSolver] State allocated: 3009 nodes, 5747 elements, device=cpu, dtype=torch.float64
[StaggeredSolver] Assembling solver (quasi_static)...
[FEM0perators] Initializing (spectral split, AT2)...
[FEM0perators] dt_CFL = 3.276809e-09 (c_p=5439282.93)
[PhaseFieldDamageSolver] Initializing CG solver (model=AT2, tol=1.0e-08, max_iter=1000,
bounds=projected_cg)...
[PhaseFieldDamageSolver] CG device: cpu, dtype: float64
[PhaseFieldDamageSolver] Ready (preconditioner=jacobi).
[StaggeredSolver] QuasiStaticSolver ready
[StaggeredSolver] State allocated: 3009 nodes, 5747 elements, device=cpu, dtype=torch.float64
increment 60: the same history and previous damage given to both routes
```

route	max d	mean d	projected KKT L2
A finite elements	1.000000	0.150415	1.695e-07
B the network	1.000000	0.154698	7.768e-01

relative L2 difference between the two damage fields: 3.68 %
The projected KKT norm reports how well each field satisfies the damage equation, measured the same way for both routes.

Exercise 4 — Mesh to graph

Why does the model produce exactly one output per FEM node? What changes when the mesh is refined, and what remains frozen?

Exercise for independent work.

Hint

The graph connectivity and node/edge feature arrays change. The learned matrices and ten processing blocks remain the same.

11. Run FEM mechanics with GNN damage prediction

Create a fresh solver state and repeat the loading schedule. The GNN now receives the history produced by its own evolving simulation. Its damage prediction affects the following mechanics solve: this is a recursive coupled simulation, not a comparison on reference inputs alone.

Code cell 30

```
# 11. Solve again with the trained damage predictor inside the solver loop
# Begin again from the same initial state. Mechanics is still calculated by
# FEM. The trained DL model replaces every damage solve and its prediction affects
# the next mechanics calculation. No reference damage enters the GNN inputs.
# Predictions map directly to the same FEM nodes. Within a triangle,
#  $d_h(x) = \sum_a N_a(x) d_a$  represents the field through FEM shape functions.

# Passing `adapter` is the entire change. damage_update() now takes ROUTE B,
# and the damage equation is not solved at any increment of this run.
gnn_result = simulate(adapter)
```

```
[StaggeredSolver] Assembling solver (quasi_static)...
[FEM0perators] Initializing (spectral split, AT2)...
[FEM0perators] dt_CFL = 3.276809e-09 (c_p=5439282.93)
[PhaseFieldDamageSolver] Initializing CG solver (model=AT2, tol=1.0e-08, max_iter=1000,
bounds=projected_cg)...
[PhaseFieldDamageSolver] CG device: cpu, dtype: float64
```

(continues on next page)

(continued from previous page)

```
[PhaseFieldDamageSolver] Ready (preconditioner=jacobi).
[StaggeredSolver] QuasiStaticSolver ready
[StaggeredSolver] State allocated: 3009 nodes, 5747 elements, device=cpu, dtype=torch.float64
```

```
[QuasiStaticSolver] mechanics backend: scipy (SciPy SuperLU sparse-direct LU); tangent=frozen-
state secant sparse tangent; free_dofs=5814
```

12. Examine the learned simulation

Compare this evolution with the FEM GIF before studying the numerical error. Use the same prescribed displacement and damage colour scale when comparing.

Code cell 32

```
# 12. Visualise the live GNN-coupled simulation and save its GIF.
visualise_and_animate(gnn_result, "Live FEM + GNN damage", "gnn_damage")
```

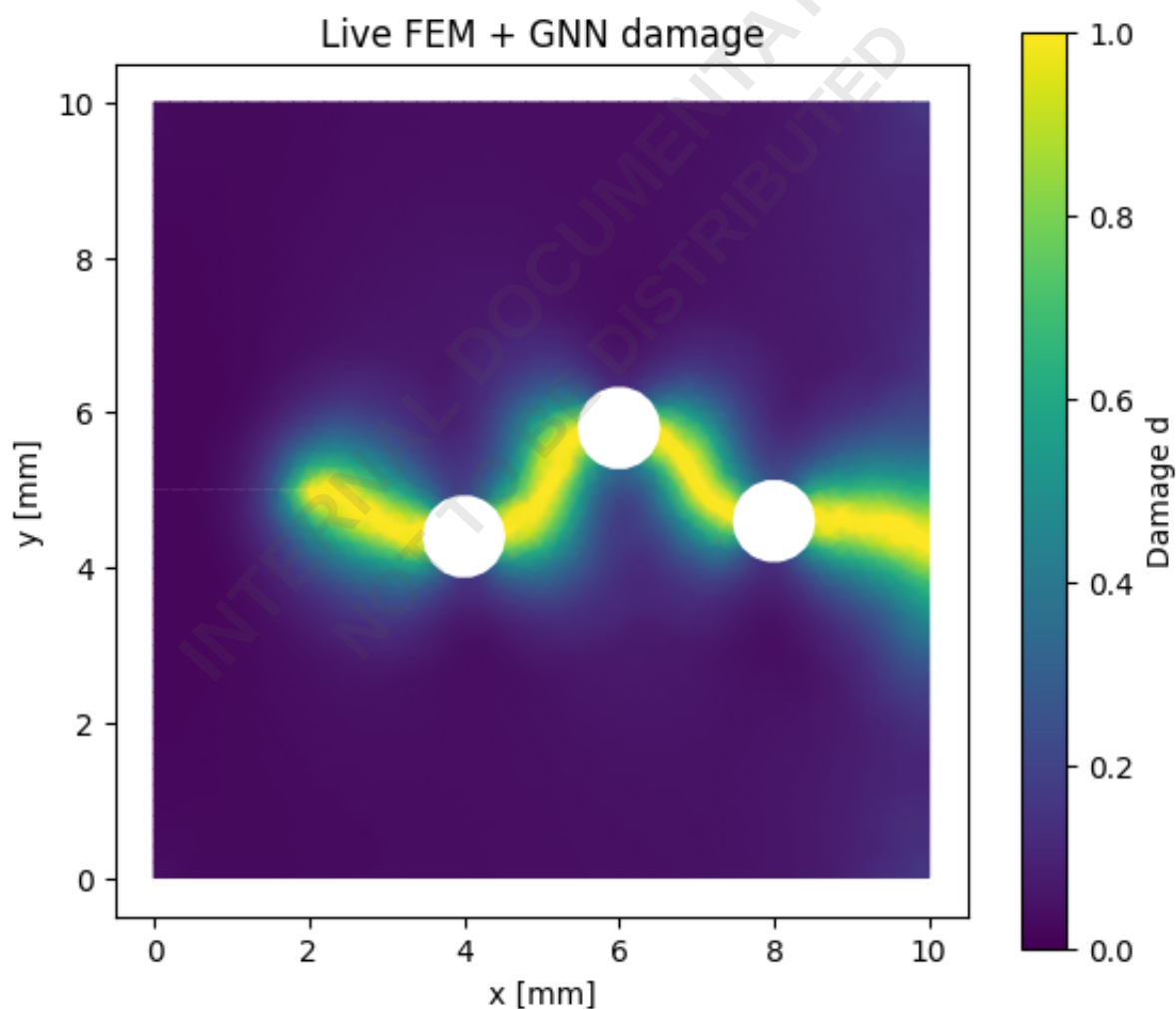


Fig. 28: Saved output from code cell 32.

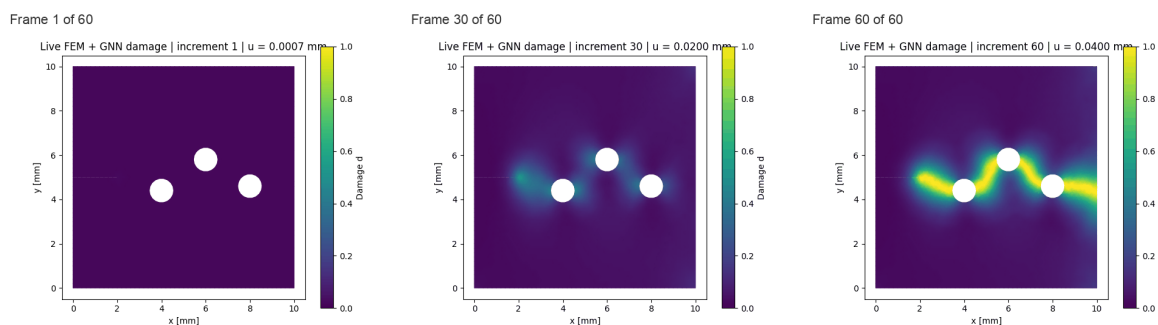


Fig. 29: Three stages from the saved animation (60 frames). [Play the animation online.](#)

13. Compare fields at matched loads

The third panel displays $|d_{\text{GNN}} - d_{\text{FEM}}|$. Its colour scale is fixed over the complete animation. This reveals when and where the two independent loading paths diverge, rather than just their final appearance.

Code cell 34

```
# 13. Compare FEM damage, live predicted damage and error
# Use the same colour limits for both damage plots. The third panel is the
# absolute nodal error. Similar-looking plots are not an accuracy certificate:
# inspect the reaction curve, field error and damage residual as well.

reference = fem_result["fields"][-1]
prediction = gnn_result["fields"][-1]
fig, axes = plt.subplots(1, 3, figsize=(13, 4), constrained_layout=True)
for ax, values, title, vmax in zip(axes, [reference, prediction, abs(prediction-reference)],
    ["FEM damage", "Frozen-GNN damage", "Absolute damage error"],
    [1., 1., max(1e-8, float(abs(prediction-reference).max()))]):
    im = ax.tripcolor(tri, values, shading="gouraud", cmap="viridis", vmin=0, vmax=vmax)
    ax.set(title=title, xlabel="x [mm]", ylabel="y [mm]", aspect="equal")
    fig.colorbar(im, ax=ax)
fig.savefig(OUTPUT / "damage_comparison.png", dpi=180)
plt.show()

# Animate both full recursive trajectories at matched load increments.
# The error scale is fixed over the entire animation so colour changes
# represent changing error, not a changing colour-bar range.
error_fields = np.abs(gnn_result["fields"] - fem_result["fields"])
error_limit = max(float(error_fields.max()), 1e-8)
indices = np.unique(np.linspace(0, N_STEPS-1, min(GIF_FRAMES, N_STEPS), dtype=int))
fig, axes = plt.subplots(1, 3, figsize=(12, 4), constrained_layout=True)
artists = []
for ax, field, label, limit in zip(
    axes, [fem_result["fields"][0], gnn_result["fields"][0], error_fields[0]],
    ["FEM damage", "Live FEM + GNN damage", "Absolute damage error"],
    [1., 1., error_limit]):
    artist = ax.tripcolor(tri, field, shading="gouraud",
        cmap="viridis", vmin=0, vmax=limit)
    artists.append(artist)
    ax.set(title=label, xlabel="x [mm]", ylabel="y [mm]", aspect="equal")
    fig.colorbar(artist, ax=ax)
title = fig.suptitle("")
def update_comparison(index):
    for artist, field in zip(artists, [fem_result["fields"][index],
        gnn_result["fields"][index], error_fields[index]]):
        artist.set_array(field)
```

(continues on next page)

(continued from previous page)

```

displacement = fem_result["rows"][index]["displacement"]
title.set_text(f"Increment {index+1}/{N_STEPS} | prescribed displacement {displacement:.4f} mm")
return (*artists, title)
animation = FuncAnimation(fig, update_comparison, frames=indices, blit=False)
comparison_gif = OUTPUT / "fem_gnn_error.gif"
animation.save(comparison_gif, writer=PillowWriter(fps=GIF_FPS), dpi=100)
plt.close(fig)
display_gif(comparison_gif)

```

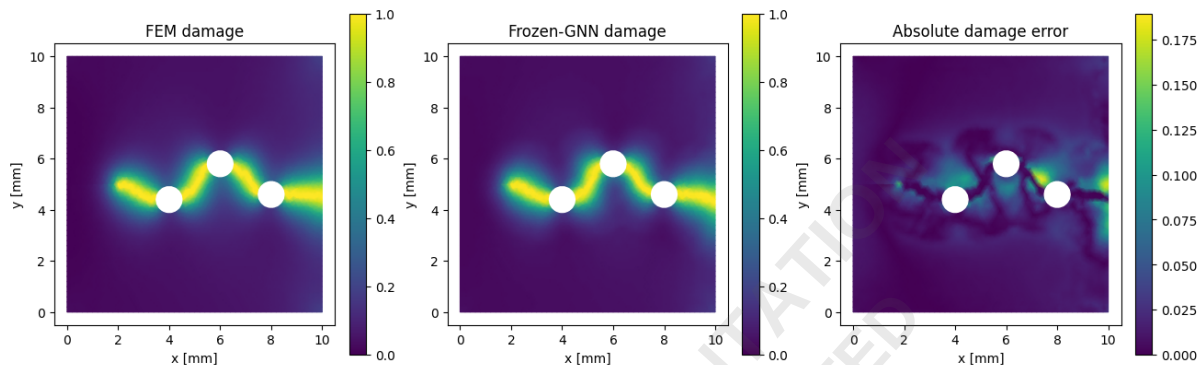


Fig. 30: Saved output from code cell 34.

14. Compare reactions and equation residuals

The reported final relative nodal error is

$$e_d = \frac{\|d_{\text{GNN}} - d_{\text{FEM}}\|_2}{\|d_{\text{FEM}}\|_2}.$$

This is an unweighted nodal norm, not a mesh-independent integral norm. The projected damage residual measures consistency with the constrained FEM damage equation. A small field error and a small residual are different tests. Reaction forces here are evaluated after the damage update in the one-pass scheme; the displacement is not re-equilibrated at that same increment.

Code cell 36

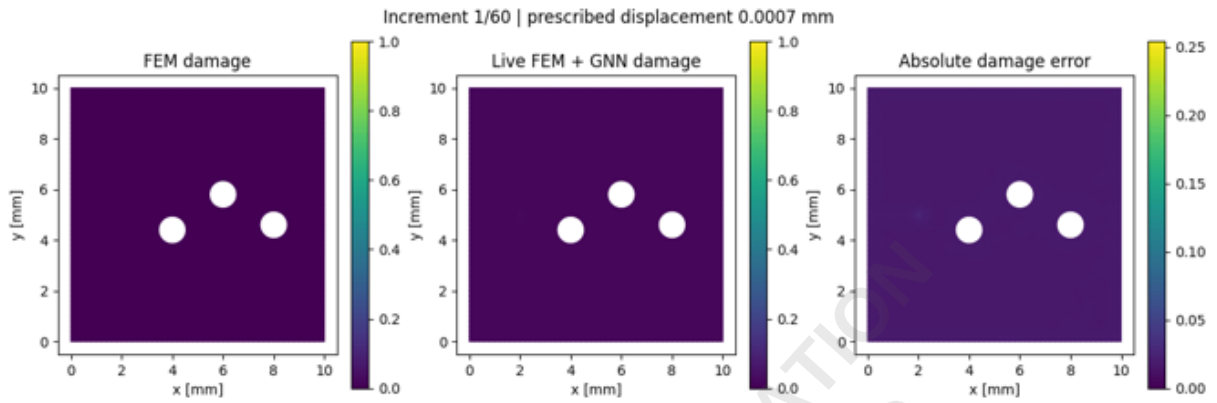
```

# 14. Compare reactions and residuals.
fig, axes = plt.subplots(1, 2, figsize=(10, 4), constrained_layout=True)
for result, name in [(fem_result, "FEM"), (gnn_result, "FEM + frozen GNN")]:
    x = [r["displacement"] for r in result["rows"]]
    axes[0].plot(x, [r["reaction"] for r in result["rows"]], label=name)
    axes[1].semilogy(x, [max(r["residual_L2"], 1e-18) for r in result["rows"]], label=name)
axes[0].set(xlabel="Prescribed displacement [mm]", ylabel="Post-update reaction [N]")
axes[1].set(xlabel="Prescribed displacement [mm]", ylabel="Projected damage residual L2")
for ax in axes: ax.legend(); ax.spines[["top", "right"]].set_visible(False)
fig.savefig(OUTPUT / "reaction_and_residual.png", dpi=180)
plt.show()
relative_error = np.linalg.norm(prediction-reference)/max(np.linalg.norm(reference), 1e-12)
print(f"Final relative nodal L2 error: {relative_error:.3%}")

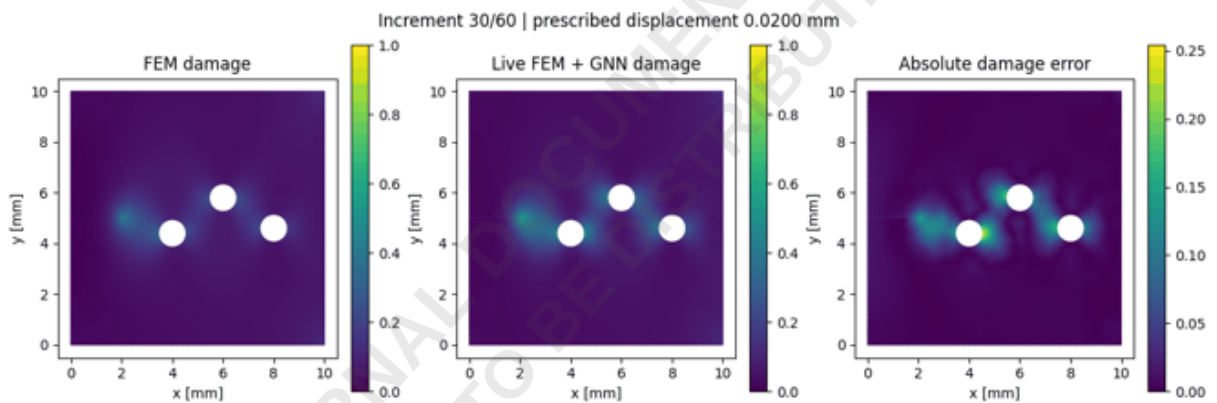
```

Final relative nodal L2 error: 10.257%

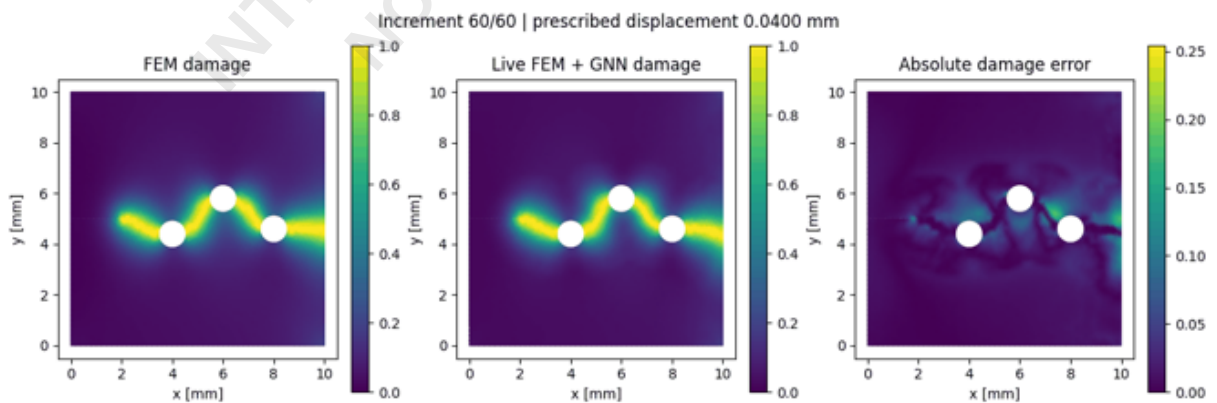
Frame 1 of 60



Frame 30 of 60



Frame 60 of 60

Fig. 31: Three stages from the saved animation (60 frames). [Play the animation online.](#)

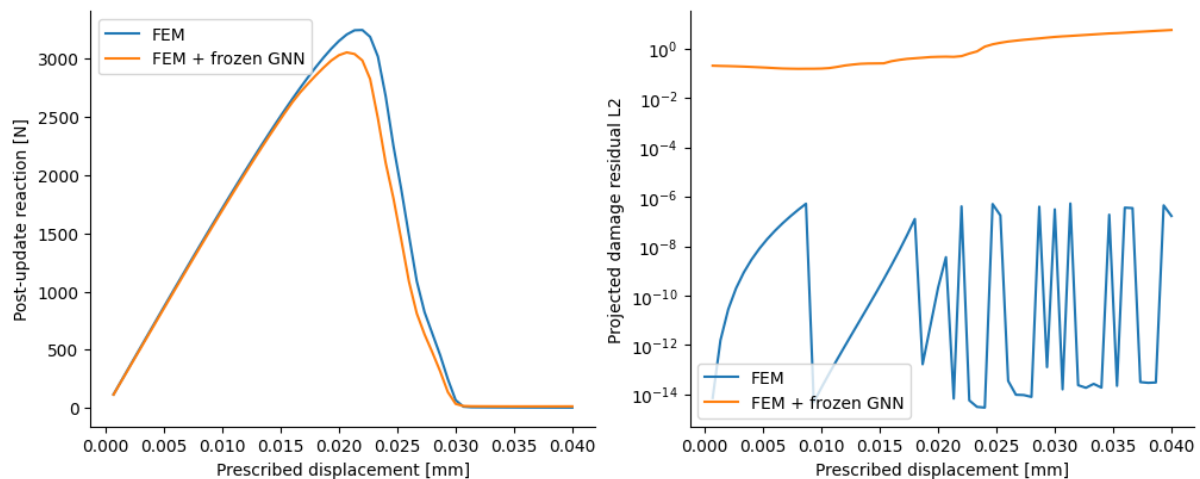


Fig. 32: Saved output from code cell 36.

Exercise 5 — Compare accuracy measures

Find a load range where the field plots look similar but the residuals differ. Does projection onto the damage bounds solve the FEM damage equation?

Exercise for independent work.

Hint

Projection enforces admissibility; it does not in general minimise the damage energy or eliminate its constrained residual.

15. Measure cost over the two loading paths

Separate mechanics time, damage-update time and complete-loop time. Damage timing includes feature construction, inference or FEM damage calculation, host/device transfers, projection and residual evaluation. Complete-loop time also includes history, reactions and array storage, but excludes initial setup, model loading and figures. It is therefore not total notebook execution time.

The recursive paths can develop different states and costs. The next section also compares both damage methods on identical input states.

Code cell 39

```
# 15. Measure time savings in the damage subproblem
# Damage time includes features/prediction or the FEM damage solve, transfers,
# bounds, finite checks, residual and KKT evaluation.
# Mechanics is untouched by the swap and is reported separately below.
# The two recursive paths develop different states, so the separate benchmark
# in Section 16 additionally measures the two updates on identical input states.

plt.rcParams.update({
    "font.family": "STIXGeneral", "mathtext.fontset": "stix",
    "font.size": 12, "axes.titlesize": 14, "axes.labelsize": 12,
})

for result in (fem_result, gnn_result):
    result["damage_total_s"] = sum(r["damage_s"] for r in result["rows"])
    result["mechanics_total_s"] = sum(r["mechanics_s"] for r in result["rows"])

saved = fem_result["damage_total_s"] - gnn_result["damage_total_s"]

routes = ("FEM", "FEM + GNN")
```

(continues on next page)

(continued from previous page)

```

totals = (fem_result["damage_total_s"], gnn_result["damage_total_s"])
colours = ("#1f77b4", "#d62728")

fig, ax = plt.subplots(figsize=(8.6, 2.6))
bars = ax.barh(routes, totals, height=0.45, color=colours)
for bar, total in zip(bars, totals):
    ax.text(total + 0.02 * max(totals), bar.get_y() + bar.get_height() / 2,
            f"{total:.1f} s ({1000 * total / N_STEPS:.0f} ms per increment)",
            va="center", fontsize=11, color="#333333")

ax.invert_yaxis()
ax.set_xlim(0, max(totals) * 1.45)
ax.set_xlabel("Damage-subproblem time over the whole trajectory [s]")
ax.set_title("Cost of the damage subproblem", pad=14)
ax.grid(axis="x", alpha=0.25)
ax.set_axisbelow(True)
for side in ("top", "right", "left"):
    ax.spines[side].set_visible(False)

verdict = "faster" if saved > 0 else "slower"
fig.text(0.5, -0.08,
        f"Damage stage on the learned route: {abs(saved):.2f} s {verdict} "
        f"over {N_STEPS} increments on this hardware.\n"
        f"Mechanics is unchanged by the swap: "
        f"{fem_result['mechanics_total_s']:.1f} s and "
        f"{gnn_result['mechanics_total_s']:.1f} s.",
        ha="center", fontsize=10, color="#333333")
fig.tight_layout()
fig.savefig(OUTPUT / "damage_time_saving.png", dpi=180, bbox_inches="tight")
plt.show()

```

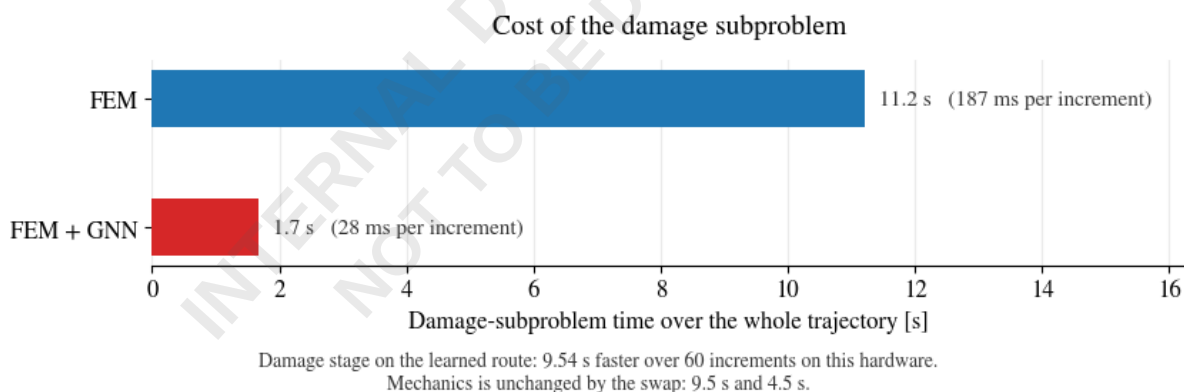


Fig. 33: Saved output from code cell 39.

16. Benchmark identical damage-subproblem inputs

Four stored FEM states are replayed with identical history and previous damage. One warm-up precedes three timed repetitions per method and state. CUDA is synchronised around timing so asynchronous work is not omitted.

Arithmetic counting is a separate, untimed pass. Multiply-add counts as two operations. Sparse factorisation work is estimated from factor sparsity. Special functions, comparisons and memory movement are not included in that FLOP estimate, although their elapsed time is included in the timed update. The counter is an algorithmic estimate, not a hardware performance counter.

Code cell 41

```

# 16. Measure damage cost on identical states
# Replay four FEM input states with identical history and previous damage.
# This section uses the same arithmetic-counting method as the research figure.
# Unlike that figure's archived five-step transitions, this script stores
# the actual previous increment, so each test represents one damage update.
#
# Count multiply-add as two operations. Dense layers and tensor arithmetic are
# counted; sparse LU is estimated from factor sparsity. Square roots/logarithms,
# comparisons, memory transfers, sparse ordering and supernodal padding are not
# included in the arithmetic estimate, but their runtime IS included in timing.
# Counting runs separately because instrumentation changes execution time.

cost_rows = []
profile_solver = new_solver()
for step, history, previous in fem_result["replay"]:
    for name, model in [("FEM", None), ("GNN", adapter)]:
        call = lambda: damage_update(profile_solver, history, previous, model)
        call(); synchronize() # warm-up excluded
        wall = []
        for repetition in range(3):
            synchronize(); start = time.perf_counter()
            _, measures = call()
            synchronize(); wall.append(time.perf_counter()-start)
        count = count_call(call)
        cost_rows.append(dict(route=name, step=step, wall_s=wall,
                             estimated_flops=count["estimated_flops"],
                             count_details=count, residual_l2=float(measures["kkt_projected_
l2"])))

```

```

[StaggeredSolver] Assembling solver (quasi_static)...
[FEMOperators] Initializing (spectral split, AT2)...
[FEMOperators] dt_CFL = 3.276809e-09 (c_p=5439282.93)
[PhaseFieldDamageSolver] Initializing CG solver (model=AT2, tol=1.0e-08, max_iter=1000,
bounds=projected_cg)...
[PhaseFieldDamageSolver] CG device: cpu, dtype: float64
[PhaseFieldDamageSolver] Ready (preconditioner=jacobi).
[StaggeredSolver] QuasiStaticSolver ready
[StaggeredSolver] State allocated: 3009 nodes, 5747 elements, device=cpu, dtype=torch.float64

```

17. Interpret the measured comparison

Code cell 43

```

# 17. Plot the measured cost of one damage update
# Report the measured ratio, even if it is below one. The ratio measures cost,
# not matched-accuracy acceleration: evaluating a GNN residual does not force
# its prediction to meet the FEM convergence tolerance. Colab hardware and the
# smaller mesh differ from the research benchmark, so its 17.7 ratio is not an
# expected result or a value to insert into this plot.

plt.rcParams.update({
    "font.family": "STIXGeneral", "mathtext.fontset": "stix",
    "font.size": 12, "axes.titlesize": 14, "axes.labelsize": 12,
})

names = ["FEM", "GNN"]
means = [np.mean([np.mean(r["wall_s"]) for r in cost_rows if r["route"] == n]) for n in names]
ratio = means[0] / means[1]

```

(continues on next page)

(continued from previous page)

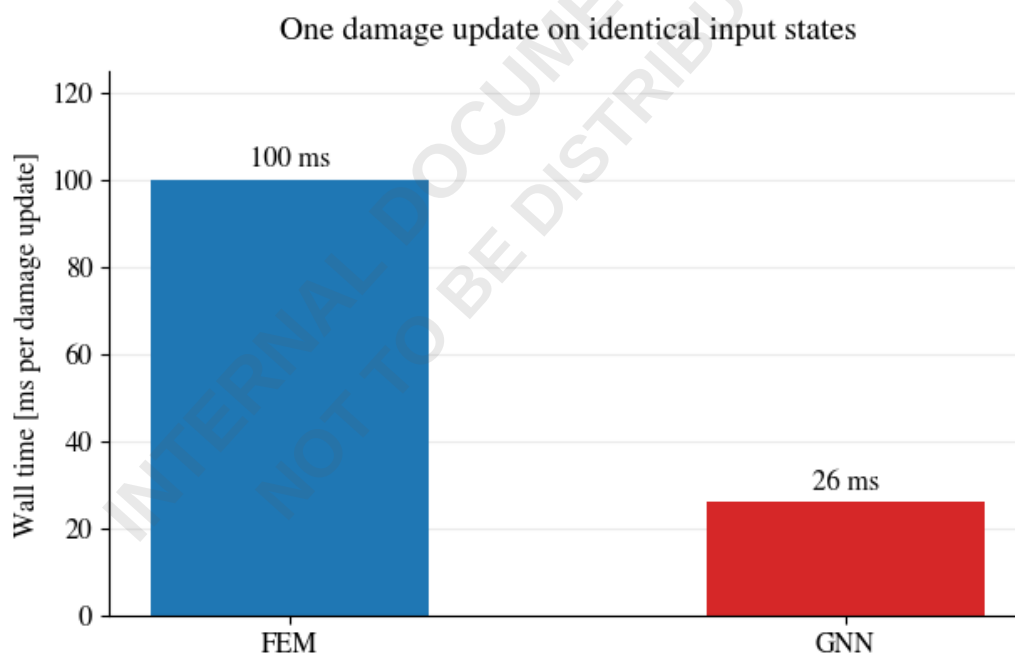
```

values = np.array(means) * 1000
fig, ax = plt.subplots(figsize=(6.4, 4.2))
ax.bar(names, values, color=["#1f77b4", "#d62728"], width=0.5)
for i, v in enumerate(values):
    ax.text(i, v + max(values) * 0.03, f"{v:.3g} ms", ha="center", fontsize=12)

ax.set_ylabel("Wall time [ms per damage update]")
ax.set_ylim(0, max(values) * 1.25)
ax.set_title("One damage update on identical input states", pad=14)
ax.grid(axis="y", alpha=0.25)
ax.set_axisbelow(True)
ax.spines[["top", "right"]].set_visible(False)

fig.text(0.5, -0.06,
        f"Same input states; bounds and residual evaluation included. "
        f"FEM/GNN time ratio = {ratio:.2f}.\n"
        "FEM mechanics excluded. GNN residual accuracy may differ; lower update "
        "cost on its own describes cost, not full-simulation acceleration.",
        ha="center", fontsize=9, color="#333333")
fig.tight_layout()
fig.savefig(OUTPUT / "damage_cost.png", dpi=200, bbox_inches="tight")
fig.savefig(OUTPUT / "damage_cost.svg", bbox_inches="tight")
plt.show()

```



Same input states; bounds and residual evaluation included. FEM/GNN time ratio = 3.85.
 FEM mechanics excluded. GNN residual accuracy may differ; lower update cost on its own describes cost, not full-simulation acceleration.

Fig. 34: Saved output from code cell 43.

Exercise 6 — State a defensible timing conclusion

Report the GPU model, mesh size, mean update times, time ratio and final field error together. Is the GNN faster for the damage stage? Is the complete loop faster? Explain any difference.

Exercise for independent work.

Hint

Mechanics, history and other loop work remain even if prediction is cheap. Do not call a stage-only ratio the total simulation speedup.

18. Save the calculation

Save fields, measurements, mesh and checkpoint identities, software versions, device information and the generated visuals. These records allow an instructor to distinguish differences in hardware from differences in the calculation.

Code cell 46

```
# 18. Save results
# Keep configuration, checkpoint identity, time samples and error measures
# alongside the saved fields and animations.
receipt = dict(configuration=dict(case="three_hole", steps=N_STEPS, h=.2, ell=.4,
                                hole_centres=HOLE_CENTRES, hole_radius=HOLE_RADIUS,
                                slot_length=2., slot_width=SLOT_WIDTH,
                                final_displacement=FINAL_DISPLACEMENT,
                                history_order="fresh", checkpoint_sha256=CHECKPOINT_SHA),
               environment=dict(torch=torch.__version__, network_device=DEVICE,
                                gpu=torch.cuda.get_device_name() if DEVICE=="cuda" else None,
                                threads=THREADS), cost_rows=cost_rows,
               fem_rows=fem_result["rows"], gnn_rows=gnn_result["rows"],
               final_relative_nodal_l2_error=float(relative_error),
               damage_stage_saved_s=saved, matched_state_time_ratio=ratio)

import scipy
import gmsh
receipt["environment"].update(numpy=np.__version__, scipy=scipy.__version__,
                             gmsh=gmsh.__version__, python=sys.version,
                             cuda_matmul_allow_tf32=torch.backends.cuda.matmul.allow_tf32)

receipt["archive_sha256"] = sha256(ARCHIVE)
receipt["mesh_sha256"] = sha256(mesh_path)
(OUTPUT / "measurements.json").write_text(json.dumps(receipt, indent=2))
np.savez_compressed(OUTPUT / "fields.npz", nodes=nodes, elements=elements,
                   fem=fem_result["fields"], gnn=gnn_result["fields"])
print("Saved results to:", OUTPUT)
```

Saved results to: /content/phast_demo_work/results_three_hole_classroom

19. Record your Colab result

The following summary uses only this execution. Export it with the figures and measurement samples so that your result can be reviewed after Colab disconnects.

Code cell 48

```
from IPython.display import display, Markdown
import platform
summary = {
    "device": DEVICE,
    "gpu": torch.cuda.get_device_name(0) if DEVICE == "cuda" else None,
    "nodes": len(nodes), "elements": len(elements), "steps": N_STEPS,
```

(continues on next page)

(continued from previous page)

```

"mean_fem_damage_ms": float(means[0] * 1000),
"mean_gnn_damage_ms": float(means[1] * 1000),
"damage_time_ratio": float(ratio),
"complete_loop_time_ratio": float(fem_result["total_s"] / gnn_result["total_s"]),
"final_relative_nodal_error": float(relative_error),
"fem_loop_s": fem_result["total_s"], "gnn_loop_s": gnn_result["total_s"],
"platform": platform.platform(), "require_gpu": REQUIRE_GPU,
"checkpoint_sha256": CHECKPOINT_SHA, "mesh_sha256": sha256(mesh_path),
}
(OUTPUT / "classroom_summary.json").write_text(json.dumps(summary, indent=2))
rows = [
    ("Inference device", summary["gpu"] or DEVICE),
    ("FEM damage update", f"{means[0]*1000:.2f} ms"),
    ("GNN damage update", f"{means[1]*1000:.2f} ms"),
    ("Damage-update time ratio (FEM/GNN)", f"{ratio:.2f}"),
    ("Complete-loop time ratio (FEM/GNN)", f"{summary['complete_loop_time_ratio']:.2f}"),
    ("Final relative nodal damage error", f"{relative_error:.2%}"),
]
display(Markdown("| Quantity | Measured value |\n|---|---|\n" +
    "\n".join(f"| {k} | {v} |" for k, v in rows)))
print("GNN damage updates were faster in this benchmark." if ratio > 1 else
    "GNN damage updates were not faster in this benchmark.")
print("This compares measured cost, not equal residual accuracy.")

```

Quantity	Measured value
Inference device	Tesla T4
FEM damage update	100.03 ms
GNN damage update	26.02 ms
Damage-update time ratio (FEM/GNN)	3.85
Complete-loop time ratio (FEM/GNN)	3.33
Final relative nodal damage error	10.26%

GNN damage updates were faster in this benchmark.
 This compares measured cost, not equal residual accuracy.

Optional extension — Repeat one experiment

After saving the default result, change one setting: use five timing repetitions instead of three, or deliberately run CPU inference. Restart and repeat the calculation, save each bundle separately and explain what changed. Do not change the geometry, material, mesh and device simultaneously.

Refining the mesh is a further experiment, not a guaranteed improvement. Preserve the physical phase-field length and record the new node/edge counts. Do not retrain or select different weights during a frozen-weight transfer test.

20. Download your results

This creates a ZIP of your calculation outputs, not another solver installation. The ZIP includes the GIFs, plots, mesh, field arrays and measurements. Download the executed notebook separately from Colab to retain your answers.

Code cell 50

```

import shutil
bundle = Path(shutil.make_archive(str(WORK / "three_hole_classroom_results"),
    "zip", root_dir=OUTPUT))

```

(continues on next page)

(continued from previous page)

```
print("Result bundle:", bundle)
if IN_COLAB:
    from google.colab import files
    files.download(str(bundle))
```

```
Result bundle: /content/phast_demo_work/three_hole_classroom_results.zip
```

Key takeaways

- Mechanics remains in PhAST for both loading paths.
- The frozen graph neural network directly replaces the damage subproblem in the hybrid route.
- Compare field accuracy and complete runtime; the matched damage-stage benchmark measures a separate part of the calculation.

Consolidation

What evidence is needed to assess a direct damage replacement?

Hint

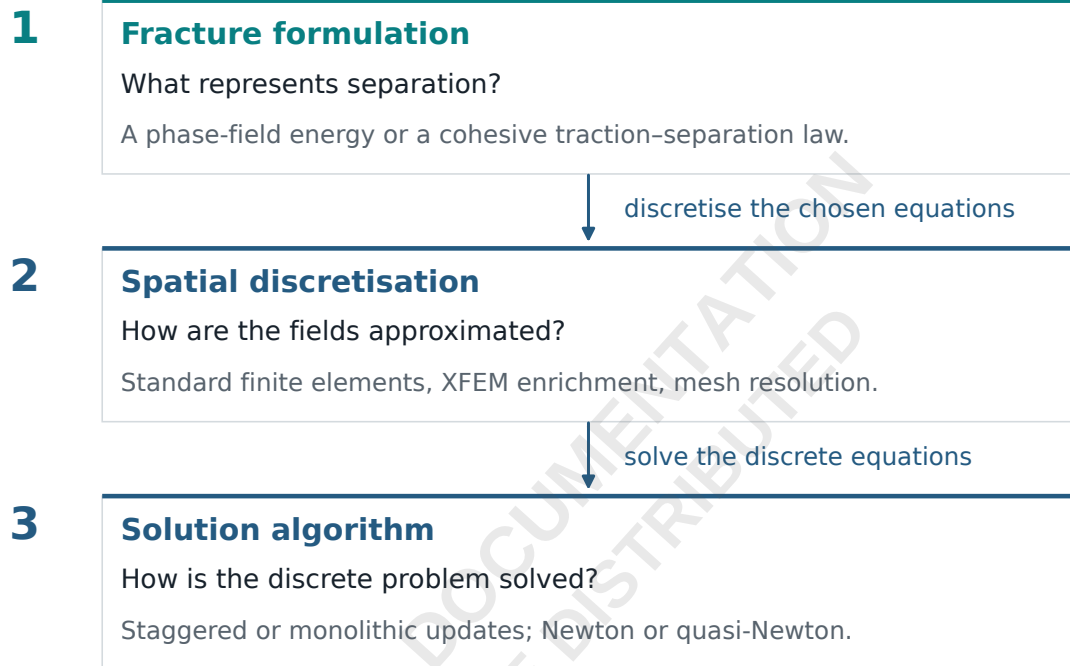
Compare the damage equation, constraints and the reference field.

Conceptual answer

Compare projected residuals, damage bounds, irreversibility and field differences with the classical reference. Measure complete runtime, including prediction and residual evaluation. This notebook does not apply an FEM correction or fallback to the GNN prediction.

Crack Representations: Sharp, Cohesive, and Diffuse

Three choices, three different questions



Choose a fracture model, spatial approximation and solution algorithm.

Fig. 35: Fracture formulation, spatial discretisation and solution algorithm are separate choices in a computational model.

How Should We Represent a Crack?

A brittle crack creates new surfaces and changes the transfer of force through a solid. A computational model must describe both the deformation of the body and the evolving fracture.

Sharp-crack models represent a surface inside the body. Cohesive models relate the force across an interface to its opening. Phase-field models describe fracture using a continuous damage field spread over a length scale ℓ .

The Starting Point: A Sharp Crack

In classical linear elastic fracture mechanics (LEFM), a crack is modeled as an exact lower-dimensional geometric boundary Γ embedded inside a solid body Ω . A Griffith-type energy balance compares the bulk elastic strain energy, external work, and the energy required to create new crack surface:

$$\mathcal{E}(u, \Gamma) = \int_{\Omega \setminus \Gamma} \psi(\varepsilon(u)) \, dx + G_c \mathcal{H}^{d-1}(\Gamma) - \mathcal{W}_{\text{ext}}(u).$$

Here:

- u is the displacement field,
- $\psi(\varepsilon(u))$ is the elastic strain energy density,
- G_c is the critical energy release rate (fracture toughness), and
- $\mathcal{H}^{d-1}(\Gamma)$ measures the crack surface area (or crack length in 2D).

In a mesh that conforms to the crack, growth can require changes to the mesh. Enriched and embedded discretisations provide other ways to represent sharp cracks, as the XFEM example below illustrates.

The energy balance principle originated with [Griffith \(1921\)](#) and was given its modern variational formulation for evolving cracks by [Francfort and Marigo \(1998\)](#).

Cohesive-zone models: separation on an interface

A cohesive-zone model (CZM) places a constitutive traction–separation law on an interface. If δ denotes displacement jump, a simple scalar picture is

$$T = T(\delta), \quad \Phi(\delta) = \int_0^\delta T(s) \, ds.$$

The area below $T(\delta)$ supplies an interface fracture energy. A CZM is especially natural when an interface is known in advance: an adhesive layer, a ply interface, or a prescribed material boundary. Its parameters can express a peak traction and a separation scale in addition to an energy.

The interface still needs to be represented numerically. It may coincide with element faces, embedded interface elements, or a tracked surface. A cohesive law is therefore a fracture formulation or constitutive law whose numerical representation is chosen separately. The classical computational formulation of [Xu and Needleman \(1994\)](#) is a standard reference.

Formulation and discretisation

A cohesive law can be coupled to several discretisations; it has a different physical idealisation from a diffuse crack-density regularisation.

XFEM: enrich an approximation space

The extended finite-element method (XFEM) represents a discontinuity or a near-tip feature by enriching a conventional finite-element approximation. A schematic displacement approximation is

$$u_h(x) = \sum_{i \in I} N_i(x) u_i + \sum_{j \in J} N_j(x) H(x) a_j + \sum_{k \in K} N_k(x) \sum_{\alpha} F_{\alpha}(x) b_{k\alpha}.$$

N_i are ordinary shape functions; H may encode a jump across a crack; and F_{α} can encode near-tip asymptotic behaviour. The index sets J and K select enriched nodes. XFEM changes the spatial approximation, allowing a crack to cut through elements independently of the element-edge alignment.

XFEM can reduce repeated remeshing for propagating cracks, but brings its own implementation questions: enrichment support, integration of cut cells, conditioning, branch handling, and the geometry used to describe the crack. The foundational paper is [Moes, Dolbow, and Belytschko \(1999\)](#).

Phase field: replace a surface by a narrow band

In a phase-field formulation, a scalar field $d(x)$ spreads the crack over a band of width governed by a length ℓ . With the convention used here, $d = 0$ is intact and $d = 1$ is broken. A typical crack-density functional is

$$\Gamma_\ell(d) = \frac{1}{c_0} \int_{\Omega} \left(\frac{w(d)}{\ell} + \ell |\nabla d|^2 \right) dx, \quad c_0 = 4 \int_0^1 \sqrt{w(s)} ds.$$

As ℓ becomes small under appropriate refinement and modelling assumptions, this regularised functional approximates a sharp fracture surface. Crack propagation is represented by the evolution of a continuous field d , whose diffuse band must be resolved on the mesh.

This is a regularised fracture formulation. It is commonly discretised with ordinary continuous finite elements, but the formulation and the discretisation are conceptually separate. The diffuse approach was developed for brittle fracture by [Bourdin, Francfort, and Marigo \(2000\)](#).

Comparing Crack Representations

Each representation offers distinct advantages depending on the engineering application:

Sharp Crack (LEFM)

- Geometric Representation: Exact lower-dimensional surface Γ
- Mesh Requirements: Depends on discretisation; a conforming approach aligns the mesh with the crack
- Crack Branching & Merging: Requires a numerical representation of evolving crack geometry
- Primary Engineering Applications: Fatigue crack propagation along known paths

Cohesive Zone (CZM)

- Geometric Representation: Traction–separation interface law
- Mesh Requirements: Interface elements or embedded interfaces
- Crack Branching & Merging: Depends on how new interfaces are introduced
- Primary Engineering Applications: Delamination in composites, adhesive joints, masonry

XFEM

- Geometric Representation: Displacement approximation enriched with jumps and near-tip functions
- Mesh Requirements: Background mesh with selected enriched nodes
- Crack Branching & Merging: Requires management of crack geometry and enrichment
- Primary Engineering Applications: Crack growth on a background mesh

Phase Field (PFM)

- Geometric Representation: Continuous scalar damage field $d(x) \in [0, 1]$
- Mesh Requirements: Mesh fine enough to resolve the damage band; assess sensitivity to h/ℓ
- Crack Branching & Merging: Represented through damage-field evolution; depends on the model, loading and resolution
- Primary Engineering Applications: Complex crack topologies, branching and coalescence in 2D/3D

When to Choose Phase-Field Fracture

Phase-field methods represent crack growth through an evolving field on the mesh. This makes branching and coalescence convenient to describe. The computed path and initiation load still depend on the energy model, length scale, loading and numerical resolution.

Separating Formulation, Discretization, and Solver

To reason clearly about any computational mechanics calculation, we maintain a clean separation between three levels:

1. **Continuum Formulation:**
The governing energy functional or constitutive equations (e.g., linear elasticity coupled to an AT1 or AT2 phase-field damage model).
2. **Spatial Discretization:**
The mathematical approximation space and mesh topology used to discretize continuous fields (e.g., standard continuous linear triangular T3 elements).
3. **Nonlinear Solution Strategy:**
The iterative numerical algorithm used to solve the coupled nonlinear system of equations (e.g., staggered alternating minimization, or monolithic Newton–Raphson).

Exercise: Identifying the Computational Layers

Consider the following statement from a research paper:

“We simulate crack branching using an AT2 phase-field model on triangular finite elements solved with a staggered alternating minimization scheme.”

Break this down by identifying:

1. The fracture formulation.
2. The spatial discretization.
3. The nonlinear solver.

Solution

1. **Fracture Formulation:** AT2 phase-field regularisation, with $w(d) = d^2$. The statement leaves the degradation law unspecified.
2. **Spatial Discretization:** Triangular finite elements. Their interpolation order is unspecified.
3. **Nonlinear Solver:** A staggered (alternating minimization) algorithm that decouples the mechanical equilibrium and damage updates within each load increment.

Summary of Key Concepts

- **Sharp vs. Diffuse:** Sharp crack models treat cracks as surfaces of discontinuity; phase-field models diffuse the crack over a narrow zone of characteristic width governed by the length scale ℓ .
- **Energy Balance:** In a phase-field model, the total energy consists of bulk elastic strain energy and surface fracture dissipation.
- **Modularity:** Distinguishing formulation, discretization, and solver ensures clarity when comparing numerical results and computational costs.

Fracture Energy: AT1, AT2, Degradation, and Bandwidth

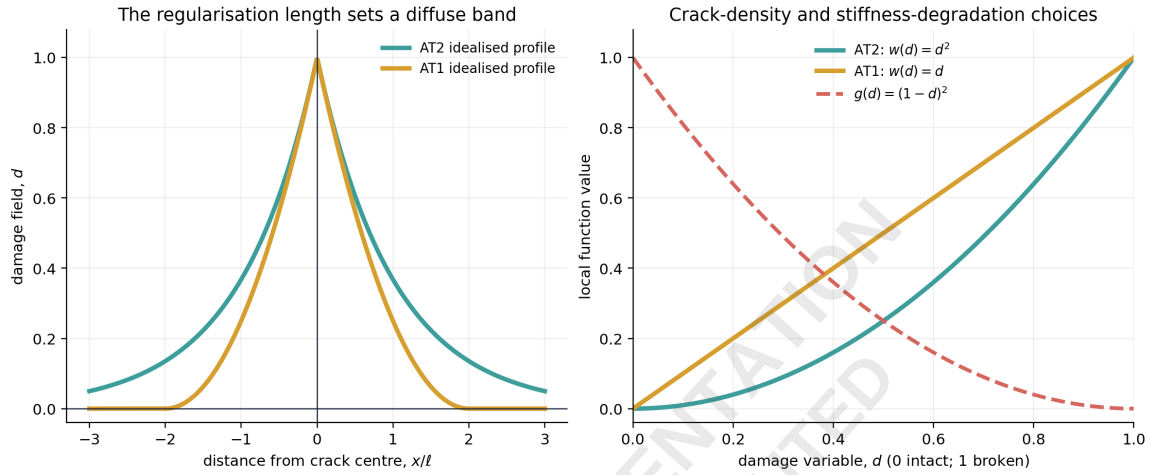


Fig. 36: The crack-density choice and the stiffness degradation law are independent mathematical functions. Together, they govern how damage nucleates, diffuses, and softens the solid.

The Energy Balance: Why Do Materials Break?

Loading stores elastic energy in a solid. Crack extension releases some of that energy and consumes energy in creating new surfaces. In the quasistatic Griffith description, the onset of crack extension satisfies

$$G = -\frac{\partial \Pi}{\partial A} = G_c$$

where Π is the mechanical potential energy, A is crack area, and G_c is the critical energy release rate, measured in J/m^2 or N/m .

In phase-field fracture modeling, we approximate Griffith's surface energy balance by introducing a continuous scalar damage field $d(x) \in [0, 1]$ across the entire solid domain Ω :

- $d = 0$ (Intact Material): The material is completely undamaged, fully elastic, and bears its full load capacity.
- $d = 1$ (Fully Damaged Material): The selected elastic stiffness reaches its residual value, set here by η_{res} .
- $0 < d < 1$ (Diffuse Crack Band): Damage changes smoothly over a distance governed by ℓ and the selected crack-density model.

The Phase-Field Energy Functional

For small-strain brittle fracture, the total potential energy of the coupled displacement-damage system is formulated as:

$$\mathcal{E}_\ell(u, d) = \int_{\Omega} \left[g_\eta(d) \psi^+(\varepsilon(u)) + \psi^-(\varepsilon(u)) + \frac{G_c}{c_0} \left(\frac{w(d)}{\ell} + \ell |\nabla d|^2 \right) \right] dx - \mathcal{W}_{\text{ext}}(u).$$

The ingredients have different jobs:

- ψ^+ is the part of elastic energy selected to drive fracture, often a tensile contribution;
- ψ^- retains its full contribution in a tension–compression split;
- $g_\eta(d)$ reduces tensile stiffness as damage grows;
- G_c sets the energy scale of fracture;
- $w(d)$ selects a crack-density family; and
- ℓ controls the regularised width of the damage band.

Spectral, volumetric–deviatoric, and other decompositions define ψ^+ and ψ^- differently. State the split before interpreting compression, crack closure, or mixed-mode behaviour.

The damage convention and degradation derivative

This book uses $d = 0$ for intact material and $d = 1$ for fully broken material. A standard residual-stiffness degradation law is

$$g_\eta(d) = (1 - \eta_{\text{res}})(1 - d)^2 + \eta_{\text{res}}, \quad 0 < \eta_{\text{res}} \ll 1.$$

It satisfies $g_\eta(0) = 1$ and $g_\eta(1) = \eta_{\text{res}}$. The residual term helps prevent the elastic operator from becoming singular in a damaged zone. It should be reported because it affects the numerical problem and the post-peak response.

This normalised quadratic law is the reference convention used in this chapter. NB1 writes the closely related additive form $(1 - d)^2 + \eta$, for which $g(0) = 1 + \eta$. NB3 retains its trained model’s rational law. Keep these definitions distinct when comparing derivatives or implementing an example.

For a hand calculation or a tensor implementation, differentiate explicitly:

$$g'_\eta(d) = -2(1 - \eta_{\text{res}})(1 - d), \quad g''_\eta(d) = 2(1 - \eta_{\text{res}}).$$

The negative first derivative is meaningful: increasing d lowers the degraded tensile energy. The resulting damage evolution also depends on fracture density, gradients, boundary data, and irreversibility through the total variation.

Hands-On Lab: Verifying the Degradation Derivative

In the accompanying computational lesson, you will implement this quadratic degradation law in PyTorch and compute its derivative:

- Analytically: Using explicit calculus, $g'_\eta(d) = -2(1 - \eta_{\text{res}})(1 - d)$.
- Automatically: Using PyTorch’s automatic differentiation engine (`torch.autograd`).
- Numerically: Using central finite differences with varying perturbation step sizes.

Optional Tutorial (Autograd vs. Finite Differences for Degradation)

In [labs/02 degradation autograd](#), compare the analytic derivative of the degradation law with automatic differentiation and finite differences.

Automatic differentiation applies the chain rule to the operations in the code. The analytic and finite-difference calculations provide independent checks for this example.

AT2 and AT1 are normalised choices

The crack-density contribution is often written

$$\Gamma_\ell(d) = \frac{1}{c_0} \int_{\Omega} \left(\frac{w(d)}{\ell} + \ell |\nabla d|^2 \right) dx, \quad c_0 = 4 \int_0^1 \sqrt{w(s)} ds.$$

With this convention:

$$\begin{aligned} \text{AT2: } w(d) &= d^2, & c_0 &= 2, \\ \text{AT1: } w(d) &= d, & c_0 &= \frac{8}{3}. \end{aligned}$$

The labels AT1 and AT2 are widely used, but coefficients in papers can look different because authors absorb factors into G_c , c_0 , or the gradient term. Compare the complete functionals and their normalisation.

Why the profiles look different

At a fully developed straight crack in an idealised one-dimensional setting, the AT2 profile has exponential tails,

$$d_{\text{AT2}}(x) = \exp(-|x|/\ell),$$

whereas the corresponding AT1 profile is compactly supported,

$$d_{\text{AT1}}(x) = \left(1 - \frac{|x|}{2\ell} \right)_+^2, \quad (a)_+ = \max(a, 0).$$

These ideal profiles explain the left panel of the preceding figure. Applications in finite domains require a mesh-convergence study with their boundary conditions and loading path.

In common one-dimensional analyses, AT1 exhibits a finite damage-initiation threshold, whereas AT2 can begin to reduce stiffness from the onset of tensile loading. That statement depends on the surrounding energy, loading, and history construction, which determine how the initiation thresholds compare.

Deriving the strong-form damage residual

Hold u fixed momentarily and take a virtual change δd . The damage-dependent part of the first variation is

$$\delta_d \mathcal{E}_\ell = \int_{\Omega} \left[g'_\eta(d) \psi^+ \delta d + \frac{G_c}{c_0} \left(\frac{w'(d)}{\ell} \delta d + 2\ell \nabla d \cdot \nabla(\delta d) \right) \right] dx.$$

Integrating the gradient term by parts gives, away from active constraints,

$$r_d = g'_\eta(d) \psi^+ + \frac{G_c}{c_0} \left(\frac{w'(d)}{\ell} - 2\ell \Delta d \right) = 0.$$

A natural boundary condition for an unconstrained boundary is $\nabla d \cdot n = 0$. The equation has three visible influences:

1. $g'_\eta(d) \psi^+$ drives damage where the selected elastic energy is high;
2. $w'(d)/\ell$ penalises local damage; and
3. $-2\ell \Delta d$ penalises abrupt spatial variation.

The derivation identifies a diffusion-like gradient term coupled to the mechanical driving energy and a one-way damage history. These latter terms give the equation its fracture interpretation.

Enforcing irreversibility

For brittle damage, a common admissible set at load step n is

$$d_n(x) \geq d_{n-1}(x), \quad 0 \leq d_n(x) \leq 1.$$

The equation $r_d = 0$ applies away from active constraints. At an active lower bound, a multiplier balances the residual. Displaying the lower-bound constraint gives the local complementarity form

$$r_d - \lambda = 0, \quad \lambda \geq 0, \quad d - d_{n-1} \geq 0, \quad \lambda(d - d_{n-1}) = 0.$$

The upper bound introduces a second multiplier. Some implementations impose these bounds directly; others use a history field to retain the previous maximum driving energy. The relation between these approaches depends on the formulation. Inspect the stored damage history when checking irreversibility.

Width, mesh, and what must be checked

The parameter ℓ is a modelling and numerical length. It spreads the regularised crack and therefore interacts with mesh spacing h , geometry, and material calibration. A credible study reports ℓ , the mesh, and a convergence or sensitivity check appropriate to that combination.

Changing ℓ can change:

- the apparent width of the damaged band;
- the peak load and softening response in a finite discretisation;
- the required mesh density and nonlinear conditioning; and
- the meaning of a comparison with an observed fracture process zone.

Interpreting the regularised width as a physical crack width requires calibration for the chosen material and model.

Exercise: one derivative and one scale argument

1. Evaluate $g_\eta(0)$, $g_\eta(1)$, and $g'_\eta(1)$ for the law above.
2. In the damage residual, which term becomes large when ℓ is made small at fixed d and ∇d ?
3. Why is it incomplete to infer mesh independence from an unchanged full-domain average of d ?

i Solution

1. $g_\eta(0) = 1$, $g_\eta(1) = \eta_{\text{res}}$, and $g'_\eta(1) = 0$. The derivative vanishes at the fully degraded endpoint for this quadratic law.
2. The local crack-density contribution $w'(d)/\ell$ scales inversely with ℓ . This scaling describes the local term at fixed fields; the full solution also depends on changes in the gradient and solution fields.
3. A spatial average can remain similar while the crack band moves, changes width, or is under-resolved. Compare fields, energies, and observables at stated refinements.

In the computational lesson, the first result compares the analytic, automatic-differentiation, and finite-difference derivatives of the selected degradation law at one stated damage value, with the remaining material inputs held fixed.

Sources for this chapter

The Ambrosio–Tortorelli regularisation idea underlies the AT terminology. For phase-field fracture derivations and comparisons, see [Miehe, Welschinger, and Hofacker \(2010\)](#), [Pham et al. \(2011\)](#), and the review by [Kristensen, Niordson, and Martínez-Pañeda \(2021\)](#).

INTERNAL DOCUMENTATION
NOT TO BE DISTRIBUTED

The Staggered Solution Algorithm: Alternating Mechanics and Damage

Alternate fields at one load level

Load increment n : initialise $d^{(0)} = d_{n-1}$; iteration $k = 0$.

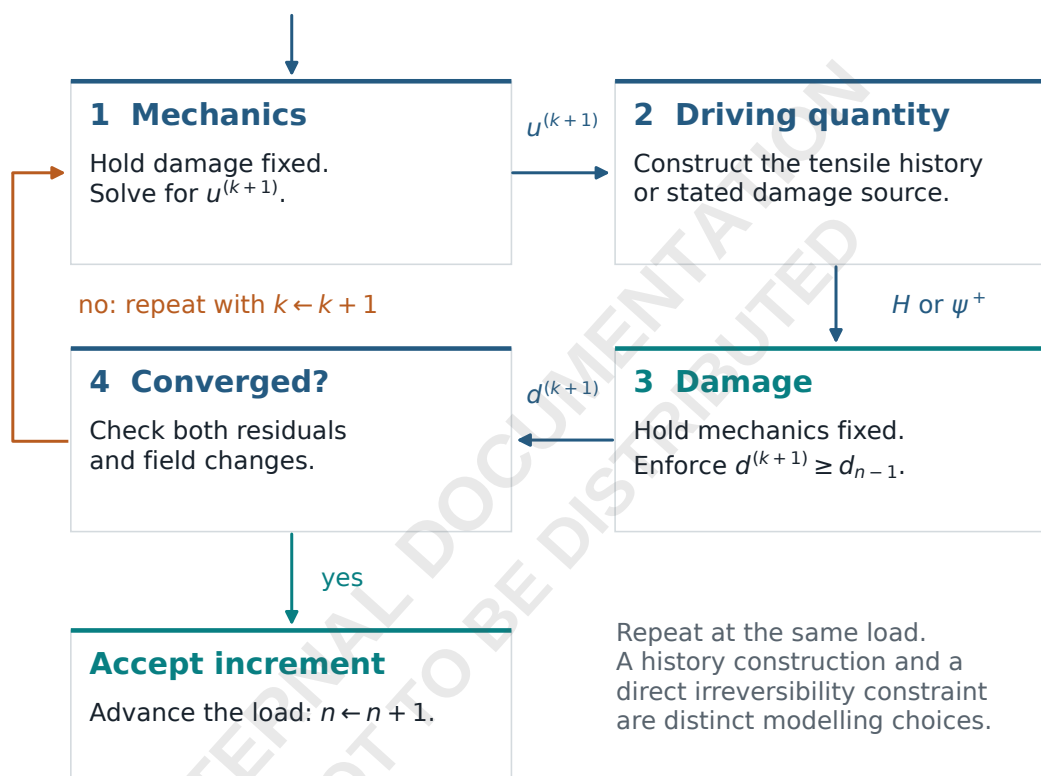


Fig. 37: Within each load increment, the staggered scheme alternates between updating displacement with damage fixed, and updating damage with displacement fixed, until equilibrium residuals converge.

Alternating the coupled fields

This chapter describes a converged quasistatic staggered scheme. NB1 instead advances dynamic mechanics explicitly, while NB3 uses one mechanics–damage pass per load increment. Neither practical should be interpreted as executing the full outer convergence loop illustrated above.

Displacement and damage influence one another. Deformation supplies the energy that drives damage, and damage changes the stiffness and redistributes stress.

A staggered scheme solves these coupled equations in successive subproblems:

1. Hold damage fixed and solve the mechanical problem.
2. Hold displacement fixed and solve the damage problem with its irreversibility constraints.

3. Check the field changes and residuals, then repeat at the same load level until the convergence criteria are satisfied.

The material law and energy split determine whether the mechanical subproblem is linear. Convexity of the damage subproblem depends on the chosen energy and constraints. Staggering simplifies each update, although strong coupling can require many iterations.

The Mechanical Subproblem

For a quasistatic, small-strain continuum body, the balance of linear momentum requires:

$$\nabla \cdot \sigma(u, d) + b = 0 \quad \text{in } \Omega,$$

with displacement or traction boundary conditions on complementary portions of the boundary. The strain is

$$\varepsilon(u) = \frac{1}{2} (\nabla u + \nabla u^T).$$

In a phase-field model the stress depends on damage through the selected energy split. A representative relation is

$$\sigma(u, d) = g_\eta(d) \frac{\partial \psi^+}{\partial \varepsilon} + \frac{\partial \psi^-}{\partial \varepsilon}.$$

Holding d fixed isolates the mechanical subproblem. Its linearity depends on the material law, strain measure, contact, and boundary conditions.

The weak form asks for a displacement u such that, for all admissible test fields v ,

$$\int_{\Omega} \sigma(u, d) : \varepsilon(v) \, dx = \int_{\Omega} b \cdot v \, dx + \int_{\Gamma_t} \bar{t} \cdot v \, ds.$$

This is the bridge from a continuum equation to finite elements. Shape functions turn u and v into finite vectors of degrees of freedom; an assembly or matrix-free operation then evaluates the residual.

Why the damage equation resembles diffusion

From the fracture-energy chapter, the unconstrained damage residual contains

$$\frac{w'(d)}{\ell} - 2\ell \Delta d.$$

The Laplacian has the same mathematical form as a diffusion operator. Compare the steady heat equation

$$-\nabla \cdot (k \nabla T) = q$$

with a schematic fixed-mechanics damage equation

$$-2\ell \Delta d + \frac{w'(d)}{\ell} = -\frac{c_0}{G_c} g'_\eta(d) \psi^+.$$

Both equations spread a field through a gradient penalty or conductivity-like term. The analogy helps explain the role of boundary conditions and element gradients. The damage problem has three additional features:

- the right-hand side is coupled to displacement and a selected tensile energy;
- d follows a constrained, nondecreasing damage history; and
- the coefficients and source can change during nonlinear iteration.

These coupling and history terms give the damage equation its fracture interpretation.

One load increment, two coupled updates

At load increment n , start with the previously accepted damage d_{n-1} . A simple staggered pattern is:

1. initialise $d_n^{(0)} = d_{n-1}$;
2. solve the mechanics problem for $u_n^{(k+1)}$ with $d_n^{(k)}$ fixed;
3. construct the specified damage-driving quantity from $u_n^{(k+1)}$;
4. solve the constrained damage problem for $d_n^{(k+1)}$;
5. test convergence of both fields and the relevant residuals; and
6. either repeat at this load level or accept the increment.

The figure above is a visual version of this algorithm. A history-field implementation may use, for example,

$$H_n(x) = \max(H_{n-1}(x), \psi^+(\varepsilon(u_n(x))))$$

to prevent loss of the tensile driving force on unloading. This is one commonly used history construction. Other approaches impose $d_n \geq d_{n-1}$ directly in the damage solve.

Staggered, monolithic, Newton, and quasi-Newton

These terms answer different algorithmic questions.

Staggered solve. Update u and d in blocks. It can be easier to implement and diagnose because each sub-problem has a recognisable role. It may need several block iterations per load step and can be sensitive to coupling strength and stopping criteria.

Monolithic solve. Solve for the coupled state (u, d) together. This can better represent strong coupling, but requires a larger coupled residual, Jacobians or Jacobian actions, and careful treatment of inequality constraints.

Newton method. Linearise a residual around the current state using its Jacobian. Quadratic local convergence requires a sufficiently smooth residual, a nonsingular Jacobian, and an iterate close enough to the solution.

Quasi-Newton method. Update an approximate Jacobian or inverse Jacobian from changes in residuals and states. BFGS is a well-known example for optimisation. Quasi-Newton therefore names the nonlinear solve strategy applied to the chosen fracture formulation and spatial approximation.

Assess convergence using the block iteration count, tolerance, residual definition, and accepted load increments.

Static and dynamic fracture

The quasistatic balance above neglects inertia. A dynamic model includes it:

$$\rho \ddot{u} - \nabla \cdot \sigma(u, d) = b.$$

Now time integration, mass treatment, wave propagation, damping choices, and time-step size affect the numerical result. A dynamic calculation may show a different crack path or timing from a quasistatic calculation even with the same stored energy, because the kinetic-energy and loading-rate terms differ.

The phrase “dynamic phase field” should therefore be accompanied by the time integrator, time step, inertia/damping assumptions, and the quantity used to enforce or approximate irreversibility.

Interpreting convergence

At an accepted increment, consider at least the following checks:

- mechanical balance: a stated residual norm, reaction balance, or weak form defect;
- damage update: a stated residual or constrained-solver criterion;
- irreversibility: verify $d_n \geq d_{n-1}$ within the documented numerical tolerance;
- bounds: inspect whether d respects the intended interval;
- energy interpretation: evaluate the balance using stated terms and sign conventions; and
- field inspection: view u , d , and the driving quantity alongside the scalar convergence history.

Interpret the iteration count together with the residual definition. Field inspection helps identify an incorrect boundary condition, a flipped damage convention, or an under-resolved band even when the residual decreases.

Applying the equations in PhAST

The next chapter introduces a dynamic notched-plate calculation. Inspect how the displacement update, damage solve and physical time step appear in its configuration. Compare successive damage fields to locate crack growth, and use the energy histories to interpret the accompanying mechanical response.

Exercise: write the update before reading the code

Suppose a notched specimen is loaded in displacement control. Put the following actions in a sensible order and identify the action that enforces one-way damage:

- solve for displacement;
- accept the new load step;
- update the damage-driving quantity;
- check field/residual changes;
- solve for damage;
- initialise from the previous accepted damage.

Solution

Start by initialising from the previous accepted damage. Solve displacement at the current trial damage, update the stated driving quantity, and solve the damage subproblem subject to its irreversibility rule. Then check residuals and field changes, and accept the load step when the checks pass. The lower-bound constraint $d_n \geq d_{n-1}$, or a history construction designed to enforce its effect, is the one-way component. This ordering describes the quasistatic staggered scheme. In the dynamic classroom example, the mechanical update also evolves velocity and acceleration over physical time.

Sources for this chapter

For a variational phase-field treatment and a staggered implementation route, see [Miehe, Welschinger, and Hofacker \(2010\)](#). A discussion of numerical formulations and length-scale effects is provided by [Gerasimov and De Lorenzis \(2019\)](#).

From Weak Forms to Discrete Tensors: The PhAST Pipeline

One FEM model, two operator routes

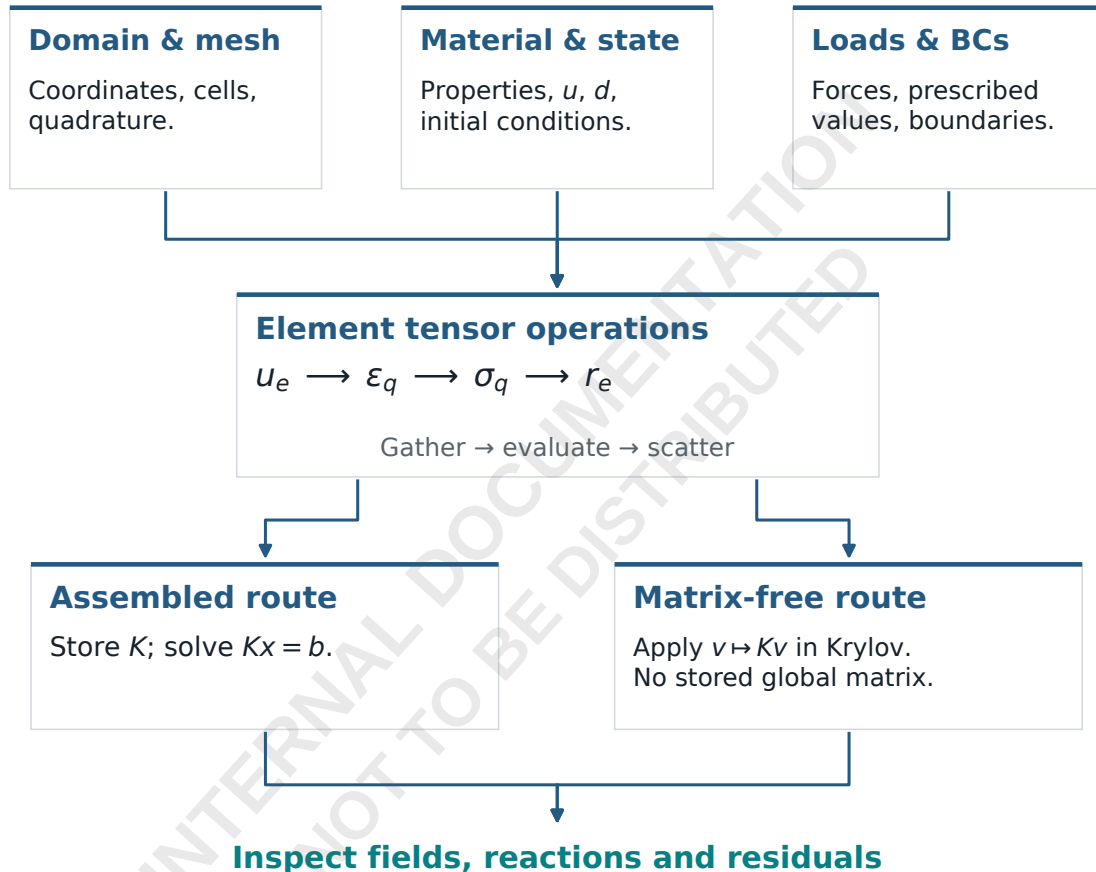


Fig. 38: The finite element pipeline translates physical boundary-value problems into discrete tensor operations, connecting continuous weak forms to PyTorch data structures.

Expressing finite elements with tensors

A finite-element calculation repeats similar operations over many elements and quadrature points. Tensor operations group these calculations so that libraries such as PyTorch can evaluate them in batches.

For a mesh of linear triangles, the main arrays contain node coordinates, element connectivity, shape-function gradients and field values. Their dimensions identify the element, quadrature point, local node and physical component used by each operation.

Batching can improve performance on CPUs and GPUs. Automatic differentiation is available when the operations and solver components support the required derivatives.

Defining the Boundary-Value Problem

Every computational mechanics simulation begins with a well-posed boundary-value problem:

- Domain Geometry (Ω): The specimen dimensions, material boundaries, and initial notch geometry.
- Boundary Conditions: Prescribed displacements $\bar{u}$ on Dirichlet boundaries Γ_u , and applied tractions $\bar{t}$ on Neumann boundaries Γ_t :

$$u = \bar{u} \quad \text{on } \Gamma_u, \quad \sigma \cdot n = \bar{t} \quad \text{on } \Gamma_t.$$

- Precrack Representation: Whether a notch is modeled as a geometric cut or an initial seeded damage field ($d_0 = 1$ along the notch line).
- Mesh Resolution: Choose the element size h to resolve the damage band, and examine how the results change under refinement at fixed ℓ .

State how the notch is represented: a prescribed initial damage field or a geometric cut. These choices encode different computational objects and can produce visually similar pictures.

From cells to fields

For a two-dimensional mesh, common arrays are:

- coordinates $X \in \mathbb{R}^{N_{\text{node}} \times 2}$;
- cell connectivity $C \in \{0, \dots, N_{\text{node}} - 1\}^{N_{\text{cell}} \times n_e}$;
- nodal displacement $U \in \mathbb{R}^{N_{\text{node}} \times 2}$; and
- nodal damage $D \in \mathbb{R}^{N_{\text{node}}}$.

The cell connectivity identifies which rows of X , U , and D belong to each element; the field arrays store the corresponding values. For an element e with local nodes a , finite-element interpolation takes the form

$$u_h(x) = \sum_{a=1}^{n_e} N_a(x) u_a, \quad d_h(x) = \sum_{a=1}^{n_e} N_a(x) d_a.$$

Differentiating the shape functions maps nodal displacement to strain. In matrix notation at one quadrature point,

$$\varepsilon_h = B_e u_e.$$

The element residual is computed by integrating the weak form. A schematic mechanical contribution is

$$r_e(u, d) = \int_{\Omega_e} B_e^T \sigma(u_h, d_h) dx - f_e^{\text{ext}}.$$

Assembly scatters each local residual into the appropriate entries of the global residual. The same operation can be expressed with index operations, scatter-adds, or framework-specific sparse primitives.

Tensor dimensions and indexing

Write expected shapes next to a tensor operation. For example, with batched cells and quadrature points, an implementation might use:

- cell coordinates: $(N_{\text{cell}}, n_e, 2)$;
- shape-function gradients: $(N_{\text{cell}}, N_q, n_e, 2)$;
- gathered displacement: $(N_{\text{cell}}, n_e, 2)$;
- strain or stress: $(N_{\text{cell}}, N_q, n_e)$; and
- quadrature weights: (N_{cell}, N_q) .

The exact ordering is a convention. A transpose that preserves the total number of entries can still mix components, cells, and quadrature points. Use small assertions and one element-level hand calculation before relying on a large field plot.

i Indexing check

Select one cell and one quadrature point. Write the gathered nodal values, shape gradients, strain, stress, and residual contribution on paper. If those five objects agree with the tensor code, then vectorising over cells has a clear reference calculation.

Assembled and matrix-free routes

For a linearised subproblem, an assembled finite-element method forms a global matrix

$$K = \sum_e A_e^T K_e A_e$$

and solves $Kx = b$ or a linearised residual equation. A_e symbolically represents the local-to-global map.

A matrix-free method implements the action

$$v \mapsto Kv$$

by gathering v to elements, applying local operations, and scattering contributions back. An iterative Krylov solver can use this implicit representation of K .

Matrix-free evaluation uses local operator actions and associated element data. Its cost depends on residual/Jacobian actions, preconditioning, data movement, and stopping criteria. Compare memory use and runtime at matched physics, mesh, tolerance, and reported observable.

Static and dynamic arrays

In a quasistatic calculation, the state often consists primarily of U and D . A dynamic calculation adds at least velocity and acceleration or their time-integration equivalents. A schematic time step must then track

$$(U_n, V_n, A_n, D_n) \longrightarrow (U_{n+1}, V_{n+1}, A_{n+1}, D_{n+1}).$$

The update rules for these arrays express inertia and affect the computed path. A sequence of prescribed displacement increments represents load stepping; a dynamic calculation also evolves the inertial state.

Where automatic differentiation fits

PyTorch provides tensors and a reverse-mode automatic-differentiation system. The user or a library defines mesh data, interpolation, constitutive response, residuals, boundary conditions, and solver behaviour.

JAX likewise provides array transformations such as differentiation, compilation, and vectorisation. JAX-FEM is an additional finite-element library built on that ecosystem and includes worked mechanics examples. A suitable choice depends on the existing solver, device support, reproducibility needs, and whether the relevant operations are differentiable in the intended computation.

See the official [PyTorch autograd documentation](#), the [JAX automatic-differentiation guide](#), and the [JAX-FEM quickstart](#).

Inspecting results: four views, one interpretation

For a tiny evolving-fracture run, inspect at least:

1. the geometry, mesh, and loading direction;
2. displacement or deformed configuration with a stated scale factor;
3. damage field with a labelled colour range and the $d = 0/d = 1$ convention; and
4. a scalar observation such as reaction force, energy term, or residual.

Hands-On Lab: Running an End-to-End PhAST Fracture Simulation

In the accompanying computational notebook, you will execute a complete phase-field fracture calculation using the PhAST solver:

1. Geometry & Meshing: Generate a 40 mm square specimen with a 20 mm edge notch and a graded triangular Gmsh mesh.
2. Boundary Conditions: Use named regions for symmetric vertical opening and horizontal restraint on the lateral edges.
3. Solver Execution: Run explicit dynamics with an implicit AT2 damage update over 50 microseconds, using a mesh-dependent stable time step.
4. Post-Processing: Reload saved fields, inspect displacement, strain, stress and damage, and animate them with fixed colour scales.

Hands-On Tutorial: Lab 01 (End-to-End PhAST Fracture Simulation)

In [classroom/01 simulate fracture](#), configure the notched plate, run PhAST and interpret the saved fields and energy histories.

This notebook serves as our reference simulation pipeline throughout the course. You will see how tensor representations map directly to physical finite element fields.

Exercise: identify a silent shape error

A tensor containing displacement at cell nodes has shape $(N_{\text{cell}}, n_e, 2)$. A tensor of shape-function gradients has shape $(N_{\text{cell}}, N_q, n_e, 2)$. Which new axes must be aligned or introduced before forming a displacement gradient at every quadrature point, and what physical quantity should you recover for an affine displacement field?

Solution

The nodal displacement needs a quadrature axis, for example by treating it as $(N_{\text{cell}}, 1, n_e, 2)$, so that it aligns with the N_q axis of the shape gradients. Contract the local-node axis and preserve the displacement component and spatial-derivative axes according to the chosen convention. For an affine displacement field, the displacement gradient and its symmetric strain should be constant across all elements and quadrature points (up to round-off). This gives a simple reference calculation for checking the tensor implementation.

Take-away

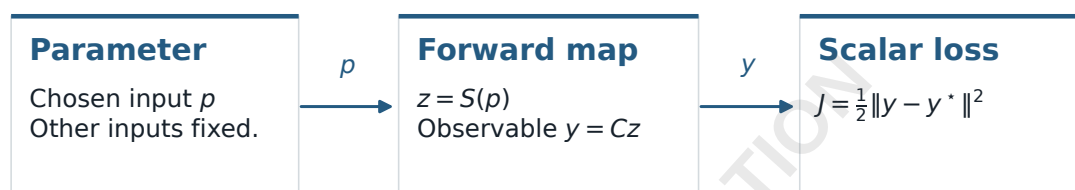
Tensors make data movement and differentiation explicit. The finite-element description specifies what is interpolated, where it is integrated, how boundary conditions are enforced, and how the resulting field is checked.

INTERNAL DOCUMENTATION
NOT TO BE DISTRIBUTED

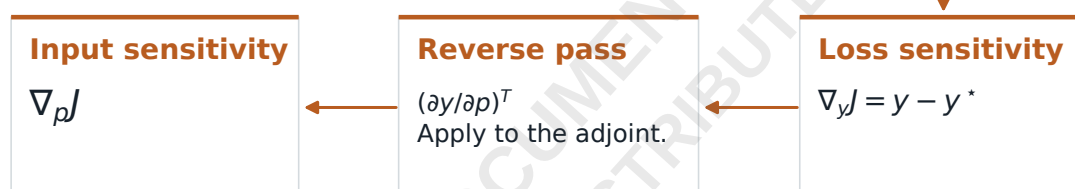
Differentiable Mechanics and Inverse Parameter Discovery

Gradients and optimisation updates

FORWARD · evaluate the stated computation



REVERSE · accumulate vector-Jacobian products



$$\nabla_p J = (\partial y / \partial p)^T \nabla_y J$$

CHECK $\frac{J(p + hq) - J(p - hq)}{2h} \approx (\nabla_p J)^T q$

OPTIONAL Optimise: $p_{\text{new}} = p - \alpha \nabla_p J$

Inverse uniqueness depends on the observations, loading and parameterisation.

Fig. 39: The forward solver maps parameters to observations. Reverse-mode automatic differentiation computes a loss gradient that an optimiser can use to update the parameter estimates.

Estimating a material property from measurements

A tensile test provides measurements such as reaction force and displacement. An inverse problem uses those measurements to estimate a material property. For example, if the geometry and applied force are known, the extension of an elastic bar can be used to estimate Young's modulus.

We compare the simulated observation $y(p)$ with the measured value and express their difference through a loss J . Automatic differentiation computes how that loss changes with the parameter p by applying the chain rule to the supported simulation operations. For an observation depending on the displacement u , this gives

$$\frac{\partial J}{\partial p} = \frac{\partial J}{\partial y} \cdot \frac{\partial y}{\partial u} \cdot \frac{\partial u}{\partial p}.$$

An optimiser uses this gradient to update the estimate. Recovery depends on how strongly the measurements constrain the parameter, as well as measurement noise, model assumptions and numerical accuracy.

Formulating the Forward and Inverse Maps

Let p denote the physical parameter we wish to discover (for example, Young's modulus $E > 0$). The forward computational graph maps the parameter to a physical state, and then to a measurable observation:

$$p \mapsto (u(p), d(p)) \mapsto y(p).$$

With a target observation $y^\star$, an illustrative least-squares objective is

$$J(p) = \frac{1}{2} \|y(p) - y^\star\|_2^2.$$

This objective introduces the chain rule, tensor shapes, and optimisation bookkeeping. Parameter identifiability depends on the observations, loading conditions, and chosen parameterisation.

Before calling a gradient routine, write down:

- the parameter and its units or nondimensionalisation;
- the geometry, loads, and boundary conditions held fixed;
- the observable y and the target $y^\star$;
- every mathematical operation from p to J ; and
- the stopping tolerances and convergence criteria of any iterative solver.

These choices define the mathematical map being differentiated.

Three ways to obtain a sensitivity

Finite differences

A directional central difference uses a chosen direction q and step h :

$$D_h J(p; q) = \frac{J(p + hq) - J(p - hq)}{2h}.$$

It is simple and independent of the autodiff implementation, which makes it valuable as a check. It also has a step-size trade-off: a large h includes nonlinear curvature, while a very small h can suffer cancellation and solver tolerance effects.

Unrolled automatic differentiation

If a program executes a fixed differentiable sequence of operations, reverse mode can differentiate the sequence actually run. For example, unrolling K iterations gives

$$x_0 \rightarrow x_1 \rightarrow \dots \rightarrow x_K \rightarrow J.$$

The derivative is of that truncated algorithm. If K changes by parameter value or a non-differentiable branch is taken, the interpretation must mention it. Memory cost can also increase when many states are retained for a reverse pass.

Implicit differentiation

Suppose a converged state is defined by a residual

$$R(x, p) = 0.$$

Under appropriate differentiability and nonsingularity assumptions,

$$\frac{dx}{dp} = -\left(\frac{\partial R}{\partial x}\right)^{-1} \frac{\partial R}{\partial p}.$$

In practice, an adjoint formulation often avoids forming this inverse explicitly. This method differentiates a local equation for a converged state. Differentiating the iterations gives the derivative of the executed algorithm. Inequality constraints, active-set changes, damage irreversibility, and non-smooth constitutive choices require particular care.

Follow the gradient one update at a time

The worked section below expands the chain rule into local derivatives, backward-propagated sensitivities and the sum of all uses of a shared parameter. Work through the three scalar updates before interpreting an autograd call as a derivative of a fracture calculation.

Backpropagation through a stepped update

This algebraic recurrence makes the bookkeeping behind reverse-mode automatic differentiation visible. For a complementary visual introduction to reverse-mode differentiation in physics, see the [Physics-Based Deep Learning discussion of differentiable physics](#). The worked example and diagram below were developed for this course.

Start from a sequence of update maps

Let a state evolve through N discrete updates,

$$z_{n+1} = S_n(z_n, p), \quad n = 0, \dots, N-1,$$

where the same parameter vector p may appear in every S_n . A final field or state is observed by $y = Cz_N$, and a scalar loss is formed as

$$J = \ell(y, p) = \ell(Cz_N, p).$$

Here C is assumed fixed with respect to p . If an observation operator also depends on p , its derivative belongs in the direct parameter dependence of the final observable/loss.

Define the local state and parameter derivatives

$$A_n = \frac{\partial S_n}{\partial z_n}, \quad B_n = \frac{\partial S_n}{\partial p}.$$

Define $\lambda_n = \partial J / \partial z_n$. This vector, called the adjoint, measures how a small change in each component of the state at step n affects the final loss. At the final step,

$$\lambda_N = C^T \nabla_y \ell,$$

and reverse accumulation is

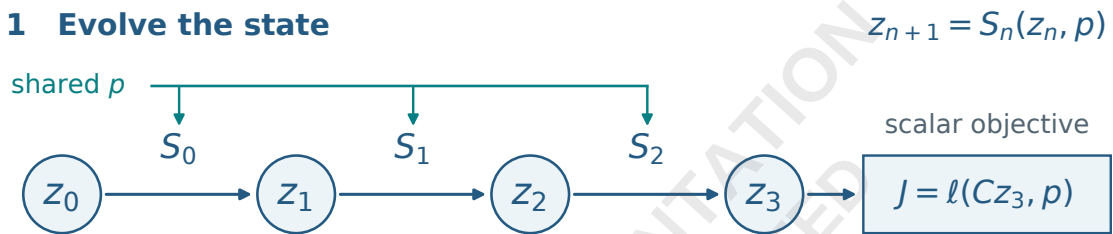
$$\lambda_n = A_n^T \lambda_{n+1} \quad (n = N-1, \dots, 0).$$

The resulting total derivative is

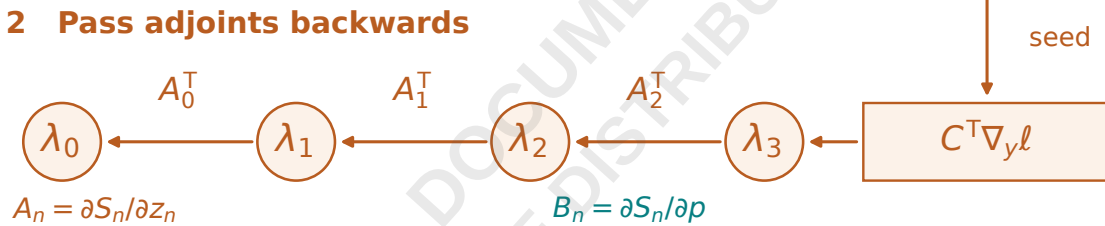
$$\frac{dJ}{dp} = \frac{\partial \ell}{\partial p} + \sum_{n=0}^{N-1} B_n^T \lambda_{n+1} + \left(\frac{\partial z_0}{\partial p}\right)^T \lambda_0.$$

Backpropagation through three updates

1 Evolve the state



2 Pass adjoints backwards



3 Add every parameter contribution

from updates

$$\frac{dJ}{dp} = \underbrace{\frac{\partial \ell}{\partial p}}_{\text{direct loss}} + \underbrace{\sum_{n=0}^2 B_n^T \lambda_{n+1}}_{\text{all three updates}} + \underbrace{\left(\frac{\partial z_0}{\partial p}\right)^T \lambda_0}_{\text{initial state}}$$

The gradient is an input to a separate optimisation step.

Fig. 40: A parameter can affect the final loss at several updates. Reverse mode carries the sensitivity of the loss backwards through those updates and adds the contribution from each use of the parameter.

This is dJ/dp , the gradient of the final scalar observable/loss with respect to the input parameter p . An optimisation algorithm uses that gradient in a *separate* choice such as

$$p_{\text{new}} = p - \alpha \frac{dJ}{dp},$$

where the learning rate α and any line search, regularisation, or constraint handling specify the optimisation method that uses the accumulated gradient.

If z_N is a field, λ_N has one entry per field degree of freedom. For a scalar loss, the loss fixes the seed. For a vector-valued output, choose a seed v to obtain a vector–Jacobian product $A_n^\top v$. These products can be evaluated directly as operator actions. This is why a scalar `loss.backward()` can be economical even when the state is large.

A three-step scalar calculation that can be checked by hand

Use the explicit recurrence

$$x_{n+1} = (1 - \Delta t p)x_n + \Delta t f, \quad J = \frac{1}{2}(x_3 - y_\star)^2.$$

This scalar relaxation model isolates the shared-parameter sum: the parameter p appears at every step. For this scalar homogeneous relaxation, choosing $p > 0$ and $0 < \Delta t p < 1$ makes $0 < 1 - \Delta t p < 1$. This condition describes decay of the homogeneous part of this recurrence; a spatial time integrator requires its own stability analysis.

Take

$$\Delta t = 0.2, \quad p = 0.8, \quad f = 1.5, \quad x_0 = 0.4, \quad y_\star = 0.9.$$

Then $A_n = \partial x_{n+1}/\partial x_n = 1 - \Delta t p = 0.84$ and $B_n = \partial x_{n+1}/\partial p = -\Delta t x_n$. The forward states and local parameter derivatives are:

step n	x_n	x_{n+1}	$B_n = -\Delta t x_n$
0	0.400000	0.636000	-0.0800000
1	0.636000	0.834240	-0.1272000
2	0.834240	1.000762	-0.1668480

The terminal seed is $\lambda_3 = x_3 - y_\star = 0.1007616$. Therefore

$$\lambda_2 = 0.084639744, \quad \lambda_1 = 0.07109738496, \quad \lambda_0 = 0.0597218033664.$$

The corresponding reverse contributions are:

step n	λ_{n+1}	$B_n \lambda_{n+1}$
0	0.07109738	-0.005687791
1	0.08463974	-0.010766175
2	0.10076160	-0.016811871

The displayed loss depends on p through the evolved state, and x_0 is fixed. Its direct-loss and initial-state derivative terms are therefore zero. Adding the three entries in the last column gives

$$\frac{dJ}{dp} = -0.0332658376704.$$

A parameter-dependent observation/loss or parameter-dependent initial state contributes through the first or last term of the total-derivative formula.

The accompanying script independently evaluates the same recurrence with PyTorch and a central finite difference. Its retained execution record lists

$$\begin{aligned} \text{reverse accumulation} &= -0.0332658376704, \\ \text{PyTorch autograd} &= -0.0332658376704, \\ \text{central difference } (h = 10^{-6}) &= -0.0332658376618. \end{aligned}$$

The small finite-difference discrepancy arises from truncation and floating-point roundoff. Reproduce the calculation with:

```
python source/book/scripts/build_backprop_example.py
```

Run this command from the course directory. It updates the figure under `source/book/figures/` and writes a new validation record under `reviews/`.

The [machine-readable validation record](#) retains the measured runtime, all forward/reverse values, and the analytic, automatic-differentiation, and central-difference assertions used here.

The comparison checks the derivative of the stated three-step scalar recurrence.

From a scalar recurrence to a spatial field

The optional [time-stepping companion notebook](#) extends the same reasoning to a one-dimensional diffusion field. Predict a forward trajectory, differentiate its final observation, reproduce reverse mode with a visible loop, and compare the gradient with finite differences. Four exercises include worked answers.

[Download the executable notebook](#) and follow its CPU setup instructions. The example uses NumPy, Matplotlib and PyTorch to evolve a scalar diffusion field on a fixed grid, with synthetic observations generated in the notebook.

An AT2 subproblem with fixed history and mechanics

The PhAST source defines an AT2 damage weak form and an adjoint CG backward for selected damage-solve inputs in `vendor/PhAST/src/phast/solvers/damage_solver.py`.

Consider this damage solve with mechanics, the history field H and the length scale ℓ held fixed. Assume spatially constant G_c , omit the mesh correction γ , and restrict the residual to unconstrained damage degrees of freedom. The reduced problem can then be written as

$$R_d(d; G_c, H) = \left[\frac{G_c}{\ell} M + G_c \ell K + 2M_H \right] d - 2b_H = 0,$$

where M and K are mass and stiffness assemblies, while M_H and b_H collect the history-weighted assembly. With H , mechanics, and ℓ held fixed, let

$$A = \frac{\partial R_d}{\partial d}, \quad A^\top \lambda = \frac{\partial J}{\partial d}.$$

Implicit differentiation of this *reduced* residual gives

$$\frac{dJ}{dG_c} = \frac{\partial J}{\partial G_c} - \lambda^\top \left[\left(\frac{M}{\ell} + \ell K \right) d \right].$$

The direct term accounts for any explicit dependence of the chosen loss on G_c . The minus sign follows from differentiating $R_d = 0$ and solving for dd/dG_c .

In a full load history, mechanics can change H ; an irreversibility update may use a non-smooth maximum; damage bounds can activate; and nonlinear/staggered iterations and stopping criteria matter. Active-set changes, history changes, and convergence failures must be accounted for by the selected differentiation rule. Differentiability at a discrete model change depends on that rule and the local behaviour of the state.

Check your understanding

1. In the scalar example, how do the three $B_n \lambda_{n+1}$ terms account for the repeated use of the scalar parameter p ?
2. Suppose the initial state depends on the parameter through $x_0 = x_0(p)$. Which term in the total derivative changes, and why does it use λ_0 after propagation from the terminal seed λ_3 ?
3. Which assumptions must remain fixed before the displayed AT2 G_c identity can be treated as a reduced-subproblem calculation?

i Short answers

1. The single parameter is used once in each of the three update maps. Each local use affects the final loss through the remaining steps, so reverse mode adds one local parameter contribution for each use.
2. The initial-state term $(\partial z_0 / \partial p)^\top \lambda_0$ becomes nonzero. The adjoint at step zero contains the sensitivity of the final loss to a perturbation of the initial state after every later update has been traversed backwards.
3. The mechanics-dependent history field H , the mechanics state, and ℓ are held fixed. A total derivative through a fracture load path additionally requires the dependencies of history maxima, bounds/active sets, and the converged staggered states.

A bounded parameterisation

An unconstrained update can turn a positive material multiplier negative. One simple way to enforce a scalar interval is to optimise an unconstrained variable θ and map it to

$$p(\theta) = p_{\min} + (p_{\max} - p_{\min}) s(\theta), \quad s(\theta) = \frac{1}{1 + \exp(-\theta)}.$$

The chain rule gives

$$\frac{dJ}{d\theta} = \frac{dJ}{dp} (p_{\max} - p_{\min}) s(\theta)(1 - s(\theta)).$$

This construction enforces bounds but introduces saturation near the limits: the sigmoid derivative becomes small. The choice of transform therefore affects the conditioning of the optimisation problem.

A minimal inverse experiment

For a first experiment, choose one parameter, one geometry, one loading protocol, and one stated observable. For example:

1. define a reference parameter p_{ref} within a known interval;
2. compute a synthetic observation $y^\star = y(p_{\text{ref}})$ using a documented forward map;
3. start from a different θ and map it to $p(\theta)$;
4. compute $J(\theta)$ and a gradient;
5. take a bounded update; and
6. report the parameter trajectory, loss, observable mismatch, and a derivative check.

This controlled recovery uses synthetic observations from the same forward model and specified noise assumptions. Experimental recovery additionally requires assessment of measurement noise and model discrepancy.

Hands-On Lab: Differentiating Mechanics and Parameter Recovery

In the accompanying computational lesson, you will implement a complete differentiable mechanics problem in PyTorch:

1. Forward Solution: Solve deformation in a one-dimensional elastic bar under tensile loading.
2. Sensitivity: Use automatic differentiation (`loss.backward()`) through PhAST and interpret the sign using the analytical extension formula. Finite differences and Taylor tests are optional checks in the reference material.
3. Inverse Identification: Use SGD with momentum to recover one uniform modulus from one synthetic tip displacement at a fixed 4000 N force. Inspect the modulus, loss and displacement histories.

i Hands-On Tutorial: Lab 2 (Gradients and Parameter Recovery)

In [classroom/02 gradients and recovery](#), solve the elastic bar and follow the modulus estimate through successive optimisation steps.

Directional Derivative Verification

Compare the autograd directional derivative with a central finite difference:

$$D_h J(p; q) = \frac{J(p + hq) - J(p - hq)}{2h} \approx g^\top q.$$

Evaluate the comparison across several perturbation sizes h . The following scaled discrepancy measures agreement:

$$\text{Scaled discrepancy} = \frac{|D_h J - g^\top q|}{\max(1, |D_h J|, |g^\top q|)}.$$

Large perturbations introduce truncation error; very small perturbations can amplify cancellation and solver-tolerance effects. Useful step sizes depend on the parameter scale, precision and forward calculation. Apply the displayed scaling to nondimensional quantities, or choose a reference scale with the derivative's units.

i Interpreting Gradient Verification

Agreement over a suitable range of h supports local consistency for the tested parameter and direction. Parameter identifiability and agreement with experimental behaviour require separate assessment.

Why inverse recovery can be difficult

Several different parameter fields can produce similar limited observations. This non-uniqueness can arise from insufficient loading diversity, a coarse observable, parameter correlation, regularisation choices, or weak sensitivity of the forward response. A decreasing loss should therefore be reported with the parameter path, held-out or alternative observables where available, and the stated prior/bounds.

For a spatial parameter field $p(x)$, an additional smoothness term might be

$$J_{\text{reg}}(p) = J_{\text{data}}(p) + \frac{\beta}{2} \int_{\Omega} |\nabla p|^2 dx.$$

This selects smoother solutions and changes the inverse problem. The coefficient β controls the balance between fitting observations and penalising spatial variation.

Exercise: interpret a gradient result

An implementation reports a small central-difference discrepancy for one direction q at a parameter p . Which conclusions below are supported?

1. The local scalar derivative is consistent with this finite-difference check.
2. The inverse problem has a unique solution.
3. The phase-field model is calibrated to a real specimen.
4. The automatic-differentiation graph likely connects the chosen parameter to the stated loss along this tested path.

i Solution

Conclusions 1 and 4 follow for the tested step, direction, parameter and forward solve. Conclusions 2 and 3 require additional studies: identifiability requires a study of observations, parameters, and priors; calibration requires suitable experimental comparison and uncertainty treatment.

The computational lesson applies this reasoning to a one-dimensional elastic bar with a positive log-modulus parameterisation and the displayed scalar observable.

Sources for this chapter

The [PyTorch autograd documentation](#) and [JAX automatic-differentiation guide](#) describe their respective differentiation systems. For a finite-element example of implicit gradient computation, see the [JAX-FEM gradient example](#).

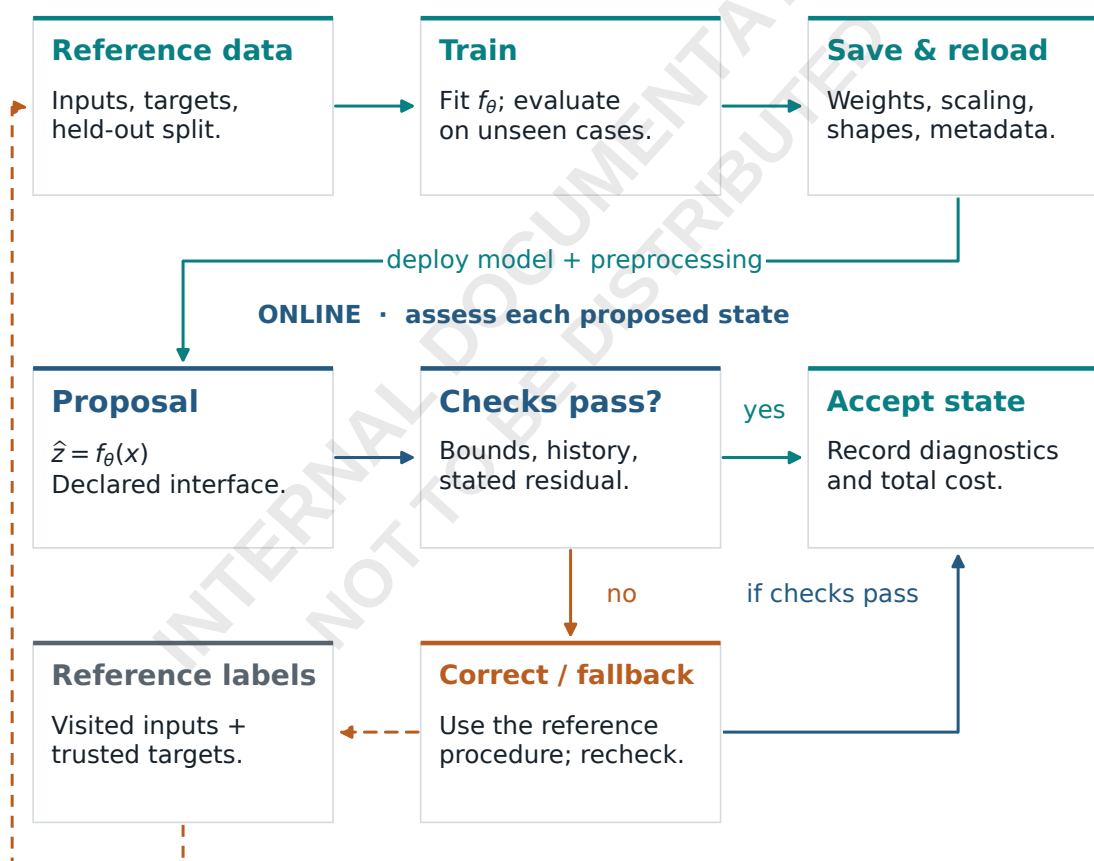
Learning field predictions and combining them with a solver

The Day 3 classroom application is [classroom/03 learning and hybrid](#), which compares a supplied graph network with the classical PhAST damage solve. The Helmholtz examples below remain optional training and interface tutorials.

The correction and fallback strategies described in this reference chapter are possible extensions, not operations performed by NB3. In that practical, the GNN prediction is projected onto the damage bounds and its residual is measured; there is no subsequent FEM damage correction.

Learn a proposal; retain a checked solution

OFFLINE · fit and preserve the model contract



Dashed loop: aggregate reference-labelled visited states and retrain.

A conceptual DAgger extension; notebooks 04-05 demonstrate toy components.

Fig. 41: A learning component has defined inputs, outputs, and physics checks, just as the reference calculation does.

Neural predictions within a mechanics calculation

A hybrid calculation combines a learned field prediction with the equations and constraints of a mechanics model.

Finite elements approximate the governing equations on a mesh. Numerical iterations reduce the discrete residual to a specified tolerance while applying boundary conditions and damage constraints. Repeated solutions during crack growth contribute to the computational cost.

A trained neural network or neural operator maps the current input state to a predicted field. Its usefulness depends on prediction error, computational cost and physical consistency, including the bounds $0 \leq d \leq 1$, irreversibility and mechanical equilibrium.

One approach uses three operations:

1. Field prediction: A neural surrogate supplies a trial displacement or damage field $\hat{d}$.
2. Residual-based assessment: Substituting the field into the discrete equations gives the residual $\|R(\hat{d})\|$. Boundary conditions and damage admissibility are checked alongside this value.
3. Numerical refinement: A prediction satisfying the stated criteria can be used directly. Otherwise, a numerical solve refines the prediction or supplies a reference field. The resulting field is checked again.

This chapter develops the input and output specification, model saving and reloading, and residual assessment. Comparing the complete calculation with the corresponding reference solve establishes its accuracy and runtime.

Decide What is Being Learned

The phrase “use machine learning for fracture” hides several different tasks. Name the role precisely.

- Surrogate: approximate a map from stated inputs to a stated output.
- Proposal model: supply an initial damage field, displacement field, or parameter guess to a mechanics calculation.
- Adapter: convert one data representation to another, such as a mesh ordering or normalised tensor layout.
- Corrector: update a proposal using a residual, an iterative solve, or a trusted reference computation.
- Emulator for an observable: approximate a specified scalar response.

These roles can be combined. A model that predicts an initialisation supports the subsequent mechanics solve; a model intended to replace that solve requires evaluation of the resulting mechanical fields and observables.

Define the supervised-learning problem

Let x_i be an input representation and z_i a reference target. A simple training objective is

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N \|f_{\theta}(x_i) - z_i\|_2^2.$$

Before training, write down:

1. what each channel of x_i means, including coordinates, parameters, fields, and units;
2. what target z_i means and at which load step or time it is sampled;
3. how meshes, nodes, or quadrature points are ordered;
4. the training, validation, and test split rule; and
5. the metric and physical diagnostic reported on each split.

Normalisation is part of this definition. If an input component is transformed as $\tilde{x} = (x - \mu)/s$, save μ and s with the model. The preprocessing state is part of the computational map reconstructed during reload.

Where does the training gradient travel?

Distinguish three computational graphs before choosing a learning method.

Supervised fitting compares a prediction with stored targets:

$$\mathcal{L}_{\text{sup}} = \|f_{\theta}(x) - z_{\text{ref}}\|^2.$$

A solver may have generated the targets offline. This training graph differentiates the predictor against those fixed targets.

Residual-based training evaluates a stated equation at the prediction:

$$\mathcal{L}_{\text{res}} = \|R(f_{\theta}(x); p)\|^2.$$

The gradient passes through the residual evaluated at the prediction. Differentiating a converged state requires the solve's derivative. Boundary conditions and branch selection matter in both cases: a derivative must correspond to the equation and solution branch actually being evaluated.

Solver-in-the-loop training evaluates the consequences of a prediction:

$$z_0 = f_{\theta}(x), \quad z_{n+1} = S(z_n, p), \quad \mathcal{L} = \|Cz_N - y_*\|^2.$$

Here C extracts the observed quantity. With fixed p and a loss depending only on z_N , reverse propagation gives

$$\bar{z}_N = 2C^T(Cz_N - y_*), \quad \bar{z}_n = \left(\frac{\partial S}{\partial z_n}\right)^T \bar{z}_{n+1}, \quad \nabla_{\theta} \mathcal{L} = \left(\frac{\partial f_{\theta}}{\partial \theta}\right)^T \bar{z}_0.$$

These vector–Jacobian products can be evaluated directly as operator actions. An implicit adjoint may replace differentiation through every solver iteration. The [differentiable-physics introduction](#) develops this operator viewpoint. For fracture, history updates, active constraints, stopping tolerances and any detached arrays require explicit derivative conventions and checks.

The following notebooks use supervised training and residual-checked proposals on a scalar Helmholtz teaching problem.

Train, save, and reload reproducibly

A minimal saved model package should include:

- model weights and architecture configuration;
- input and target normalisation parameters;
- feature/channel names and tensor-shape convention;
- training data identifier and split definition;
- software/library version and numerical precision; and
- a small reload test that compares a saved prediction against the in-memory prediction for the same declared input.

Computational lesson: train, save, reload, and compare a toy field model

This optional lesson trains a model for a scalar Helmholtz field, saves it and reloads it into a new model object. Compare the predictions before and after reloading. The example shows why the saved weights must be accompanied by the feature order, normalisation and model configuration. The field calculation uses the course's `ToyHelmholtzProblem`.

i Optional tutorial: Training and Model Reloading

In `labs/04 train save reload adapter`, train the field model and check that saving and reloading preserves its predictions.

i Reload test

Choose one fixed input record. Run the model before saving, reload into a new model object, run the same record, and compare the output after applying the same preprocessing. If the values differ, inspect normalisation, device/dtype, model mode, feature order, and serialised configuration before training again.

Mapping between representations

An adapter is a stated map between representations,

$$a : \mathcal{X}_{\text{source}} \longrightarrow \mathcal{X}_{\text{target}}.$$

For example, it may reorder nodes, concatenate a coordinate channel, project a cell-centred value to nodes, or convert units. The mapping requires enough source information to define the target and compatible physical meanings on both sides.

Ask four questions before accepting an adapter:

- Does it preserve the geometry and coordinate frame?
- Does it preserve the field meaning, units, and load/time index?
- Does it preserve or deliberately change interpolation order?
- Is there a small round-trip or reference example that checks the mapping?

A change of mesh, material parameterisation, or loading regime introduces a transfer assumption that needs its own evaluation.

From a proposal to a checked hybrid calculation

Suppose a learned model proposes $\hat{d}$ for a damage field. A conservative hybrid workflow is

$$\text{input} \longrightarrow \hat{d} \longrightarrow \text{bounds/residual checks} \longrightarrow \text{mechanics correction or fallback} \longrightarrow \text{reported output}.$$

The checks might include damage bounds, irreversibility relative to the prior state, a mechanics residual, and a comparison to a reference solve on a predefined evaluation set. If a correction is used, report the cost and result of the full route, including prediction, conversion, checks, correction, and any fallback. Assess runtime improvement by comparing this complete cost with the corresponding reference calculation.

Computational lesson: accept a compatible proposal or fall back

The next lesson reuses the toy field-model input and output specification to test a compatible proposal, reject a deliberately perturbed proposal using a stated residual criterion, and fall back to the reference field. Read the compatibility and residual checks when interpreting the four field panels. The residual is defined by the scalar `ToyHelmholtzProblem`. Applying this pattern to fracture requires the fracture residual and its damage admissibility conditions.

i Optional tutorial: Physical Assessment and Correction

In [labs/05 hybrid reference correction](#), assess a proposed field using the Helmholtz residual and examine when the reference correction is used.

DAGger-style data aggregation

Distribution shift can appear when a learned proposal visits states absent from its original training records. Dataset Aggregation (often called DAGger) is one way to describe an iterative collection loop:

1. train a policy or proposal model on an initial reference dataset;
2. let the current model visit stated inputs or intermediate states;
3. query a trusted reference procedure for targets at those visited states;
4. add the labelled records to the aggregate dataset; and
5. retrain while keeping the final evaluation data and labels separate from model selection.

This idea is useful when the model's own trajectory changes the data it sees. It relies on a trusted labelling procedure, documented data sources and processing steps, and separate training and final evaluation data. In mechanics, the reference procedure may fail or become inappropriate outside its assumptions; retain those outcomes when evaluating the method.

Evaluation: Physical Metrics and Model Validation

Compare both field error and physical consistency. A small average error can coexist with a misplaced crack or a large local error, so inspect the spatial fields alongside scalar measures.

Useful comparisons include:

1. Field-Level Discrepancy:

$$e_d = \frac{\|\hat{d} - d_{\text{ref}}\|_2}{\max(\|d_{\text{ref}}\|_2, \epsilon)}$$

measures the discrepancy on the evaluated field. Comparing different meshes also requires a consistent spatial weighting or a common evaluation grid.

2. Physical Constraints: Verify that predicted fields respect admissibility conditions (such as $0 \leq d \leq 1$ and non-decreasing damage history $d_n \geq d_{n-1}$).
3. Mechanics Residuals: Substitute the predicted field directly into the governing mechanical weak form. A physically consistent prediction yields low equilibrium residuals.
4. Engineering Observables: Compare integral quantities of direct engineering interest, such as global reaction curves, peak load capacity, and total dissipated fracture energy.
5. Performance on Unseen Cases: Report performance across varying load levels, mesh densities, and unseen geometries to characterize the model's domain of applicability.

Exercise: describe the information needed to reuse a model

A colleague shares a file called "best_model" and says it predicts damage on a new mesh. List four items you need before interpreting the output as a mechanics field.

i Solution

Any four of the following are essential: the architecture and weights; input feature definitions and ordering; normalisation state; source and target mesh representation; coordinate frame and units; load/time step represented; the training/validation/test split; a reload comparison; and the physical checks used

on the proposed field. These records give the predicted tensor its mechanical meaning. The two computational lessons provide a concrete version of this answer: the first records the model's input conventions and reload check; the second refuses an incompatible feature order and performs a stated residual/fallback check.

Sources for this chapter

The notebook practices draw on standard reproducible machine-learning ideas: separate evaluation data, explicit preprocessing, and persistent model state. For a practical notebook pattern with open exercises and solutions, see the [Inria scikit-learn MOOC](#). For the tensor/autodiff ecosystem used in the companion activities, see the [PyTorch documentation](#).

INTERNAL DOCUMENTATION
NOT TO BE DISTRIBUTED

Practice, Solutions, Glossary, and References

Connecting the four topics

The course connects fracture modelling, numerical solution, differentiation and learning. Use the following topics to review those connections.

1. Phase-field fracture Explain how Griffith's surface energy is approximated by a volume integral over a continuous scalar damage field $d(x)$, governed by the regularisation length ℓ , the crack-density function $w(d)$, and the stiffness degradation law $g(d)$.
2. Simulation with PhAST Trace the geometry, mesh, boundary conditions, mechanical update and damage solve, and explain the resulting fields and energy histories.
3. Differentiation and recovery Explain how the chain rule produces a parameter gradient, compare it with a finite-difference check, and use it to estimate an unknown parameter.
4. Learning within a mechanics solver Explain the role of a learned prediction, assess its residuals and constraints, and compare the complete hybrid calculation with its numerical reference.

Best Practices for Computational Exploration

When experimenting with computational mechanics models in Jupyter or Colab:

- Vary One Parameter at a Time: For example, change ℓ and examine the damage-band width, or change G_c and compare the resulting peak load. Keep the other model inputs fixed and check that the mesh resolves each case.
- Examine Both Fields and Curves: Inspect the full spatial damage and stress fields alongside global load-displacement curves.
- Verify Gradients Early: When writing custom differentiable loss functions, verify gradients against finite differences before launching long optimization runs.
- Check Physical Bounds: Ensure the damage field respects $0 \leq d \leq 1$ and non-decreasing history $d_n \geq d_{n-1}$.

Three classroom notebooks

Practical	Focus
Lab 1 — Simulate fracture	Geometry, supports, loading, PhAST results and their interpretation.
Lab 2 — Gradients and recovery	One force, one tip observation and Young's-modulus recovery through PhAST.
NB3 — Hybrid FEM+DL with PhAST	Direct graph-network damage replacement on a three-hole plate, compared with conventional FEM.

Each page includes Google Colab, notebook downloads and a conceptual answer. The learned lab downloads the supplied source and compatible frozen checkpoint from the PhAST GNN assets ZIP and verifies their hashes. Visit [further practice](#) for the original individual notebooks and longer implementation walkthroughs.

Continue with the PhAST documentation

The [PhAST tutorial sequence](#) extends the same workflow used here: define a specimen, inspect mesh regions, apply supports and loading, solve, and interpret the saved fields.

Activity	Physical question	Next reading
Course Lab 01	How do a graded mesh and imposed separation affect dynamic crack growth?	Inspect displacement, strain, stress and damage animations in the square-plate notebook.
PhAST SENT setup	How do a Gmsh geometry and named regions become a configured problem?	Step-by-step problem setup
B3 dynamic SENT results	How does a damaged band extend across a tensile specimen during a dynamic calculation?	Public B3 example and retained animation

B3 uses a 40 mm square with a 20 mm notch and explicit dynamics. Its retained animation provides a qualitative crack-growth example. When comparing it with Lab 01, identify the geometry, units, loading history and inertial terms in each model. For a fresh B3 calculation, review its configuration and mesh resolution and measure the complete runtime in your environment.

Worked consolidation exercise

Consider a two-dimensional notched specimen represented by displacement u and damage d . The calculation uses a length scale ℓ , a residual stiffness η_{res} , and a displacement-controlled load increment.

1. Identify the local damage term, the gradient term and the energy prefactor in the fracture contribution.
2. Write the qualitative order of a staggered update.
3. Name two visual outputs and one scalar output you would retain.
4. A learned model proposes d . Name two checks required before using it as a starting point for a mechanics update.
5. State a conclusion supported by the exercise and identify an additional study needed to apply it more broadly.

i Solution

1. The fracture part uses a local crack-density term $w(d)/\ell$, a gradient term $\ell|\nabla d|^2$, and the fracture energy scale G_c/c_0 .
2. Start from accepted prior damage; solve mechanics with damage fixed; update the stated tensile driving quantity; solve damage subject to irreversibility; then check and accept or repeat.
3. Retain the geometry/mesh with loading direction, a damage field with a colour scale, and a displacement/deformed-field view. A suitable scalar is reaction force, an energy contribution, or a stated residual.
4. Check the damage bounds/irreversibility and a stated mechanics residual or correction step. The precise checks depend on the proposal's intended use.
5. The result describes the defined discretisation and load path. Material calibration requires experimental comparison, mesh independence requires refinement, and acceleration requires matched measurements of the full computational cost.

Glossary

Adapter

A declared map that changes a data representation, for example node order, units, or tensor layout, with physical meaning checked across the mapping.

Automatic differentiation (AD)

A method for evaluating derivatives of a program by applying the chain rule to its recorded operations. Its result concerns the executed computation.

Cohesive-zone model (CZM)

A fracture/interface formulation based on a traction–separation law.

Crack-density function $w(d)$

The local part of a phase-field fracture regularisation. AT1 and AT2 use different choices.

Damage field d

A scalar field used here with $d = 0$ intact and $d = 1$ fully broken.

Dagger-style aggregation

Iterative collection of reference labels at states visited by the current proposal/policy, followed by re-training on the aggregate data.

Degradation function $g(d)$

A function that reduces selected elastic energy as damage grows.

Finite difference

A derivative approximation from nearby function evaluations; useful as an independent local check.

Irreversibility

The condition that brittle damage is nondecreasing over the stated loading history.

Matrix-free action

An implementation of $v \mapsto Kv$ or a Jacobian action using local operator evaluations and accumulation into global degrees of freedom.

Phase-field fracture

A regularised fracture formulation in which a diffuse scalar field approximates a sharp crack surface.

Quadrature

Numerical integration on an element using selected points and weights.

Quasi-Newton method

A nonlinear optimisation/solution strategy that updates an approximate Jacobian or inverse Jacobian for a chosen system of equations.

Residual

The imbalance in a stated discrete equation. Interpret a residual norm using its definition, scaling, tolerance, and field context.

XFEM

Extended finite elements: an enriched finite-element approximation for discontinuities or near-tip features.

Reference trail

Further interactive study

[Dive into Deep Learning](#) combines exposition, mathematics, computational examples and exercises in one reading sequence. For a physics-oriented progression, [Advanced Deep Learning for Physics](#) connects numerical simulation, sensitivity analysis and learning through longer assignments. These resources informed the teaching format; the exercises and worked answers in this book were written specifically for the PhAST course.

Scientific references

The original diagrams also draw visual-design inspiration from the [TU Delft surrogate-model explanation](#), [TUM's differentiable-physics lecture](#) and the [ETH Zurich SPCL DaCeML research presentation](#). These sources illustrate labelled physical interfaces, aligned forward and reverse paths, and overview-to-detail explanations. The original course diagrams adapt these design principles to the notation and models used here.

The following sources support the formulation boundaries and teaching conventions used here.

- A. A. Griffith, “The phenomena of rupture and flow in solids,” *Philosophical Transactions of the Royal Society A* 221 (1921), DOI: [10.1098/rsta.1921.0006](#).
- G. A. Francfort and J.-J. Marigo, “Revisiting brittle fracture as an energy minimization problem,” *Journal of the Mechanics and Physics of Solids* 46 (1998), DOI: [10.1016/S0022-5096\(98\)00034-9](#).
- B. Bourdin, G. A. Francfort, and J.-J. Marigo, “Numerical experiments in revisited brittle fracture,” *Journal of the Mechanics and Physics of Solids* 48 (2000), DOI: [10.1016/S0022-5096\(99\)00028-9](#).
- C. Miehe, F. Welschinger, and M. Hofacker, “Thermodynamically consistent phase-field models of fracture: Variational principles and multi-field FE implementations,” *International Journal for Numerical Methods in Engineering* 83 (2010), DOI: [10.1002/nme.2861](#).
- K. Pham, H. Amor, J.-J. Marigo, and C. Maurini, “Gradient damage models and their use to approximate brittle fracture,” *International Journal of Damage Mechanics* 20 (2011), DOI: [10.1177/1056789510386852](#).
- N. Moës, J. Dolbow, and T. Belytschko, “A finite element method for crack growth without remeshing,” *International Journal for Numerical Methods in Engineering* 46 (1999), DOI: [10.1002/\(SICI\)1097-0207\(19990910\)46:1%3C131::AID-NME726%3E3.0.CO;2-J](#).
- X.-P. Xu and A. Needleman, “Numerical simulations of fast crack growth in brittle solids,” *Journal of the Mechanics and Physics of Solids* 42 (1994), DOI: [10.1016/0022-5096\(94\)90003-5](#).
- P. K. Kristensen, C. F. Niordson, and E. Martínez-Pañeda, “An assessment of phase field fracture: crack initiation and growth,” *Philosophical Transactions of the Royal Society A* 379 (2021), DOI: [10.1098/rsta.2021.0021](#).
- PyTorch contributors, [Autograd documentation](#).
- JAX contributors, [Automatic differentiation guide](#).
- JAX-FEM contributors, [Quickstart and examples](#).